GCP Professional Cloud Architect: Day 2 Training Guide

VPC Networking, Firewall Rules and Compute Engine (Deep Dive + Labs + Exam Scenarios)

40-hour program | Day 2 | 8 labs | 20 exam scenarios

Course: Google Cloud Professional Cloud Architect (40-hour program)

Session: Day 2

Format: Concepts, live demonstrations (labs), then exam-style scenarios

Structure: Each module has core concepts followed by numbered **Deep dive** sections (2.9–2.15, 3.5–3.9, 4.10–4.20, 5.4–5.8). The labs in Section 6 are unchanged.

Note to trainer: Product limits, machine series, and defaults change over time. Verify any number marked with a ⚠️ against the current Google Cloud documentation before the session.

Contents

1. Session Overview and Agenda

- 1.1 Why this day matters for the exam
- 1.2 Learning objectives
- 1.3 Suggested agenda (about 8 hours; adjust to your actual schedule)
- 1.4 Prerequisites and lab setup

2. Module 1: VPC, Firewalls and Networking Basics

- 2.1 What is a VPC in Google Cloud?
- 2.2 VPC network modes
- 2.3 Subnets in depth
- 2.4 IP addressing
- 2.5 Routing
- 2.6 Connectivity concepts you must recognize (overview only today)
- 2.7 Firewalls in depth
- 2.8 Diagnosing network problems (tools to know)
- 2.9 Deep dive: how Google Cloud networking actually works
- 2.10 Deep dive: IP addressing, alias IPs and DNS
- 2.11 Deep dive: routing
- 2.12 Deep dive: Private Google Access, Private Service Connect and API reachability
- 2.13 Deep dive: Cloud NAT, egress costs, and Network Service Tiers
- 2.14 Deep dive: Shared VPC, VPC Peering, and choosing a connectivity model
- 2.15 Deep dive: firewall engine, patterns and defense in depth

3. Module 2: Implementing VPC Networks and Firewall Rules

- 3.1 VPC design methodology (use this for exam answers too)
- 3.2 Reference architecture used in the labs
- 3.3 Firewall implementation checklist
- 3.4 Common implementation pitfalls
- 3.5 Deep dive: CIDR math and IP address planning
- 3.6 Deep dive: reference network architectures
- 3.7 Deep dive: governing networks with Organization Policy and IAM
- 3.8 Deep dive: Infrastructure as Code for networks (Terraform)
- 3.9 Deep dive: operating firewalls over time

4. Module 3: Compute Engine Instances and Customization

- 4.1 What Compute Engine is
- 4.2 Resource location model
- 4.3 Machine families, series, and types
- 4.4 Provisioning models and pricing
- 4.5 Boot disks, images, and storage options
- 4.6 VM lifecycle and availability
- 4.7 Customizing a VM
- 4.8 Instance templates and instance groups
- 4.9 Cost optimization summary for Compute Engine
- 4.10 Deep dive: Compute Engine architecture under the hood
- 4.11 Deep dive: machine series comparison
- 4.12 Deep dive: disks and storage performance
- 4.13 Deep dive: the guest environment, metadata and boot sequence
- 4.14 Deep dive: identity, access and security of VMs
- 4.15 Deep dive: managed instance groups (MIGs)
- 4.16 Deep dive: availability, SLAs and resilience patterns on Compute Engine
- 4.17 Deep dive: cost optimization, discounts and worked comparisons
- 4.18 Deep dive: choosing Compute Engine versus other compute options
- 4.19 Deep dive: migrating VMs to Compute Engine
- 4.20 Deep dive: operating and troubleshooting VMs

5. Module 4: Creating and Customizing VM Instances (Hands-on)

- 5.1 Creating a VM in the Console: field-by-field walkthrough
- 5.2 Customization levels and when to use each
- 5.3 Day-2 operations on a running VM
- 5.4 Deep dive: a VM design worksheet (use for every workload)
- 5.5 Deep dive: configuration as code for VMs and MIGs (Terraform)
- 5.6 Deep dive: building golden images in a pipeline
- 5.7 Deep dive: customization patterns and when to use each
- 5.8 Deep dive: mapping Console choices to gcloud and Terraform

6. Hands-on Labs (Demonstration Track)

- Lab 1: Create a custom-mode VPC with subnets (≈ 20 min)
- Lab 2: Create service accounts and firewall rules (≈ 30 min)
- Lab 3: Create tiered VMs and test the firewall model (≈ 30 min)
- Lab 4: Cloud NAT and Private Google Access (≈ 20 min)
- Lab 5: Customize a VM: custom machine type, disks, metadata, Spot (≈ 30 min)
- Lab 6: Images, snapshots, and snapshot schedules (≈ 25 min)
- Lab 7: Instance template and Managed Instance Group with autoscaling and autohealing (≈ 40 min)
- Lab 8: Troubleshooting drill (≈ 20 min)
- Cleanup (run at the end of the day)

7. Quick Reference Cheat Sheets

- 7.1 VPC and networking at a glance
- 7.2 Firewall at a glance
- 7.3 Compute Engine at a glance
- 7.4 Decision table: which compute and purchase option?

8. Exam-Style Scenarios

- Scenario 1: Network mode selection
- Scenario 2: Firewall priority resolution
- Scenario 3: Securing three tiers
- Scenario 4: Private VMs with no public IPs
- Scenario 5: Managed instance group keeps recreating VMs
- Scenario 6: Cost: batch processing
- Scenario 7: Right-sizing odd requirements
- Scenario 8: Change a VM's capacity with minimal disruption
- Scenario 9: Stateful database and high availability
- Scenario 10: Zero-downtime rollout
- Scenario 11: Org-wide guardrail
- Scenario 12: Hybrid connectivity and routes
- Scenario 13: Compliance and licensing
- Scenario 14: Access control for administrators
- Scenario 15: Troubleshooting
- Scenario 16: Image strategy
- Scenario 17: Case-study style (fictional retailer)
- Scenario 18: Subnet exhaustion
- Scenario 19: Logging blocked traffic
- Scenario 20: Multi-step reasoning (long-form style)
- Bonus: Fast-fire true/false round (instructor-led, 5 minutes)

9. Common Exam Traps and Keyword Map

- 9.1 Keyword → likely answer
- 9.2 Top traps

10. Recap, Homework and Day 3 Preview

- 10.1 Recap: five things to remember from Day 2
- 10.2 Self-check questions (answer without notes)
- 10.3 Homework (60–90 minutes)
- 10.4 Day 3 preview (adjust to your agenda)
- 10.5 Suggested resources

Table of Contents

1. Session Overview and Agenda
2. Module 1: VPC, Firewalls and Networking Basics
3. Module 2: Implementing VPC Networks and Firewall Rules
4. Module 3: Compute Engine Instances and Customization
5. Module 4: Creating and Customizing VM Instances (Hands-on)
6. Hands-on Labs (Demonstration Track)
7. Quick Reference Cheat Sheets
8. Exam-Style Scenarios
9. Common Exam Traps and Keyword Map
10. Recap, Homework and Day 3 Preview

1. Session Overview and Agenda

1.1 Why this day matters for the exam

Networking and compute decisions appear in almost every Professional Cloud Architect (PCA) question domain:

Exam domain	Where today's content appears
Designing and planning a cloud solution architecture	Choosing compute type, sizing, regional vs. zonal design, IP planning
Managing and provisioning a solution infrastructure	VPC configuration, firewall rules, VM provisioning, instance groups
Designing for security and compliance	Firewall design, private VMs, service accounts, Shielded VMs, OS Login
Analyzing and optimizing technical and business processes	Spot VMs, committed use discounts, rightsizing
Ensuring solution and operations excellence	Health checks, autohealing, autoscaling, snapshots, monitoring agents

1.2 Learning objectives

By the end of the session, a participant can:

- Explain what a VPC is in Google Cloud and how it differs from VPCs in other clouds (global scope, regional subnets).
- Choose between auto mode and custom mode networks and justify the choice.
- Design a non-overlapping IP plan, including secondary ranges.
- Explain how routes are selected and how traffic leaves and enters a VPC.
- Write correct firewall rules (direction, priority, targets, sources, protocols) and predict the result of rule conflicts.
- Choose between network tags and service accounts as firewall targets.
- Pick the right machine family, machine type, disk type, and provisioning model for a given workload.
- Create, customize, image, snapshot, and template VMs using Console and `gcloud`.
- Build a managed instance group (MIG) with autoscaling and autohealing.
- Answer scenario-based exam questions on all of the above.

1.3 Suggested agenda (about 8 hours; adjust to your actual schedule)

Time	Block	Type
0:00 – 0:20	Day 1 recap, Day 2 objectives, lab environment check	Discussion

Time	Block	Type
0:20 – 1:40	Module 1: VPC fundamentals (modes, subnets, IPs, routes)	Concepts
1:40 – 2:30	Module 1: Firewall rules and policies	Concepts
2:30 – 2:45	Break	
2:45 – 4:00	Module 2: Implementing VPCs and firewall rules	Labs 1 – 3
4:00 – 4:45	Lunch	
4:45 – 6:00	Module 3: Compute Engine deep dive	Concepts
6:00 – 7:15	Module 4: Creating and customizing VMs, templates, MIGs	Labs 4 – 8
7:15 – 7:45	Exam-style scenarios (discussion)	Practice
7:45 – 8:00	Recap, homework, Q&A	

1.4 Prerequisites and lab setup

- A Google Cloud project with billing enabled (a training or sandbox project is recommended).
- Owner or Editor permissions on the project (for the lab only; the exam will always push least privilege).
- Cloud Shell (no local setup needed) or a local `gcloud` installation.

Run these once in Cloud Shell:

```
# Set variables used by all labs
export PROJECT_ID=$(gcloud config get-value project)
export REGION=us-central1
export ZONE=us-central1-a
export REGION2=europe-west1
export ZONE2=europe-west1-b

gcloud config set compute/region $REGION
gcloud config set compute/zone $ZONE

# Enable the APIs used today
gcloud services enable \
  compute.googleapis.com \
  iap.googleapis.com \
  networkmanagement.googleapis.com \
  logging.googleapis.com \
  monitoring.googleapis.com
```

> **Cost note:** All labs use small machine types (`e2-small` , `e2-medium`) and the cleanup section at the end of Section 6 removes every resource. Total cost for the day is typically a few dollars at most if cleaned up the same day. ⚠️ Verify with the pricing calculator.

---

2. Module 1: VPC, Firewalls and Networking Basics

2.1 What is a VPC in Google Cloud?

A **Virtual Private Cloud (VPC)** network is a global, logically isolated, software-defined virtual network that provides connectivity for your resources (VMs, GKE nodes, serverless connectors, and so on). It is implemented in Google's production network by the Andromeda network virtualization stack, not by physical appliances you manage.

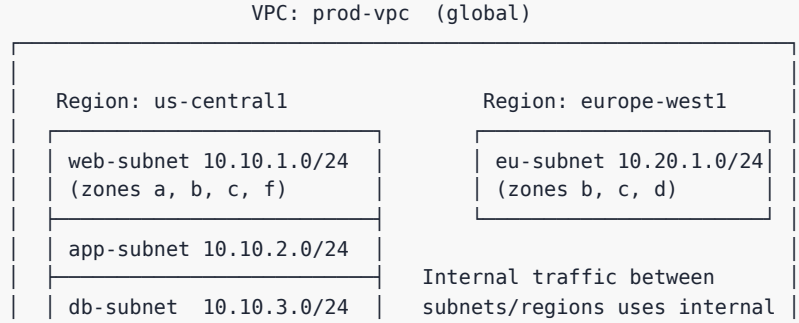
Key characteristics (memorize these, they are exam favorites):

Property	Behavior in Google Cloud
Scope of a VPC network	**Global** resource. One VPC can span all regions.
Scope of a subnet	**Regional** resource. A subnet spans all zones in its region.
IP range on the VPC itself	**None.** IP ranges are defined on subnets only.
Cross-region communication inside a VPC	Works over Google's private backbone by internal IPs, **no VPN or peering needed**, as long as firewall rules allow it.
Routing between subnets in the same VPC	**Automatic** (system-generated subnet routes).
Firewall	Distributed and stateful, enforced at the VM's virtual NIC. **No central firewall appliance bottleneck.**
Ownership	A VPC belongs to a **project**. A project can have multiple VPCs (default quota is limited per project).
Broadcast / multicast	Not supported at the VPC layer.

Contrast with other clouds (useful when participants have AWS or Azure background):

- AWS VPCs are regional; Google VPCs are global.
- AWS security groups attach to instances; Google firewall rules are defined at the VPC level and target instances via tags or service accounts.
- AWS subnets are zonal; Google subnets are regional.

Diagram: Global VPC with regional subnets



```
|  └──────────────────────────┘   IPs over Google's backbone    |
└──────────────────────────────────────────────────────────────┘
```

2.2 VPC network modes

Mode	How subnets are created	Typical use
Auto mode	One subnet **automatically created in every region**, using predefined ranges from `10.128.0.0/9` (each subnet is a `/20`). New regions get a subnet automatically.	Quick starts, learning, proofs of concept
Custom mode	**You** create each subnet and choose the range. No automatic subnets.	**Production.** Required when you need controlled IP plans, hybrid connectivity, or peering.
Legacy	A single global IP range, no subnets	**Deprecated.** Do not recommend.

The default network

- Every new project (unless an organization policy skips it) gets a network named `default`, an **auto mode** VPC.
- It comes with pre-populated firewall rules: `default-allow-internal`, `default-allow-ssh`, `default-allow-rdp`, `default-allow-icmp`. They are all **priority 65534** and are ordinary rules you can edit or delete.
- ⚠️ **Exam stance:** The default network is convenient but **not recommended for production** because its ranges are predetermined, may overlap with on-prem or other VPCs, and the default rules are permissive (SSH and RDP from anywhere).

Auto mode limitations (why they get replaced):

1. All auto mode networks use the **same** `10.128.0.0/9` space, so **two auto mode VPCs cannot be peered** and an auto mode VPC will likely overlap with on-prem or other networks over VPN.
2. The `/20` per-region size cannot match your sizing needs. You can expand it, but you do not control the base range.
3. Subnets exist in regions you may not use, which is poor for governance.

Converting modes

- Auto → Custom is **supported and one-way**. It cannot be reversed.
- You can convert at any time with no downtime.

```
gcloud compute networks update default --switch-to-custom-subnet-mode
```

2.3 Subnets in depth

A **subnet** is a regional IP range inside a VPC. VMs get their **primary internal IP** from the subnet's **primary range**.

2.3.1 Primary and secondary ranges

- **Primary range:** Used for VM NICs' internal IPs and internal load balancer IPs.

- **Secondary ranges:** Used for **alias IPs**. These are essential for **GKE VPC-native clusters**, where one secondary range is used for **Pods** and another for **Services**.
- A subnet can have **multiple secondary ranges** (⚠️ check the current quota).

2.3.2 Reserved addresses in the primary range

For every primary IPv4 subnet range, Google reserves **four** addresses:

Reserved address	Purpose
First address (network address)	Network
Second address	Default gateway
Second-to-last address	Reserved by Google for future use
Last address (broadcast)	Broadcast

Example for `10.10.1.0/24` (256 addresses):

- `10.10.1.0` network
- `10.10.1.1` gateway
- `10.10.1.254` reserved
- `10.10.1.255` broadcast
- Usable: **252**

> **Exam hint:** If a question asks for the number of usable addresses, subtract **4** from the total, not 2.

2.3.3 Valid ranges and sizing

- Valid ranges generally include **RFC 1918** (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`), plus other supported ranges such as **RFC 6598** (`100.64.0.0/10`) and certain privately used public ranges. ⚠️ Check the current list of valid and restricted ranges.
- Smallest primary IPv4 subnet is **/29**; largest is **/4**. ⚠️ Confirm limits in docs.
- Subnet ranges **within one VPC cannot overlap** each other.
- Subnet ranges **cannot overlap** with ranges of peered VPCs or with on-prem networks that you connect by VPN or Interconnect, **if you want routing to work correctly**.

2.3.4 Expanding a subnet

- You can **expand** a primary range (for example, `/24` → `/20`) with **no downtime** and no VM restart.
- You can **never shrink** a range.
- The expanded range must not overlap other subnets or peering ranges.

```
gcloud compute networks subnets expand-ip-range web-subnet \
  --region=us-central1 --prefix-length=22
```

> **Planning rule:** Start with generous ranges. Expansion is easy; renumbering is painful.

2.3.5 Subnet-level features

Feature	What it does	Notes
Private Google Access (PGA)	Lets VMs with **only internal IPs** reach Google APIs and services (Cloud Storage, BigQuery, etc.)	Enabled **per subnet**. No NAT or external IP required.
VPC Flow Logs	Samples network flows for monitoring, forensics, and cost analysis	Enabled **per subnet**. Adjustable sampling and aggregation interval. Logs go to Cloud Logging and can be exported.
Purpose = proxy-only	Reserved for managed proxy-based load balancers (Envoy-based, e.g., regional external or internal Application Load Balancers)	Needs its own dedicated subnet per region and VPC.
Private Service Connect subnet	Used for publishing services	Covered later in the course.
IPv6	Dual-stack subnets possible	⚠️ Supported with specifics; mention only if time permits.

2.4 IP addressing

2.4.1 Internal IP addresses

- Assigned from the subnet's primary range.
- **Ephemeral** (default): retained while the VM exists, released when the VM is deleted. They stay the same across stop/start.
- **Static (reserved):** You reserve it, and it persists independently of the VM.
- **Internal DNS:** Every VM is automatically resolvable.
- Zonal DNS (the default for new projects): `VM_NAME.ZONE.c.PROJECT_ID.internal`
- The older **global DNS** form `VM_NAME.c.PROJECT_ID.internal` exists only in older projects (⚠️ it is project-wide, so name collisions across zones are possible).
- Metadata server (also acts as DNS resolver for internal names): **169.254.169.254**.

2.4.2 External IP addresses

Type	Behavior
Ephemeral external IP	Assigned at creation. Released when the VM is stopped or deleted (a new one is assigned on start).
Static external IP	Reserved. Persists. Can be **regional** (for VMs, regional LBs) or **global** (for global external LBs).
No external IP	The VM is private. It can still reach Google APIs via PGA, and the internet via **Cloud NAT**.

> ⚠️ **Billing:** Google charges for external IPv4 addresses, including those attached to running VMs. Static IPs that are reserved but unused cost more. Good practice: use Cloud NAT and load balancers instead of giving every VM a public IP.

2.4.3 Network Service Tiers

Tier	Behavior	Use when
Premium Tier (default)	Traffic travels on Google's global backbone as long as possible and exits near the user. Required for **global** external load balancing.	Latency-sensitive, global users
Standard Tier	Traffic uses the public internet for most of its path. Regional only. Lower cost.	Cost-sensitive, regional, tolerant of variable latency

2.5 Routing

Routing decides where a packet goes next. In Google Cloud, **routes are defined at the VPC level**, but programmed per VM by the SDN control plane.

2.5.1 Types of routes

Type	Created by	Examples
System-generated: Default route	Google, when a VPC is created	`0.0.0.0/0` → **default internet gateway** (priority 1000)
System-generated: Subnet routes	Google, per subnet (and per secondary range)	`10.10.1.0/24` → local VPC
Static custom routes	You	Destination CIDR → next hop (instance, VPN tunnel, internal LB, IP address, internet gateway)
Dynamic custom routes	Cloud Router via BGP	Learned from on-prem through Cloud VPN or Cloud Interconnect
Peering routes	VPC Network Peering	Subnet routes exchanged between peers
Policy-based routes	You	Match on more than destination (source, protocol), covered later

2.5.2 How the next hop is chosen

1. **Longest prefix match** wins (most specific destination).
2. If multiple routes match with the same prefix length, the **lowest priority value** wins (priority is 0–65535, **lower = higher precedence**).
3. If still tied, traffic may be **ECMP-balanced** across the tied routes (for equal priority custom dynamic routes) ⚠️ nuances apply.
4. **Subnet routes always win** over custom routes for the same destination (subnet routes cannot be overridden by a custom route with an identical or less specific range).

> **Important:** A route existing does **not** mean traffic is allowed. A **firewall rule** must also permit it.

2.5.3 Network tags on routes

A static route can apply only to VMs carrying specific **network tags**. This is how you send some VMs through a network virtual appliance (NVA) while others use the default path.

2.5.4 Dynamic routing mode

Set at the VPC (applies to Cloud Routers in it):

Mode	Cloud Router advertises/learns
Regional (default)	Only routes for subnets in the **same region** as the Cloud Router
Global	Routes for **all** regions in the VPC

> **Scenario hint:** Hybrid connection to on-prem in one region, but VMs in other regions also need to reach on-prem → switch to **global dynamic routing**.

2.6 Connectivity concepts you must recognize (overview only today)

These are expanded in later days, but the exam will mix them with today's content, so participants should recognize them now.

Need	Solution
Private VMs need outbound internet	**Cloud NAT** (needs a **Cloud Router** in the same region and VPC)
Private VMs need Google APIs	**Private Google Access** (subnet setting)
SSH/RDP to VMs with no public IP	**Identity-Aware Proxy (IAP) TCP forwarding** (source range **35.235.240.0/20**) or a bastion host
Connect two VPCs privately	**VPC Network Peering** (non-transitive)
Share one VPC across many projects	**Shared VPC** (host project + service projects)
Connect to on-prem (encrypted over internet)	**Cloud VPN** (HA VPN: 99.99% SLA when configured per docs)
Connect to on-prem (dedicated/partner)	**Cloud Interconnect**
Expose a service privately to consumers	**Private Service Connect**

Cloud NAT facts to remember:

- Regional, software-defined, **not a VM or a proxy in the data path**.
- Provides **outbound** connectivity (and return traffic for established connections). It does **not** allow unsolicited inbound connections from the internet.
- Requires a Cloud Router for configuration (not for routing).

VPC Network Peering facts to remember:

- **Non-transitive:** if A peers with B and B peers with C, A cannot reach C through B.
- No overlapping subnet ranges.
- Firewall rules are **not** shared across peers. Each VPC enforces its own.
- Both sides must create the peering.

2.7 Firewalls in depth

2.7.1 What a VPC firewall is

- Rules are defined **per VPC network** and enforced on **VM network interfaces**, so even traffic between two VMs in the same subnet is subject to rules.
- **Stateful:** If a connection is allowed in one direction, return traffic is automatically allowed (based on connection tracking: 5-tuple). You do not need a matching rule for replies.
- **Distributed:** No chokepoint. It scales with your workload.

2.7.2 Anatomy of a VPC firewall rule

Field	Details
Direction	`INGRESS` (incoming) or `EGRESS` (outgoing). One direction per rule.
Priority	Integer **0 – 65535**. **Lower number = higher priority.** Default when created with gcloud is **1000**.
Action	`ALLOW` or `DENY` (a rule has one action).
Target	Which VMs the rule applies to: **all instances in the network**, **network tags**, or **service accounts**.
Source (ingress)	**IP ranges**, **source tags**, or **source service accounts**.
Destination (egress)	**IP ranges** only (for VPC firewall rules).
Protocols and ports	`tcp:80`, `tcp:8000-8100`, `udp:53`, `icmp`, `all`, etc.
Enforcement	Enabled or disabled (useful for troubleshooting without deleting).
Logging	Optional per-rule logging to Cloud Logging.

Rules for combining sources (ingress):

- You can specify **source IP ranges and source tags (or source SAs)** together. A packet matches if it matches **either** (logical OR).
- You **cannot mix** source tags and source service accounts in the same rule.
- If you specify **no source** on an ingress rule via the API, no traffic matches. In the console, you pick one explicitly.

Source tags limitation: A rule using source tags only matches traffic whose source is a **VM in the same VPC, sent from its primary internal IP**. It does not match traffic arriving from outside the VPC.

2.7.3 Implied rules (always present, cannot be deleted)

Rule	Priority	Effect
Implied allow egress	65535	Allow all outbound traffic to any destination
Implied deny ingress	65535	Deny all inbound traffic from any source

Consequences:

- Out of the box, **nothing can reach your VM** unless you create an ingress allow rule.
- Out of the box, a VM **can** initiate any outbound connection (egress is open).
- To lock down egress, you must add an explicit **deny egress** rule at a priority that beats the implied allow, plus targeted allow rules at an even higher priority.

2.7.4 Traffic that is always allowed or always blocked

Always allowed (cannot be blocked by VPC firewall rules)	Notes
Traffic to and from the **metadata server** (`169.254.169.254`)	Needed for VM identity, startup scripts, etc.
DHCP, **DNS** (to the metadata server resolver), **NTP**	Basic VM operation

Always blocked / restricted	Notes
Certain protocols (for example **GRE**) and some other traffic types	⚠️ Check the current list in the docs
Outbound SMTP on port 25 to external destinations	Use alternatives (ports 587/465 through approved relays)
Traffic from a VM's **external IP to another VM's external IP in the same VPC** still **goes through the firewall**; but traffic to an external IP of a VM in the same VPC is routed internally in some cases	⚠️ Edge case, skip unless asked

2.7.5 Priorities and conflict resolution

The **highest-priority (lowest number) matching rule** decides. If an allow and deny rule have the **same priority**, **deny wins**.

Worked example:

#	Direction	Priority	Action	Target	Source/Dest	Port
A	Ingress	1000	Allow	tag `web`	`0.0.0.0/0`	tcp:80
B	Ingress	900	Deny	tag `web`	`203.0.113.0/24`	tcp:80
C	Ingress	65534	Allow	all	`10.0.0.0/8`	all

- A client from `203.0.113.7` to a `web` VM on port 80: Rules A and B both match. B (900) is higher priority than A (1000) → **denied**.
- A client from `198.51.100.5` : only A matches → **allowed**.
- `10.1.1.1` to port 22 on the same VM: A does not match (port 22), B does not match, C matches → **allowed** (priority 65534 beats the implied deny at 65535).

2.7.6 Targets: network tags vs. service accounts

Aspect	Network tags	Service accounts
What it is	A free-form string attached to the VM	The identity the VM runs as
Who can change it	Anyone with permission to **edit the VM's metadata/instance** (`compute.instances.setTags`)	Only someone with permission to change the VM's service account (needs `iam.serviceAccounts.actAs` on the SA and stopping the VM)

Aspect	Network tags	Service accounts
Security strength	Weaker: a compromised or over-privileged user can add the tag and gain access	**Stronger:** tied to IAM-controlled identity
Flexibility	Many tags per VM, easy	One SA per VM
Exam guidance	Fine for simple cases	**Preferred when the question emphasizes security, least privilege, or "prevent users from bypassing firewall rules"**

> **Rule of thumb for the exam:** If the question says "ensure only the app tier can access the database" and wants **tamper-resistance**, choose **service accounts** in firewall rules.

2.7.7 Firewall Rules Logging and Firewall Insights

- **Firewall Rules Logging** can be enabled **per rule**, producing logs of connections that matched that rule (allowed or denied). Logs show the rule, the connection details, and the VM. Logging a rule counts toward Cloud Logging cost.
- **Firewall Insights** (Network Intelligence Center) detects **shadowed rules**, unused rules, overly permissive rules, and hit counts. Requires logging on the relevant rules.
- You cannot log **implied rules** directly. To observe denied traffic, create an explicit deny-all rule with **low priority (e.g., 65534)** and **logging on**.

2.7.8 Firewall policies (the newer model) ⚠️

Beyond classic VPC firewall rules, Google Cloud has **firewall policies**:

Policy type	Scope	Notes
Hierarchical firewall policies	Attached to an **organization or folder**	Central security teams enforce rules across many projects. Actions: `allow`, `deny`, `goto_next` (delegate to the next level).
Global network firewall policies	Attached to a **VPC** (global)	Support tags (IAM-governed secure tags), address groups, FQDN and geolocation objects with the appropriate NGFW tier
Regional network firewall policies	Attached to a VPC in a **region**	Similar to global, regional scope

Default evaluation order:

1. Hierarchical firewall policies (organization, then folders, top-down)
2. VPC firewall rules
3. Global network firewall policies
4. Regional network firewall policies
5. Implied rules

> A `goto_next` action passes evaluation to the next level instead of deciding. Platform teams use this to enforce **mandatory** denies/allows at the organization level while delegating the rest to project teams.

Cloud Next Generation Firewall (Cloud NGFW) tiers (⚠️ names and features change): Essentials, Standard (adds FQDN and geolocation filtering, threat intelligence), Enterprise (adds intrusion prevention with TLS inspection). For the exam: know that **advanced inspection and IPS** is a Cloud NGFW Enterprise capability, while ordinary L3/L4 filtering is the VPC firewall.

2.7.9 Important source ranges to memorize

Range	Purpose
`35.235.240.0/20`	**IAP for TCP forwarding** (SSH/RDP to private VMs).
`130.211.0.0/22` and `35.191.0.0/16`	**Google Cloud health check probes** for most load balancers and MIG autohealing (allow these on the backends' port).
`169.254.169.254`	Metadata server
`10.128.0.0/9`	Auto mode subnet space

> **Common failure:** A MIG or load balancer marks backends unhealthy because there is no firewall rule allowing health-check ranges. Always add one.

2.8 Diagnosing network problems (tools to know)

Tool	Use
Connectivity Tests (Network Intelligence Center)	Static analysis (and live data-plane probes) of the path from A to B, showing which route or firewall rule drops traffic
Firewall Rules Logging	See which rules allow/deny real traffic
VPC Flow Logs	See flows between VMs and endpoints (source, destination, bytes, etc.)
Packet Mirroring	Clone traffic to a collector for IDS or deep inspection
Network Topology	Visualize traffic and relationships
Performance Dashboard	Latency and packet loss between zones/regions

2.9 Deep dive: how Google Cloud networking actually works

Understanding the mechanics lets participants reason about questions they have never seen before, instead of memorizing answers.

2.9.1 Software-defined networking (SDN) in one page

Layer	What it does	Why it matters for architects
Control plane	Holds the *intent*: VPCs, subnets, routes, firewall rules, NAT configs, peering. Programmed by your API calls.	Changes apply globally and in seconds, with no devices to log in to.
Data plane (Andromeda)	A distributed virtual switch/router on every host that forwards, filters, and encapsulates your VM's packets. On newer machine series, parts are offloaded to dedicated hardware (Titanium) ⚠️.	There is **no central router or firewall box** to size, patch, or fail over.

Layer	What it does	Why it matters for architects
Physical network (Jupiter fabric + private backbone)	Google's datacenter fabric and global fiber network carrying encapsulated traffic.	Cross-region traffic stays on Google's backbone (Premium Tier).

What this means in practice:

1. A VPC "router" is **not a device**. There is no router VM, no router you can SSH into, no router capacity to exhaust. Routes are just entries programmed into every host's virtual network stack.
2. A VPC "firewall" is **not a device**. Rules are compiled and enforced at each VM's virtual NIC (so two VMs on the same host and subnet are still filtered).
3. The "default gateway" (second address of the subnet) is a **virtual** construct. Traffic to any destination is sent to it; the host stack then decides: local subnet, another subnet, peered VPC, VPN tunnel, or the internet gateway.
4. **No broadcast, no multicast** at the VPC layer. Protocols that depend on them (VRRP, some clustering software, gratuitous ARP failover) do not work as-is. Use **alias IPs**, **internal passthrough load balancers**, or **routes with a next-hop instance** for failover designs.

2.9.2 Life of a packet: VM A to VM B in a different subnet and region

```
VM A (web-1, 10.10.1.5)                                  VM B (eu-1, 10.20.1.7)
  │  1. App sends packet to 10.20.1.7
  │  2. Guest OS sends to default gateway (10.10.1.1)
  ▼
Host stack (Andromeda) for VM A
  │  3. EGRESS firewall evaluated (hierarchical policy → VPC rules → network policies → implied allow)
  │  4. Route lookup: 10.20.1.0/24 matches subnet route (most specific) → next hop = "VPC network"
  │  5. Connection-tracking entry created (so replies are automatically allowed)
  │  6. Packet encapsulated and sent across Google's backbone to B's host
  ▼
Host stack for VM B
  │  7. INGRESS firewall evaluated against B's target tags / service account
  │  8. If allowed → delivered to B's vNIC. If not → silently dropped (no ICMP unreachable, no RST)
  ▼
VM B
```

> **Teaching point:** a firewall **drop is silent**. The client sees a timeout, not "connection refused." "Connection refused" means the packet *reached* the VM and nothing was listening. Timeout usually means firewall, route, or NAT. This distinction is used in troubleshooting questions.

2.9.3 MTU, bandwidth, and latency

Topic	Facts
MTU	Default VPC MTU is **1460 bytes** (not 1500, because of encapsulation overhead). It can be set per VPC from **1300 to 8896** (jumbo frames) ⚠️. All VMs, VPN endpoints and peers on a path should agree, or you see fragmentation, stalls on large transfers, and "SSH works but scp hangs" symptoms.
Egress bandwidth per VM	Roughly **2 Gbps per vCPU up to a series-dependent cap** (commonly in the 16–32 Gbps range for standard networking) ⚠️. **Tier_1 networking** raises the cap (up to 100+ Gbps on supported series).

Topic	Facts
Ingress bandwidth	Not capped by the VM's egress limit, but constrained by the VM's CPU, driver, and the sender's limits.
Intra-zone latency	Typically sub-millisecond.
Inter-zone latency (same region)	Typically around 1 ms or less.
Inter-region latency	Governed by distance (tens to hundreds of ms). Use the **Network Intelligence Center Performance Dashboard** to see actual numbers.
Per-flow limits	A single TCP flow cannot use the full VM bandwidth, so use parallel streams for bulk transfers.

Architect's implications

- Chatty tiers (app ↔ database) should be in the **same zone** or at least region when latency matters. Cross-zone traffic is billed (see 2.13).
- Large data movements between regions are both **slower and costlier**. Keep processing close to the data.

2.9.4 What "global" really means (and does not)

True	False
One VPC object spans all regions; one set of firewall rules governs all of them.	It does **not** automatically give you multi-region HA. Your VMs are still zonal.
Subnets in different regions route to each other with no VPN.	It does **not** remove inter-region cost or latency.
A global external load balancer can front backends in multiple regions of one VPC.	It does **not** mean subnets are global. Each subnet belongs to one region.

2.10 Deep dive: IP addressing, alias IPs and DNS

2.10.1 Internal IP behavior in detail

Scenario	What happens
VM created with default settings	Gets an **ephemeral internal IP** from the subnet's primary range (first free address after the reserved ones).
VM stopped and started	**Keeps** its internal IP.
VM deleted	Ephemeral internal IP is **released**.
Reserved **static internal IP**	Stays reserved until you release it. Can be assigned at VM creation or later. Reserving is free while it is in use.
Subnet expanded	Existing IPs unchanged. New range is a superset of the old one.
Two VMs, same IP	Not possible in a single VPC. (Possible across separate non-peered VPCs.)

Static internal IP uses: database endpoints, DNS targets, firewall appliance next hops, anything on-prem firewalls must allow-list.

2.10.2 Alias IP ranges

An **alias IP range** gives a VM's NIC **additional IP addresses** (a single IP or a CIDR block) that the VPC knows about and routes to that VM, without creating a custom route per address.

Aspect	Details
Source of addresses	The subnet's **primary** range or one of its **secondary** ranges.
Why use them	Containers/Pods (GKE VPC-native), multiple services on one VM each with a unique IP, software that binds to specific addresses.
Advantage over routes	Anti-spoofing is preserved, firewall rules apply naturally, no route-quota consumption, and they are visible to peering and VPN.
GKE VPC-native	Pod IPs and Service (ClusterIP) IPs are alias IPs from secondary ranges, so Pods are **first-class VPC citizens**: routable, filterable by firewall rules, and visible across peering and hybrid links.

2.10.3 External IP behavior in detail

Question	Answer
Is the VM's external IP configured inside the guest OS?	**No.** The guest only sees its internal IP. Google performs a 1:1 NAT to the external IP at the network layer.
Can an ephemeral external IP be promoted to static?	**Yes**: reserve the same address as static.
Does a stop/start change an ephemeral external IP?	**Yes.** It changes. Static IPs persist.
Regional vs global static IPs	Regional: VM NICs, regional load balancers, Cloud NAT. Global: global external load balancers.
Cost of idle static IPs	Reserved but unattached addresses are charged at a **higher** rate. Clean them up. ⚠️
How do I prevent teams from creating external IPs?	Organization policy `constraints/compute.vmExternalIpAccess` (allow-list specific VMs or deny all).

2.10.4 DNS in Google Cloud

Component	What it is	Typical use
Internal DNS (metadata resolver)	Automatic. Resolves `VM_NAME.ZONE.c.PROJECT_ID.internal` to internal IPs. Resolver address: `169.254.169.254`.	Instance-to-instance by name within a VPC. Not cross-project by default.
Cloud DNS public zones	Authoritative DNS for internet-facing names. 100% availability SLA ⚠️.	`www.example.com` → load balancer IP
Cloud DNS private zones	Authoritative names visible **only to selected VPCs**.	`db.corp.internal` → internal IP, across projects in a Shared VPC
Forwarding zones (outbound)	Forward queries for a domain (e.g., `corp.example.com`) from your VPC to **on-prem DNS servers**.	Resolve on-prem names from cloud

Component	What it is	Typical use
Inbound server policy	Allocates **inbound forwarder IPs** in your VPC so **on-prem resolvers** can query Cloud DNS private zones.	Resolve cloud names from on-prem
DNS peering zones	Resolve names using **another VPC's** DNS context.	Hub-and-spoke DNS, peering VPCs where DNS must be shared
Response policy zones (RPZ)	Override answers (e.g., send `storage.googleapis.com` to a restricted VIP, or block domains).	Private Google Access custom VIPs, DNS-level filtering

Hybrid DNS pattern (common exam topic):

```
On-prem DNS ── forwards cloud.example.com ──► Cloud DNS inbound policy IP (in VPC)
Cloud DNS   ── forwards corp.example.com  ──► On-prem DNS servers (forwarding zone)
        (both directions need routes + firewall rules allowing UDP/TCP 53 over VPN/Interconnect)
```

> Firewall note: DNS forwarding traffic **from** Cloud DNS originates from the range `35.199.192.0/19`. Ensure on-prem firewalls allow it. ⚠️

2.11 Deep dive: routing

2.11.1 What a route consists of

Field	Meaning
Destination range	CIDR the route matches (e.g., `10.20.0.0/16`)
Next hop	Where to send matching packets: default internet gateway, VM instance, IP address of a VM, VPN tunnel, Cloud Interconnect attachment (VLAN), internal passthrough load balancer, or the VPC itself (subnet routes)
Priority	0–65535, lower = preferred. Used only to break ties between routes with the **same destination prefix length**.
Network tags (optional)	Restricts the route to VMs with those tags. A **route without tags applies to all VMs** in the VPC.
Network	The VPC the route belongs to.

2.11.2 Route selection algorithm (apply it exactly)

1. Collect all routes whose **destination contains** the packet's destination address and that **apply to the sending VM** (tag filter).
2. Pick the **most specific destination** (longest prefix) — e.g., `/24` beats `/16` beats `/0`.
3. If multiple routes share that prefix length, pick the **lowest priority number**.
4. If still tied, Google Cloud may **load-balance (ECMP)** flows across tied routes.
5. If **no route matches**, the packet is dropped (no route to host).

> **Policy-based routes** (a newer feature) are evaluated ahead of the normal route table so you can steer traffic by source/protocol as well as destination. ⚠️ Check current precedence rules in the documentation.

Worked example. A VPC has these routes. Where does a VM tagged `via-nva` send a packet to `8.8.8.8`, and a VM with no tags?

Route	Destination	Next hop	Priority	Tags
default-internet	`0.0.0.0/0`	default internet gateway	1000	none
via-nva	`0.0.0.0/0`	nva-1 (instance)	900	`via-nva`
to-onprem	`192.168.0.0/16`	VPN tunnel	1000	none

- VM tagged `via-nva` → both `0.0.0.0/0` routes match; priority 900 beats 1000 → **sent to nva-1**.
- Untagged VM → the via-nva route does not apply → **default internet gateway**.
- Any VM to `192.168.5.5` → `/16` is more specific than `/0` → **VPN tunnel**, regardless of priorities of the default routes.

2.11.3 Static vs dynamic routes

Aspect	Static routes	Dynamic routes (BGP via Cloud Router)
Created by	You	Cloud Router learning from the peer router
Updates	Manual	Automatic, adapts to failures
Default priority	1000	Base priority **100** + MED value from the peer ⚠️
Best for	Small, simple, fixed topologies; NVA next hops	Hybrid connectivity with HA (HA VPN, Interconnect)
Failover	Needs your own health logic or multiple routes with different priorities	Built-in via BGP

Cloud Router key facts:

- A **managed, software-defined BGP speaker**, not a VM. Regional.
- Runs BGP sessions with on-prem routers, advertises VPC subnets, learns on-prem prefixes.
- **Advertisement modes:** default (advertises subnet ranges per dynamic routing mode) or **custom** (choose which prefixes and groups to advertise, for instance to also advertise a peered VPC's range or a private service access range).
- Also **hosts Cloud NAT configuration**, even though it does not route NAT traffic.
- Regional vs global dynamic routing is the *VPC-level* setting from 2.5.4; revisit it for any hybrid question.

2.11.4 Routes and peering

- Peering always exchanges **subnet routes** (unless you opt out for certain privately used public ranges).
- You can **optionally** exchange **custom routes** (static and dynamic) with `import custom routes` / `export custom routes`. This is how a spoke VPC learns on-prem routes through a hub VPC that holds the VPN.

- **Transitivity trap:** even with custom route exchange, you cannot make A reach C through B for subnet routes. Use Network Connectivity Center or a design with an NVA/VPN where transit is needed.
- **Conflicts:** a local subnet route **always wins** over a peering route for the same destination.

2.11.5 Network virtual appliance (NVA) pattern

Ingredient	Why
NVA VM with `canIpForward` enabled	Allows it to forward packets not addressed to it
Guest OS IP forwarding enabled (`net.ipv4.ip_forward=1`)	Linux requirement
Route with next hop = NVA (instance, IP, or **internal passthrough LB** for HA)	Steers traffic through it
Multiple NICs (each in a different VPC)	Lets the NVA bridge segments (e.g., trust and untrust VPCs)
Firewall rules allowing the traffic to and from the NVA	The VPC still filters
Internal passthrough LB as next hop + health checks	**HA** for the appliance, with symmetric hashing for stateful inspection

2.12 Deep dive: Private Google Access, Private Service Connect and API reachability

2.12.1 Ways a VM reaches Google APIs

Method	Needs external IP?	Path	Typical use
Public endpoints (`storage.googleapis.com`) via external IP or NAT	Yes (or NAT)	Public Google front ends	Default for VMs with internet
Private Google Access (default domains)	No	Google API front ends via Google's network; subnet flag on	Private VMs needing APIs
`private.googleapis.com` (`199.36.153.8/30`)	No	Private VIP that supports **all** Google APIs	Private access with tighter control
`restricted.googleapis.com` (`199.36.153.4/30`)	No	VIP that only supports APIs **supported by VPC Service Controls**	**Data-exfiltration controls**: blocks access to non-VPC-SC-compatible APIs
Private Service Connect for Google APIs	No	A **private IP in your VPC** that maps to Google APIs (bundle `all-apis` or `vpc-sc`)	Hybrid access: on-prem hosts reach Google APIs through your VPN/Interconnect to that IP
Private services access	No	VPC peering to a Google-managed VPC (Cloud SQL private IP, Memorystore, etc.)	Managed services with **private IPs**
Serverless VPC Access / Direct VPC egress	n/a	Lets Cloud Run/Functions/App Engine reach **into** your VPC	Serverless → private resources

To use `private.googleapis.com` or `restricted.googleapis.com`: create a **DNS response policy/private zone** that maps `*.googleapis.com` to the VIP via CNAME, add **routes** for the VIP range to the default internet gateway (this still stays on Google's network), and allow egress to the VIP range in firewall rules.

2.12.2 PGA vs Cloud NAT vs external IP

Requirement	PGA	Cloud NAT	External IP
Reach Google APIs from a private VM	✅	✅ (but over public endpoints)	✅
Reach arbitrary internet sites	❌	✅ (outbound only)	✅
Accept unsolicited inbound from internet	❌	❌	✅ (if firewall allows)
Per-VM exposure	None	None	High
Reaches Google APIs **even if NAT is misconfigured**	✅	n/a	n/a

2.12.3 Private Service Connect (PSC) in brief

- **PSC endpoint:** a **forwarding rule with an internal IP** in your VPC that targets a published service (Google APIs, a Google-managed service, or a third party/producer service in another VPC or organization).
- **PSC backend:** a network endpoint group (NEG) used as a load balancer backend.
- **Why it matters:** gives **private, non-transitive, one-way** consumer → producer connectivity **without peering** or exchanging IP ranges. **Overlapping IP ranges are fine** because only a single IP is presented to the consumer.
- **Exam rule of thumb:** overlapping CIDRs between consumer and producer, or "expose a service to many consumers without VPC peering" → **Private Service Connect**.

2.13 Deep dive: Cloud NAT, egress costs, and Network Service Tiers

2.13.1 How Cloud NAT allocates ports

- Each NAT external IP provides **64,512 usable source ports** (1024–65535) for each protocol ⚠️.
- A NAT mapping (VM:port → NAT IP:port) is needed **per distinct destination (IP, port, protocol)** when endpoint-independent mapping is off. A VM that makes many simultaneous connections to **one** destination exhausts its ports faster than one that spreads connections.
- **Minimum ports per VM** defaults to **64** ⚠️. With **dynamic port allocation** the NAT gateway grows and shrinks each VM's allocation between configured min and max.
- **Symptoms of exhaustion:** intermittent `connection timeout` or `connection reset` to the same destination under load; NAT logs show `OUT_OF_RESOURCES` drops.
- **Fixes:** add more NAT IPs, raise min ports per VM or enable dynamic port allocation, use connection pooling/keep-alive, reduce the number of VMs per NAT gateway, shorten idle timeouts, or place **different workloads on different NAT gateways**.

NAT setting	Options	Guidance
IP allocation	Automatic or manual (reserved static IPs)	**Manual** when partners must **allow-list** your egress IPs
Subnet coverage	All subnets / specific subnets / specific secondary ranges	Narrow in regulated designs
Logging	Errors only, translations only, all	Turn on at least errors
NAT type	**Public NAT** (to internet) vs **Private NAT** (between private networks/VPCs, e.g., overlapping ranges, NCC)	Private NAT solves overlapping-range hybrid cases ⚠️

2.13.2 Data transfer pricing model (principles; verify numbers ⚠️)

Traffic	Typical charging
Ingress to Google Cloud	Free
VM ↔ VM **same zone**, internal IPs	Free
VM ↔ VM **different zones, same region**	**Charged** per GB
VM ↔ VM **different regions**	**Charged** per GB (higher; varies by continent pair)
VM → Google APIs in same region (e.g., Cloud Storage same region)	Generally free
VM → **internet** (egress)	Charged; tiered by volume and **network tier**
VM ↔ on-prem over VPN/Interconnect	Interconnect egress cheaper than internet egress; VPN egress billed as internet-like ⚠️
Traffic using **external IPs** between VMs in the same zone	Billed like external traffic, so **always use internal IPs between VMs**

Exam implication: To cut cost, keep chatty tiers in the same zone *when HA allows*, use internal IPs, push static content through a CDN, use Cloud Interconnect for heavy hybrid traffic, and prefer same-region buckets.

2.13.3 Network Service Tiers decision logic

Decide	Premium	Standard
Global load balancing	Required	Not available (regional only)
Latency/consistency	Best, traffic enters/exits Google's network close to the user	Variable (public internet carries the traffic)
Price	Higher	Lower
Typical fit	Global SaaS, gaming, streaming	Regional apps, dev/test, cost-sensitive internal tools
Per-resource choice	Yes. Tier is chosen **per external IP / forwarding rule**, so you can mix.	

2.14 Deep dive: Shared VPC, VPC Peering, and choosing a

connectivity model

2.14.1 Shared VPC mechanics

Concept	Details
Host project	The project that **owns** the VPC network, subnets, routes, firewall rules, Cloud NAT, VPN, and Cloud Router.
Service projects	Attached to the host. Their resources (VMs, GKE nodes, etc.) get IPs **from the host's subnets**. A project can be a service project of **only one** host.
Prerequisite	Both projects must be in the **same organization**.
Who controls what	**Network team** (host project) controls IP space, firewalls, routing, hybrid links. **App teams** (service projects) own VMs, billing, quotas, and IAM for their workloads.
Granting subnet use	`roles/compute.networkUser` granted at **project level** (all subnets) or **per subnet** (recommended for least privilege) to the people and service accounts that create resources in service projects.
Key admin roles	**Shared VPC Admin** (`roles/compute.xpnAdmin` , assigned at org or folder level) enables the host and attaches service projects. **Service Project Admin** is a delegated role limited to specific service projects.
Firewall ownership	Rules live **in the host project**. Service project users cannot change them unless granted `compute.securityAdmin` on the host.
Billing/quota	Compute usage billed to the **service project**; network resources (and their egress) are billed to the host project, according to where they are defined ⚠️
Why choose it	Centralized governance, consistent IP planning and security, still delegated app ownership, private IP connectivity between projects with no peering.

Typical enterprise layout

```
Organization
 └─ Folder: Networking
 │     └─ Project: net-host-prod  (Shared VPC host: prod-vpc, VPN/Interconnect, NAT, firewall policies)
 └─ Folder: Applications
       ├─ Project: shop-frontend-prod   (service project) ─┐
       ├─ Project: shop-backend-prod    (service project) ─┼─ use subnets from prod-vpc
       └─ Project: data-platform-prod   (service project) ─┘
```

2.14.2 VPC Network Peering mechanics

Aspect	Details
Direction	**Two-sided:** each VPC creates a peering to the other. Becomes active only when both exist.
What is exchanged	Subnet routes always; custom routes optionally (import/export flags).
Transitivity	**None.**
Overlap	Not allowed for subnet ranges.
Cross-project / cross-org	Allowed.
Firewalls	Independent. Each side must allow traffic from the other's ranges.

Aspect	Details
IAM	Peering admin permissions needed on both sides.
Performance	Same as same-VPC internal traffic: no extra hop, no bandwidth bottleneck.
DNS	Not shared automatically. Use **DNS peering zones** to resolve across.
Limits	Per-VPC peering and total-route limits apply ⚠️. Large hub-and-spoke designs hit them.

2.14.3 Choosing a model

Need	Shared VPC	VPC Peering	Cloud VPN (between VPCs)	Network Connectivity Center	Private Service Connect
Central network/security governance	✅ best	❌	❌	✅ (hub)	❌
Many projects share one IP space	✅	❌ (separate IP spaces)	❌	❌	❌
Connect **independently managed** VPCs privately	❌	✅	✅	✅	✅ (service-level)
Transitive connectivity	N/A	❌	Possible (with BGP design)	✅	❌
Overlapping IP ranges tolerated	❌	❌	❌ (without NAT)	❌ (use Private NAT)	✅
Cross-organization	❌	✅	✅	Limited ⚠️	✅
Bandwidth/latency overhead	None	None	Tunnel limits	Depends	Low

2.15 Deep dive: firewall engine, patterns and defense in depth

2.15.1 Connection tracking and statefulness in detail

- The firewall tracks connections by the **5-tuple** (source IP, source port, destination IP, destination port, protocol).
- If a connection is **allowed** in the direction that created it, packets in the **reverse** direction belonging to that connection are **automatically allowed**, even if an egress deny rule or the implied deny ingress would otherwise block them.
- Tracking entries **expire after an idle period** (on the order of **10 minutes** for established TCP) ⚠️. Long-idle connections (idle database sessions, SSH without keep-alive) can break. Fix with **TCP keep-alives** shorter than the timeout.
- **A connection tracked as allowed continues** even if you **change** or delete the rule mid-session (**until it expires**), which surprises engineers testing "block" changes on a long-lived connection. Re-test with a **new** connection.
- Rules with `all` **protocols** and **ICMP** are tracked too (ICMP echo reply is allowed as a response to an allowed request).

2.15.2 Firewall rule evaluation algorithm (use it for every "will this connection work?" question)

For a packet **leaving** VM A (egress evaluation) and **arriving** at VM B (ingress evaluation), both must pass.

1. **Does a tracked connection already permit this packet (reply traffic)?** If yes → allowed.
2. Evaluate **hierarchical firewall policies** (org → folders). First matching rule decides (`allow`, `deny`, or `goto_next` to continue).
3. Evaluate **VPC firewall rules** applicable to the VM (target matches): sort by **priority**; the first match decides; ties → **deny** wins.
4. Evaluate **global then regional network firewall policies** (default order; can be reordered).
5. If nothing matches → **implied rule**: **allow egress / deny ingress**.

> A packet from A to B needs **egress allowed on A** *and* **ingress allowed on B**. Many wrong answers forget one half.

2.15.3 Target and source matrix

Rule has...	Applies to / matches...	Notes
No target specified (all instances)	Every VM in the VPC	Widest scope, so use carefully
Target tags	VMs with at least one of those tags	Tags are per-VM and editable by instance admins
Target service accounts	VMs running as those SAs	Tamper-resistant
Source IP ranges	Packets with that source IP (from anywhere, including internet/on-prem)	Use for external sources
Source tags	Packets from **VMs in the same VPC** with that tag, **using primary internal IP**	Does not match external or on-prem sources, and not alias IPs
Source service accounts	Packets from VMs in the same VPC running as that SA	Similar limitation
Source ranges + source tags	**Either** matches (OR)	Mixed sources widen the rule
Target tags + target service accounts	Not allowed in the same rule	Pick one targeting method
Egress destination	**IP ranges only** (in VPC firewall rules). Use network policies for FQDN or geo	

2.15.4 Network firewall policies: richer matching ⚠️

Capability	Description
Secure tags (Resource Manager tags)	IAM-controlled key/value tags bound to VMs/VPCs; used as source and target. Better than network tags because **IAM restricts who can attach them**.
Address groups	Reusable lists of IPs/CIDRs referenced by many rules.
FQDN objects	Match by domain names (e.g., `*.windowsupdate.com`) for egress allow-lists.

Capability	Description
Geolocation	Allow or deny by country.
Google Threat Intelligence lists	Block known malicious IPs (Tor exit nodes, known bad IPs).
Layer 7 inspection (Enterprise tier)	Intrusion prevention, TLS inspection via firewall endpoints.
Policy sharing/association	One policy associated with many VPCs; changes apply everywhere.
Rule priorities	Unique per policy; the lowest number evaluated first.

2.15.5 Firewall design patterns

Pattern	Rules
Three-tier with identity	web-sa ← internet:443; app-sa ← web-sa:8080; db-sa ← app-sa:5432. No other ingress.
Admin access	Allow `35.235.240.0/20` → tcp:22 / tcp:3389 to a tag or SA. No public SSH.
Default deny egress with allow-list	Low-priority `deny egress all`; higher-priority `allow` for Google API VIP range (e.g., `199.36.153.4/30`), required internal CIDRs, and (policy-based) FQDNs.
Shared services	`allow` from all environments' SAs to shared DNS/AD/logging subnets on specific ports.
Isolation of environments	Separate VPCs (or secure tags per environment) plus a deny between dev and prod CIDRs at an early priority.
Break-glass	Pre-created **disabled** rules that on-call engineers can enable temporarily (and audit).
Compliance logging	Logging on all deny/allow rules touching regulated tiers, exported to BigQuery/SIEM via a log sink.

2.15.6 Defense in depth: which control does what?

Control	Layer	Protects against	Not a substitute for
IAM	Control plane	Unauthorized API actions	Network filtering
VPC firewall / network policies	L3/L4	Unauthorized network flows	App-level authz
Cloud Armor	L7 at the edge (global/regional external LBs)	DDoS, WAF rules (SQLi, XSS), geo/IP block, bot/rate controls	VPC firewall for east-west traffic
Cloud NGFW Enterprise / IPS	L7 inspection in-VPC	Malware, intrusion attempts	Edge DDoS protection
IAP	Identity-aware access proxy	Unauthenticated access to web apps and SSH	Network segmentation
VPC Service Controls	API perimeter	**Data exfiltration** from Google-managed services to unauthorized projects/networks	VM-level firewall
Private Google Access / PSC	Reachability	Public exposure of API traffic	Authorization
Shielded/Confidential VM, OS Login	Host/guest	Boot-level malware, memory snooping, credential sharing	Network controls

Control	Layer	Protects against	Not a substitute for
Cloud Audit Logs, flow logs, firewall logs, SCC	Detection	Blind spots	Prevention

3. Module 2: Implementing VPC Networks and Firewall Rules

This module turns the theory into a repeatable **design and implementation method**. The hands-on commands are in Section 6 (Labs 1–3); this section explains what to decide and why.

3.1 VPC design methodology (use this for exam answers too)

Step 1: Gather requirements

- How many environments (dev/test/prod)? Isolation level needed?
- Which regions? Any data residency rules?
- Hybrid connectivity to on-prem? What on-prem CIDRs exist?
- Which services will need private IPs (GKE, Cloud SQL private IP, Serverless VPC Access / Direct VPC egress)?
- Compliance demands (PCI, HIPAA): segmentation, logging, no public IPs?

Step 2: Choose a project and VPC structure

Pattern	When to use
Single VPC, single project	Small workload, one team
One VPC per environment/project (dev, test, prod)	Strong isolation, simple
Shared VPC (host project + service projects)	Central network team, many app teams, one consistent network and firewall posture
Hub-and-spoke (peering or NCC/VPN)	Separate VPCs connected via a hub with shared services or an NVA

Step 3: Plan the IP space

- Use a **documented IPAM plan**. Avoid overlaps with on-prem, other clouds, and other VPCs you may peer with.
- Allocate **per region and per tier** with room to grow. Example:

Region	Tier	Primary range	Secondary ranges
us-central1	web	10.10.1.0/24	none
us-central1	app	10.10.2.0/24	none
us-central1	db	10.10.3.0/24	none
us-central1	GKE	10.10.8.0/22	pods 10.64.0.0/14, services 10.68.0.0/20
europe-west1	all	10.20.0.0/16 reserved	

- Keep GKE Pod and Service ranges **large** and **unique**; they are hard to change later.

Step 4: Choose a subnet strategy

- **Subnet per tier** (web/app/db) is common but **not required** for security since firewall rules can target tags/SAs. Subnets help with **organization, flow logs granularity, and routing policies**.
- **Subnet per region** where workloads run.

Step 5: Design the firewall model

1. Start from **deny by default** (implied deny ingress).
2. Allow only required **ingress** by tier-to-tier with **service account** targets.
3. Allow **IAP** (35.235.240.0/20) for admin access instead of opening SSH to the internet.
4. Allow **health checks** (130.211.0.0/22, 35.191.0.0/16).
5. Consider **egress restrictions** for regulated workloads (deny all egress, allow specific destinations; use Private Google Access plus VPC Service Controls for API exfiltration controls).
6. Enable **logging** on sensitive rules; use **hierarchical policies** for org-wide guardrails.

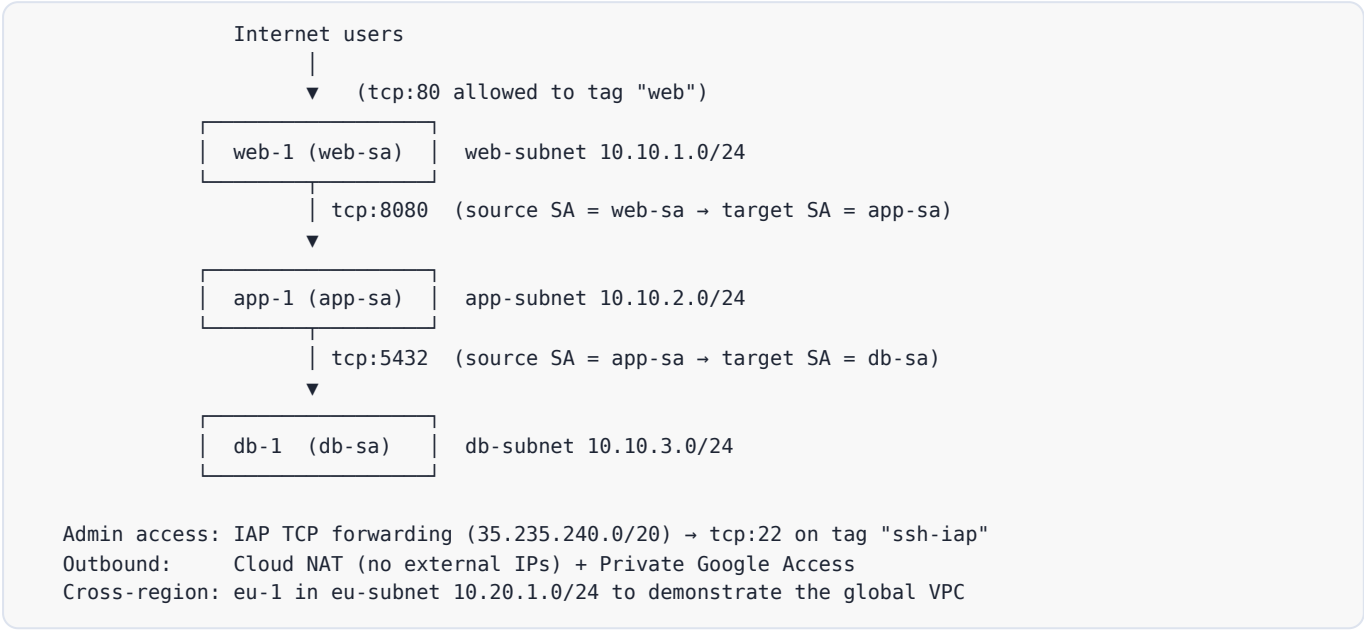
Step 6: Plan outbound access

- Private VMs → **Cloud NAT** (internet) and **PGA** (Google APIs).
- Optional: **Secure Web Proxy** or NVA for egress filtering and inspection. ⚠️ Mention only.

Step 7: Plan observability

- Flow logs on subnets (sampling tuned for cost), firewall logs on key rules, Network Intelligence Center for ongoing review.

3.2 Reference architecture used in the labs



> In Lab 3, the web VM is **not** given a public IP, to match modern best practice. The "internet users" arrow is realized in practice by a load balancer (covered on a later day). For the demo, we test port 80 internally and discuss how the LB would front it.

3.3 Firewall implementation checklist

Before creating a rule, answer:

Question	Typical answer
Direction?	Ingress for "who can connect to me"; Egress for "where can I connect"
Allow or deny?	Allow in most cases; deny to override broader allows
Priority?	Leave lots of gaps (e.g., 1000, 2000, 3000), so you can insert later
Targets?	Service account (secure) or network tag (simple)
Source/destination?	Narrowest possible CIDR or identity
Protocol/ports?	Specific ports, **not** `all`
Logging?	On for sensitive or deny rules

Naming convention suggestion: `{action}-{source}-to-{target}-{proto-port}` e.g., `allow-web-to-app-tcp8080`.

3.4 Common implementation pitfalls

1. Creating an allow rule for SSH from `0.0.0.0/0` "just for a moment". It gets forgotten.
2. Forgetting that **firewall rules are VPC-scoped**. A rule in VPC A does nothing in VPC B.
3. Using `source-tags` and expecting it to match **on-prem** or internet traffic. It will not.
4. Giving VMs the **default compute service account** and then using that SA in firewall rules, which makes every VM match.
5. Forgetting the **health check** ranges, so the MIG keeps recreating healthy instances.
6. Deleting default rules without creating replacements, then losing SSH/ICMP. (Use IAP instead.)
7. Overlapping subnets, which blocks peering later.
8. Treating network tags as security boundaries.

3.5 Deep dive: CIDR math and IP address planning

IP planning is a recurring exam theme ("overlapping ranges", "ran out of IPs", "GKE needs more Pod IPs"). Participants must be able to do the arithmetic quickly.

3.5.1 CIDR cheat table

Prefix	Total addresses	Usable in a GCP primary subnet (total − 4)	Typical use
/29	8	4	Smallest subnet; tiny management or NVA subnets
/28	16	12	Small utility subnets; Serverless VPC Access connector range
/27	32	28	Small tiers, small management subnets
/26	64	60	Small workloads

Prefix	Total addresses	Usable in a GCP primary subnet (total − 4)	Typical use
/25	128	124	
/24	256	252	Typical tier subnet
/23	512	508	
/22	1,024	1,020	Medium environments
/20	4,096	4,092	Auto mode subnet size; GKE Services range
/16	65,536	65,532	Per-region or per-environment allocation
/14	262,144	n/a (secondary ranges have no reserved addresses) ⚠️	GKE Pod range for large clusters
/8	16,777,216		Whole `10.0.0.0/8` space

Formula: total addresses = 2^(32 − prefix). Usable = total − 4 for GCP **primary** ranges.

Quick sizing method: (1) estimate the peak number of VMs, (2) add **50–100% growth headroom**, (3) add 4, (4) round up to the next power of two. *Example:* 300 VMs today, expected to double → 600 + 4 = 604 → next power of two is 1,024 → **/22**.

3.5.2 Overlap checks

Two ranges overlap if one **contains** the other or they share any addresses.

A	B	Overlap?
`10.10.0.0/16`	`10.10.5.0/24`	**Yes** (B is inside A)
`10.10.0.0/24`	`10.10.1.0/24`	No
`10.0.0.0/8`	`10.128.0.0/9`	**Yes** (auto mode inside on-prem /8)
`172.16.0.0/12`	`172.32.0.0/16`	No (`/12` ends at 172.31.255.255)
`192.168.0.0/16`	`192.168.100.0/22`	**Yes**

Where overlaps hurt: VPC peering (rejected), VPN/Interconnect (ambiguous routing, one side unreachable), Shared VPC (same VPC, so rejected on creation), mergers/acquisitions. **Mitigations:** re-IP (best), Private NAT, Private Service Connect (service-level, overlap tolerated), or an NVA doing NAT.

3.5.3 Enterprise IPAM strategy

Practice	Why
Keep a **central IPAM source of truth** (spreadsheet/CMDB/IPAM tool/Terraform variables)	Prevents teams from picking colliding ranges
Allocate **by region and environment** with hierarchical supernets, for example `10.<env>.<region>.0/..`	Easy summarization for BGP advertisements and firewall rules
Reserve ranges for **on-prem, other clouds, and growth**	Avoids future conflicts
Use **summarizable blocks**	Smaller route tables, simpler on-prem firewall rules

Practice	Why
Consider **RFC 6598 (100.64.0.0/10)** or **privately used public IPs** for GKE Pods if RFC 1918 space is scarce ⚠️	Extends space, with caveats
Document **secondary ranges** (GKE Pods/Services) separately	They're the largest consumers

Illustrative hierarchical plan

Block	Allocation
`10.0.0.0/12`	On-premises data centers
`10.16.0.0/12`	**Production** (carve /16 per region: `10.16.0.0/16` us-central1, `10.17.0.0/16` europe-west1, …)
`10.32.0.0/12`	**Non-production**
`10.48.0.0/12`	Shared services / hub
`10.64.0.0/10`	GKE Pod ranges (large)
`10.128.0.0/9`	**Avoid** (auto mode space) or reserve for future

3.5.4 GKE IP sizing worked example

A VPC-native GKE cluster needs three IP ranges: **node** (primary subnet range), **Pod** (secondary), **Service** (secondary).

Parameter	Default/rule	Consequence
Max Pods per node	110 by default	Each node reserves a **/24** of the Pod range (256 addresses, double the Pod max to allow churn)
Pod range `/14`	262,144 addresses ÷ 256 per node	Supports **up to 1,024 nodes**
Pod range `/16`	65,536 ÷ 256	Only **256 nodes**
Service range `/20`	4,096 addresses	Up to ~4,096 ClusterIP Services
Node subnet `/22`	1,020 usable	Up to ~1,020 nodes **(minus other VMs in the subnet)**

> Reducing max Pods per node (e.g., 32) lets each node use a **/26** (64 IPs), so a smaller Pod range supports more nodes. Pod and Service ranges are **hard to change** after cluster creation, so size generously.

3.6 Deep dive: reference network architectures

3.6.1 Architecture A: single project, single VPC (small team)

```
Project: app-prod
 └─ VPC: app-vpc (custom)
      ├─ subnet web   (us-central1)
      ├─ subnet app   (us-central1)
      ├─ subnet db    (us-central1)
      └─ Cloud NAT + Cloud Router, IAP for admin, firewall by service account
```

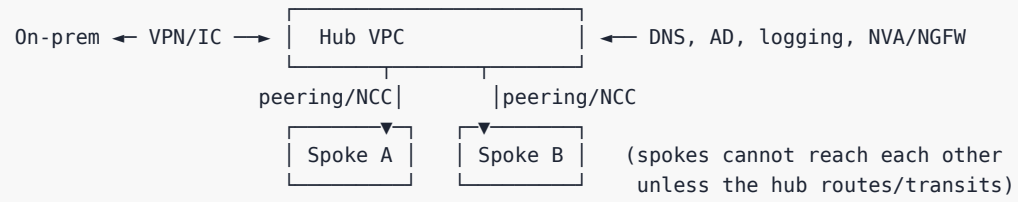
Fits: start-ups, single application, low governance needs. *Limits:* blast radius is the entire project, and quotas are shared.

3.6.2 Architecture B: environment-per-project with Shared VPC

```
Org ── Folder: prod ──► host project (prod-vpc) ◄── service projects (teams)
   └── Folder: non-prod ► host project (nonprod-vpc) ◄── service projects (teams)
```

Fits: mid/large organizations. Separate **prod and non-prod VPCs** so a firewall mistake or compromised dev workload cannot reach production. Central team owns the VPC and firewall policy; teams own their workloads.

3.6.3 Architecture C: hub-and-spoke with shared services



Fits: strict segmentation with centralized inspection and a single hybrid edge. *Watch for:* non-transitive peering (use NCC or NVA with custom routes), route and peering limits, DNS peering.

3.6.4 Architecture D: multi-region active/active

```
One VPC ─ subnet us (10.16.0.0/16)  + regional MIG ─┐
       └─ subnet eu (10.17.0.0/16)  + regional MIG ─┴─► global external Application LB (Premium Tier)
                      data layer: Spanner / multi-region storage / replicated DB
```

Fits: global users, regional failure tolerance. *Cost note:* inter-region traffic is billed; choose data stores whose replication model matches the RPO/RTO.

3.6.5 Choosing a pattern

Requirement	Pattern
One team, one app, simple	A
Many teams, central governance, same org	B (Shared VPC)
Strict segmentation, centralized inspection, hybrid hub	C

Requirement	Pattern
Global users, regional resilience	D (combined with B or C)
Independent companies/orgs connecting	Peering or PSC (not Shared VPC)

3.7 Deep dive: governing networks with Organization Policy and IAM

3.7.1 Network-related organization policy constraints (know the names)

Constraint	Effect
`compute.skipDefaultNetworkCreation`	New projects **don't** get the `default` network
`compute.vmExternalIpAccess`	Restrict which VMs can have external IPs (allow-list or deny all)
`compute.restrictSharedVpcHostProjects` / `compute.restrictSharedVpcSubnetworks`	Limit which host projects/subnets can be used
`compute.restrictVpcPeering`	Limit which networks can be peered
`compute.restrictCloudNATUsage`, `compute.restrictVpnPeerIPs`	Limit NAT/VPN usage and peer endpoints ⚠️
`compute.vmCanIpForward`	Controls which VMs may enable IP forwarding
`compute.disableSerialPortAccess`	Blocks interactive serial console
`compute.requireOsLogin`	Forces OS Login on VMs
`compute.requireShieldedVm`	Requires Shielded VM images
`compute.restrictNonCmekServices` / `compute.restrictCmekCryptoKeyProjects`	Forces CMEK for listed services/disk keys
`compute.trustedImageProjects`	Limits which projects' images can be used (supply-chain control)
`gcp.resourceLocations`	Restrict regions (data residency)
`iam.disableServiceAccountKeyCreation`	Block downloadable service account keys

> **How to answer "prevent X across the organization" questions:** a **preventive organization policy** or **hierarchical firewall policy** is almost always better than documentation, audits, or per-project manual config.

3.7.2 Network IAM roles and separation of duties

Role	Can do	Notes
`roles/compute.networkAdmin`	Create/modify networks, subnets, routes, NAT, VPN, etc.	**Cannot** manage firewall rules or SSL certificates
`roles/compute.securityAdmin`	Create/modify **firewall rules** and SSL certs	Pair with networkAdmin for full control
`roles/compute.networkUser`	Use subnets (create VMs in Shared VPC subnets)	Grant **per subnet** in Shared VPC

Role	Can do	Notes
`roles/compute.networkViewer`	Read-only networking	Auditors
`roles/compute.instanceAdmin.v1`	Full control of VMs and disks (no network config)	Includes ability to edit **network tags**, which is why tag-based firewalls are weak
`roles/compute.xpnAdmin`	Shared VPC admin	Org/folder level
`roles/iam.serviceAccountUser`	Attach/act as service accounts	Needed to create VMs running as a given SA
`roles/iap.tunnelResourceAccessor`	Use IAP TCP forwarding	
`roles/compute.osLogin` / `osAdminLogin`	SSH as normal/sudo user via OS Login	

Separation of duties example: network engineers → `networkAdmin` + `securityAdmin` on the host project; developers → `instanceAdmin.v1` + `networkUser` (specific subnets) in service projects; neither group can alter the other's domain.

3.8 Deep dive: Infrastructure as Code for networks (Terraform)

The exam expects you to recognize IaC as the way to implement repeatable, reviewable networks (Terraform and Deployment Manager are both mentioned; Terraform is the recommended tool). The following mirrors Labs 1–2.

```
variable "project_id" {
  type = string
}
variable "region" {
  type    = string
  default = "us-central1"
}

resource "google_compute_network" "prod" {
  name                    = "prod-vpc"
  project                 = var.project_id
  auto_create_subnetworks = false          # custom mode
  routing_mode            = "REGIONAL"
}

resource "google_compute_subnetwork" "web" {
  name                     = "web-subnet"
  project                  = var.project_id
  region                   = var.region
  network                  = google_compute_network.prod.id
  ip_cidr_range            = "10.10.1.0/24"
  private_ip_google_access = true

  log_config {
    aggregation_interval = "INTERVAL_5_SEC"
    flow_sampling        = 0.5
    metadata             = "INCLUDE_ALL_METADATA"
  }
}

resource "google_service_account" "web" { account_id = "web-sa" }
resource "google_service_account" "app" { account_id = "app-sa" }
```

```hcl
resource "google_compute_firewall" "web_to_app" {
  name      = "allow-web-to-app-tcp8080"
  network   = google_compute_network.prod.name
  direction = "INGRESS"
  priority  = 1000

  allow {
    protocol = "tcp"
    ports    = ["8080"]
  }

  source_service_accounts = [google_service_account.web.email]
  target_service_accounts = [google_service_account.app.email]

  log_config { metadata = "INCLUDE_ALL_METADATA" }
}

resource "google_compute_firewall" "iap_ssh" {
  name          = "allow-iap-ssh"
  network       = google_compute_network.prod.name
  direction     = "INGRESS"
  source_ranges = ["35.235.240.0/20"]
  target_tags   = ["ssh-iap"]

  allow {
    protocol = "tcp"
    ports    = ["22"]
  }
}

resource "google_compute_router" "prod" {
  name    = "prod-router"
  region  = var.region
  network = google_compute_network.prod.id
}

resource "google_compute_router_nat" "prod" {
  name                               = "prod-nat"
  router                             = google_compute_router.prod.name
  region                             = var.region
  nat_ip_allocate_option             = "AUTO_ONLY"
  source_subnetwork_ip_ranges_to_nat = "ALL_SUBNETWORKS_ALL_IP_RANGES"
  log_config {
    enable = true
    filter = "ERRORS_ONLY"
  }
}
```

Why IaC matters (exam angle):

Benefit	Detail
Repeatability	Identical dev/test/prod networks from one module
Review and audit	Pull requests on firewall changes; Git history as an audit trail
Drift detection	`terraform plan` shows manual changes made in the Console
Policy as code	Validate plans against policies (e.g., Terraform validator / OPA) before apply
Disaster recovery	Rebuild networks quickly in another region or project

Benefit	Detail
State handling	Store Terraform state remotely (a Cloud Storage bucket with versioning and locking) and protect it, since it may contain sensitive values

3.9 Deep dive: operating firewalls over time

Lifecycle step	Practice
Request	Rule changes come through code review (IaC), not ad hoc Console edits
Naming and description	Use consistent names; put ticket numbers in the description field
Staging	Create new rules with **logging on**; use a higher-priority number while testing
Validation	**Connectivity Tests** before and after
Cleanup	Use **Firewall Insights** to find shadowed, overly permissive, or unused rules (unused for N days)
Review cadence	Quarterly access review of broad rules (`0.0.0.0/0` , `all` protocols)
Quotas	Know the per-project and per-network rule limits ⚠️. Reduce rule count with **service accounts/secure tags and address groups** rather than hundreds of IP-based rules
Emergency changes	Break-glass rules with automatic expiry or follow-up tickets

4. Module 3: Compute Engine Instances and Customization

4.1 What Compute Engine is

Compute Engine (GCE) is Google Cloud's **IaaS** offering: virtual machines on Google's infrastructure (KVM-based hypervisor). You manage the OS and everything above it; Google manages the hardware, hypervisor, and physical network.

When to choose Compute Engine (vs. other compute options):

Choose...	When...
Compute Engine	You need OS-level control, custom or legacy software, specific kernels/drivers, licensing constraints, GPUs, lift-and-shift migrations, or workloads not containerized
GKE	Containerized microservices needing orchestration
Cloud Run	Stateless containers, HTTP/event-driven, scale to zero, minimal ops
App Engine	PaaS for web apps with managed runtimes
Cloud Run functions	Event-driven snippets
Bare Metal Solution ⚠️	Specialized workloads (e.g., Oracle) needing bare metal
VMware Engine	Lift-and-shift of VMware workloads

4.2 Resource location model

- **Instances are zonal.** A VM lives in one zone.
- **Persistent disks** are zonal by default (or **regional** for synchronous replication across two zones in a region).
- **Instance templates** can be **global or regional**.
- **Managed instance groups (MIGs)** can be **zonal** or **regional** (regional spreads across zones, which is the exam-preferred HA design).
- **Images** and **snapshots** are global resources by default (with storage-location options).

4.3 Machine families, series, and types

4.3.1 Terminology

- **Machine family:** A broad workload category (general-purpose, compute-optimized, memory-optimized, storage-optimized, accelerator-optimized).
- **Machine series:** A generation/CPU platform within a family (e.g., E2, N2, C3).
- **Machine type:** A specific vCPU and memory configuration within a series (e.g., `n2-standard-4`).

4.3.2 Families and typical use

Family	Example series ⚠️	Typical workloads
General-purpose	**E2** (cost-optimized, shared-core options), **N2/N2D**, **N4**, **N1**, **C3/C4** (higher performance, GP-class), **Tau T2D/T2A** (scale-out, price-performance; T2A is Arm)	Web servers, app servers, small to medium databases, dev/test, microservices
Compute-optimized	**C2, C2D, H3**	HPC, game servers, high per-core performance, media transcoding
Memory-optimized	**M1, M2, M3, X4**	In-memory databases (SAP HANA), large analytics
Storage-optimized	**Z3**	High-throughput local SSD workloads: large SQL/NoSQL, data warehouses on VMs
Accelerator-optimized	**A2, A3, G2** (GPUs)	ML training/inference, rendering, scientific computing

> The series list evolves. For the exam, focus on the **family** and the **decision logic**, not memorizing every series.

4.3.3 Machine type flavors (general-purpose naming pattern)

Flavor	Name pattern	Memory per vCPU (roughly)	Use
standard	`n2-standard-4`	~4 GB	Balanced
highmem	`n2-highmem-4`	~8 GB	Memory-heavy apps, caches
highcpu	`n2-highcpu-4`	~1 GB	CPU-bound apps
shared-core	`e2-micro`, `e2-small`, `e2-medium`, `f1-micro` / `g1-small` (N1)	Fractional vCPU with burst	Tiny/dev/test, low-traffic
custom	`n2-custom-6-20480`	You choose	Exactly matching needs

4.3.4 Custom machine types

- Available on selected series (e.g., **E2, N2, N2D, N1**). Not available on all families.
- Rules (examples for N1/N2; ⚠️ verify per series): vCPU count is **1 or an even number**, memory must be within a **per-vCPU range** (for N1 roughly 0.9 – 6.5 GB per vCPU) and in multiples of 256 MB.
- **Extended memory** lets you exceed the per-vCPU limit at extra cost.
- **Pricing:** custom types cost slightly more per unit than predefined, but you avoid paying for unused resources.
- **Exam logic:** "*The application needs 6 vCPUs and 20 GB RAM and predefined sizes waste money*" → **custom machine type**.

4.3.5 Choosing the right type: decision flow

1. **GPU or specialized accelerator?** → Accelerator-optimized.

2. **Huge RAM (hundreds of GB to TB)?** → Memory-optimized.
3. **Very high single-thread speed or HPC?** → Compute-optimized.
4. **Heavy local-SSD throughput?** → Storage-optimized.
5. **Otherwise:** General-purpose. E2 for cost; N2/N4/C3 for performance; T2D for scale-out web.
6. **Odd sizing?** → Custom machine type.
7. **Fault-tolerant and cost-sensitive?** → Add **Spot** provisioning.

4.3.6 Rightsizing

- Compute Engine produces **machine type recommendations** and **idle VM recommendations** based on utilization (Recommender).
- Cloud Monitoring and the **Ops Agent** give memory metrics (not collected by default without an agent).
- Changing machine type requires the VM to be **stopped**, then edited, then started.

4.4 Provisioning models and pricing

Model	Description	Discount/Terms	Best for
Standard (on-demand)	Runs until you stop/delete it	Full price; per-second billing (1-minute minimum)	Stable production
Spot VMs	Excess capacity that Google can **preempt** at any time with ~**30 seconds** notice	Deep discount (commonly **60–91%** off) ⚠️	Batch, CI, stateless workers, fault-tolerant jobs
Preemptible VMs (legacy)	Older version of Spot, **max 24 hours** runtime	Similar discounts	Still seen in older questions; Spot is preferred
Flex-start / reservations ⚠️	Capacity assurance models for scarce resources like GPUs	Varies	Accelerator capacity planning

Spot behavior details:

- On preemption, VM goes to **STOPPED** or **DELETED** based on the **termination action** you set.
- No automatic restart, no live migration, **not covered by an SLA**.
- Use a **shutdown script** (best effort within the ~30 second window) to checkpoint.
- MIGs can use Spot VMs and **recreate** them when capacity returns.
- Not eligible for committed use discounts.

Discount programs:

Program	How it works	Notes
Sustained use discounts (SUDs)	Automatic discount for VMs running a large share of the month	Applies to some series (e.g., N1, N2, C2). **Not E2** (E2 is already priced low) ⚠️
Committed use discounts (CUDs)	1-year or 3-year commitment for resources (vCPU, memory, GPUs, local SSD) in a region (resource-based) or a dollar spend (**flexible CUD**)	Up to ~55% for most machine types, higher for memory-optimized (3-year) ⚠️

Program	How it works	Notes
Reservations	**Reserve capacity** in a zone so VMs can start when needed	Not a discount by itself; combine with CUDs
Spot	Described above	
Free tier	e2-micro in specific US regions with limits	

Exam logic: - Steady, predictable baseline → **CUD**. - Interruptible batch → **Spot**. - Guaranteed capacity for a launch event → **Reservation**. - Unpredictable workloads, no commitment → **On-demand + SUD** (where applicable).

4.5 Boot disks, images, and storage options

4.5.1 Images

Image type	Description
Public images	Google-provided and partner OS images (Debian, Ubuntu, RHEL, SLES, Windows Server, Container-Optimized OS, Rocky, etc.)
Custom images	Images you create from disks, snapshots, other images, or files in Cloud Storage. The basis of **golden images**.
Image families	A pointer to the **latest non-deprecated image** of a series. Create a new image in the same family and instances created "from family" automatically get the newest one.
Machine images	Capture an entire VM configuration plus all attached disks, metadata, and permissions. Good for cloning/backup of a **whole VM**.

Golden image workflow:

1. Build a VM → install and harden software.
2. Stop the VM.
3. Create a custom image into a **family** (e.g., `web-golden`).
4. Reference the family in instance templates.
5. Deprecate old images; roll out new ones via MIG rolling update.

Tools such as **Packer** (run from Cloud Build or another CI system) can automate this.

4.5.2 Persistent Disk and Hyperdisk types

Disk	Backing	Characteristics	Use
pd-standard	HDD	Cheapest, low IOPS	Sequential, archival-ish
pd-balanced	SSD	Default balance of price/performance	General workloads, boot disks

Disk	Backing	Characteristics	Use
pd-ssd	SSD	Higher IOPS and lower latency	Databases, latency-sensitive
pd-extreme	SSD	Highest provisioned IOPS (limited series)	Very high-IOPS databases
Hyperdisk (Balanced, Extreme, Throughput, ML) ⚠️	Next-gen block storage with **independently provisioned** IOPS/throughput	Newer series require Hyperdisk	Performance-tuned databases
Local SSD	Physically attached NVMe, **375 GB per device**	Very high IOPS and lowest latency, **ephemeral**: data is lost if the VM is stopped or deleted (survives live migration/reboot)	Caches, scratch, temp, shuffle

Key facts (frequent exam topics):

- Persistent disks are **network-attached**, **durable**, independent of the VM lifecycle (if `auto-delete` is off).
- Disk performance **scales with size** (and with vCPU count up to limits).
- You can **resize a disk up** (never down) while the VM runs. You then **grow the partition and file system** inside the OS.
- A disk can be attached **read-write to one VM** or **read-only to many** (multi-attach read-only; some Hyperdisk types allow multi-writer).
- **Regional Persistent Disk** replicates **synchronously across two zones** in a region (HA for stateful workloads; failover by force-attaching in the other zone).
- **Snapshots** are incremental, stored as global resources (multi-region or regional storage location), and can be scheduled via **snapshot schedules**. Use **application-consistent** snapshots where possible (quiesce the app, or use VSS on Windows).
- **Encryption:** Always encrypted at rest. Options: **Google-managed keys (default)**, **CMEK** (Cloud KMS), **CSEK** (customer-supplied keys; you supply the key on every use, and Google does not store it). ⚠️ CSEK has limitations; CMEK is the usual compliance answer.
- **Filestore** (managed NFS) and **Cloud Storage FUSE** are alternatives for shared file storage.

4.6 VM lifecycle and availability

4.6.1 States

`PROVISIONING → STAGING → RUNNING → (STOPPING → TERMINATED) / (SUSPENDING → SUSPENDED) / REPAIRING`

State	Billing
RUNNING	vCPU, memory, disks, IPs
TERMINATED (stopped)	**No vCPU/memory charges.** Still pay for disks, reserved/static IPs, and licensing in some cases
SUSPENDED	Memory state saved; pay for storage of the saved state and disks, not vCPU/memory
Deleted	Nothing (unless disks are retained)

4.6.2 Host maintenance and live migration

- **Live migration** moves a running VM to another host with **no reboot** during infrastructure maintenance.
- Setting `onHostMaintenance`: `MIGRATE` (default for most standard VMs) or `TERMINATE`.
- **GPU VMs**, **Spot VMs**, and some specialized configurations **cannot live migrate** → they must use `TERMINATE`.
- **Automatic restart** (`automaticRestart=true`) restarts the VM if it crashes or is terminated by a host event (not user-initiated shutdown, not Spot preemption).

4.6.3 SLA and HA basics

- A **single VM** has a lower SLA than a **multi-zone** design. For high availability use **regional MIGs** behind a **load balancer** across multiple zones. ⚠️ Check current SLA percentages in the docs; do not quote numbers unless verified.
- Combine **autohealing** (health check) to recreate failed VMs.

4.7 Customizing a VM

This is the "customization" the training objective refers to. A VM can be customized at **creation time**, **boot time**, and **run time**.

4.7.1 At creation: configuration options

Option	Notes
Machine type	Predefined or custom
CPU platform (minimum)	Pick a minimum CPU generation for instruction-set needs
Boot disk	Image, size, type, encryption, auto-delete behavior
Additional disks	PD, Hyperdisk, Local SSD
Networking	Network, subnet, internal/external IP, **network tags**, multiple NICs, IP forwarding, network tier
Service account and access scopes	Identity of the VM (see 4.7.4)
Security	Shielded VM, Confidential VM, OS Login, block project-wide SSH keys
Labels	For billing/reporting/organization
Metadata and scripts	Key-value metadata, startup/shutdown scripts
Availability policy	Standard vs Spot, maintenance behavior, auto-restart
Sole tenancy	Node affinity labels
Resource policies	Snapshot schedules, instance schedules (start/stop), placement policies
Display device, serial port, nested virtualization	For special cases

4.7.2 Metadata, startup scripts and the metadata server

- **Metadata** = key-value pairs at **instance** or **project** level (instance overrides project).

- Reachable from inside the VM at `http://metadata.google.internal/computeMetadata/v1/` (requires header `Metadata-Flavor: Google`).
- **Startup scripts** (`startup-script` or `startup-script-url` for a Cloud Storage object) run **every boot** as root. Make them **idempotent**.
- **Shutdown scripts** (`shutdown-script`) run during graceful shutdown (limited time; best-effort on preemption).
- Use **custom metadata** to pass configuration (e.g., environment name) to your scripts.
- The metadata server also provides **identity**: access tokens for the VM's service account, and **identity tokens** (JWT) to authenticate to services.

```
# From inside a VM
curl -H "Metadata-Flavor: Google" \
  http://metadata.google.internal/computeMetadata/v1/instance/name

curl -H "Metadata-Flavor: Google" \
  http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/email
```

4.7.3 Labels vs. network tags vs. resource tags

Concept	Purpose	Visible in billing?	Used in firewall?
Labels	Key-value metadata for organizing, filtering, **cost attribution**	**Yes**	No
Network tags	Strings that identify VMs for **firewall rules and routes**	No	**Yes**
Resource Manager tags (secure tags)	IAM-governed tags bound to resources, used by **organization policies, IAM conditions, and network firewall policies**	No	Yes (with network firewall policies)

> Exam trap: "*Track costs by team*" → **labels**. "*Apply firewall rule to a group of VMs*" → **network tags** (or SAs).

4.7.4 Service accounts and access scopes

- Every VM runs as a **service account**. The **default Compute Engine service account** is created automatically and (historically) has the **Editor** role on the project. ⚠️ Organization policies in newer orgs may no longer auto-grant it.
- **Best practice:** create a **dedicated service account per workload** with **only the roles it needs**. Do **not** use the default SA with Editor.
- **Access scopes** are a legacy mechanism that limits the API access of the VM's OAuth token. Current best practice: set the scope to `cloud-platform` and **control permissions with IAM**, not scopes.
- Changing the VM's service account requires the VM to be **stopped**.
- To attach a service account to a VM, the user needs `iam.serviceAccounts.actAs` (e.g., the **Service Account User** role) on that SA.

4.7.5 SSH access and OS Login

Method	How it works	Recommendation
Metadata-based SSH keys	Public keys are stored in instance or project metadata; the guest agent creates users	Legacy, harder to audit and revoke
OS Login	SSH access governed by **IAM** (`roles/compute.osLogin`, `roles/compute.osAdminLogin`), supports **2-step verification**, maps to Google identity, audit logged	**Recommended** (`enable-oslogin=TRUE` in metadata)
IAP TCP forwarding	Tunnels SSH/RDP over HTTPS through IAP, needs **no public IP**	**Recommended** for private VMs (role: `roles/iap.tunnelResourceAccessor`)
Bastion host	A hardened VM with a public IP as a jump host	Acceptable, but IAP is usually better
Serial console	Out-of-band access for troubleshooting boot problems	Disabled by default; enable carefully

4.7.6 Shielded VM, Confidential VM, sole-tenancy

Feature	What it protects	Details
Shielded VM	Boot integrity (rootkits, bootkits)	**Secure Boot**, **vTPM**, **Integrity Monitoring**. On by default for supported images.
Confidential VM	**Data in use** (memory encryption)	Uses hardware memory encryption (AMD SEV family, Intel TDX on supported series) ⚠️
Sole-tenant nodes	Physical host isolation	VMs run on hardware dedicated to you. Used for compliance, **BYOL licensing** (per-core/per-socket licenses), and performance isolation. Use **node affinity** labels.
Organization policies	Guardrails	e.g., `constraints/compute.requireShieldedVm`, `constraints/compute.vmExternalIpAccess` (restrict external IPs), `constraints/compute.requireOsLogin`, `constraints/compute.disableSerialPortAccess`, `constraints/gcp.resourceLocations`

4.7.7 Networking customization on a VM

- **Multiple NICs:** each NIC attaches to a **different VPC** (a VM can be multi-homed across VPCs, which is the foundation for NVAs/firewall appliances). The number of NICs depends on the machine type (up to 8).
- **IP forwarding (`canIpForward`)**: required for VMs that route traffic not addressed to them (NVAs, NAT instances).
- **Alias IP ranges:** assign additional IPs from a subnet range (or secondary range) to the VM for containers or multiple services.
- **Egress bandwidth** is limited by machine type (more vCPUs → higher cap); **Tier_1 networking** gives higher bandwidth on supported series. ⚠️ Verify per series.

4.7.8 OS and fleet management (VM Manager / OS Config)

- **OS patch management:** schedule and orchestrate patching across fleets.
- **OS inventory:** collect installed packages.
- **Guest policies / OS policies:** enforce desired state (software installed, config files).
- **Ops Agent:** collects metrics and logs (including memory and disk utilization) and sends them to Cloud Monitoring and Cloud Logging.

4.7.9 Instance schedules

- **Instance schedules** (resource policy) **start and stop** VMs on a schedule (e.g., stop dev VMs at 8 pm). A common **cost optimization** answer.

4.8 Instance templates and instance groups

4.8.1 Instance templates

- An **instance template** is an **immutable** blueprint (machine type, image, disks, network, metadata, SA, labels, tags).
- To change it, **create a new template** (you cannot edit one).
- Templates can be **global** or **regional**. They are used by **MIGs** and can also create standalone VMs.

4.8.2 Managed vs. unmanaged instance groups

Type	Description
Managed Instance Group (MIG)	Identical VMs created from a template. Supports **autoscaling, autohealing, rolling updates, regional deployment, load balancer integration**. Optional **stateful** configuration (preserve disks/IPs/metadata per instance).
Unmanaged instance group	A manual collection of possibly different VMs. No autoscaling/autohealing. Used for legacy/special LB cases.

4.8.3 MIG features in detail

Feature	Details
Location	**Zonal** (single zone) or **Regional** (spread across up to 3+ zones; **recommended for HA**)
Autoscaling signals	CPU utilization, HTTP load balancing serving capacity, Cloud Monitoring custom/standard metrics, schedules (scale by time); optionally **predictive autoscaling** for CPU-based scaling
Autoscaler settings	Min/max replicas, target utilization, **cool-down period** (time for a new VM to initialize before its metrics are used), scale-in controls
Autohealing	An **application-level health check** recreates unhealthy VMs. Needs **initial delay** to allow boot time. Needs firewall rule for health check ranges.
Rolling updates	Replace instances gradually when you set a new template. Controls: **max surge**, **max unavailable**, **minimal action** (refresh/restart/replace), **replacement method** (substitute/recreate).
Canary updates	Roll the new template to a **percentage** of instances (`--canary-version ... target-size=20%`)

Feature	Details
Distribution policy	`EVEN` or `BALANCED` distribution across zones (regional MIGs); can target specific zones
Stateful MIG	Preserve disks, internal IPs, metadata when instances are recreated (e.g., for databases)
Update policy types	`PROACTIVE` (automatically rolls out) or `OPPORTUNISTIC` (applies only when instances are recreated/replaced)

MIG vs. health checks for load balancers: The **LB health check** decides whether to **send traffic** to a VM; the **autohealing health check** decides whether to **recreate** the VM. They can be the same or different checks.

4.9 Cost optimization summary for Compute Engine

1. **Rightsize** using recommendations. Use **custom machine types** where needed.
2. **Spot VMs** for fault-tolerant jobs.
3. **CUDs** for steady-state baseline. **Reservations** for guaranteed capacity.
4. **Stop or delete idle VMs.** Use **instance schedules.**
5. **Cheaper disk types** where performance is not needed. Delete unattached disks and old snapshots.
6. **Release unused static IPs.**
7. **Autoscale** to match demand, and **regional** deployment close to users.
8. Use **labels** and **billing export to BigQuery** for cost visibility; budgets and alerts for governance.

4.10 Deep dive: Compute Engine architecture under the hood

4.10.1 Virtualization stack

Component	Facts
Hypervisor	KVM-based, hardened and customized by Google. Each VM runs in a sandboxed process with a minimal device model to shrink the attack surface.
Hardware offload	Newer series offload networking and storage I/O to dedicated hardware (Titanium) ⚠️, leaving more host CPU for your workload and enabling higher network/disk throughput.
vCPU	A **vCPU is a hardware hyper-thread** (one logical processor). Two vCPUs ≈ one physical core with SMT. **Exception:** Tau **T2D** (and Arm **T2A**) map one vCPU to a **full physical core** (no SMT), which is why they excel at scale-out price-performance.
Memory	Dedicated to the VM (no memory ballooning/overcommit for standard VMs).
Shared-core / E2	`e2-micro/small/medium` get a **fraction of a vCPU with burst** capability. E2 uses **dynamic resource management**, which can rebalance resources across VMs on a host, so performance is slightly less predictable than N2/C-series.
NUMA	Large VMs span NUMA nodes. Memory-optimized and HPC workloads should be NUMA-aware, and `c2` / `m` families expose topology to the guest.

Component	Facts
Metadata server	Link-local service `169.254.169.254` reachable only from the VM itself. Supplies configuration, identity tokens, and scheduling events.

4.10.2 Live migration (how maintenance is invisible)

1. Google schedules host maintenance (firmware, kernel, hardware swap).
2. **Pre-copy phase:** VM memory pages are copied to the target host while the VM keeps running (some performance dip, "brownout").
3. **Blackout:** the VM is paused very briefly (typically sub-second to a few seconds ⚠️) for the final state sync.
4. **Cutover:** VM resumes on the target host with the **same IPs, disks, and state**. No reboot.

Detail	Behavior
Notification	The metadata server key `instance/maintenance-event` changes (`MIGRATE_ON_HOST_MAINTENANCE` , `TERMINATE_ON_HOST_MAINTENANCE` , `NONE`) shortly before the event, which apps can watch to drain gracefully.
`onHostMaintenance=MIGRATE`	Default for standard VMs.
`onHostMaintenance=TERMINATE`	Required for **GPU VMs**, **Spot VMs**, and some specialized series; the VM is stopped (and restarted if `automaticRestart` is true).
What can break it	Very large VMs with heavy memory writes may see longer blackouts; local SSD is preserved across live migration.
Sole-tenant	Maintenance policy options: default (migrate), restart in place, or migrate within node group.

4.10.3 Host errors and automatic recovery

- If the **physical host fails** (not planned maintenance), the VM **crashes** and is **restarted automatically** on a healthy host if `automaticRestart=true`. Data in memory is lost. Persistent disks survive.
- Therefore **live migration protects against planned maintenance; autohealing/MIGs and multi-zone design protect against unplanned failures.**

4.10.4 Placement policies

Policy	Purpose
Spread	Spread VMs across distinct racks/hosts to reduce correlated failure (up to a limited count per policy ⚠️)
Compact	Place VMs physically close for lowest network latency (HPC/tightly coupled workloads)
Sole-tenant node groups	Dedicated hosts you rent

4.11 Deep dive: machine series comparison

> Numbers are representative and change as new generations appear (⚠️). Use them to understand *relative* positioning.

Series	CPU platform (typical)	Max vCPUs / memory (approx.)	Positioning	Notable features
E2	Mixed Intel/AMD, **dynamic resource management**	32 vCPU / 128 GB	**Lowest cost**, general workloads	Shared-core options, **custom types**, no sustained use discounts, no GPUs, no local SSD (limited)
N1	Older Intel (Skylake and earlier)	96 vCPU / 624 GB	Legacy general-purpose	Custom types, GPUs attachable (T4, V100, etc. ⚠️), SUDs
N2	Intel Cascade Lake / Ice Lake	128 vCPU / 864 GB	Balanced price/performance	Custom types, local SSD, SUDs, Confidential VM (some)
N2D	AMD EPYC	224 vCPU / 896 GB	Lower cost than N2 for similar performance	Confidential VM (AMD SEV)
N4 / C3 / C4	Newer Intel (Emerald/Sapphire Rapids)	Up to ~192/176 vCPU ⚠️	Next-gen general purpose with higher per-core performance	Hyperdisk, Titanium offload, higher network bandwidth
T2D	AMD EPYC (1 vCPU = 1 core)	60 vCPU / 240 GB	**Scale-out** web/microservices, best price-performance	No custom types, no GPU/local SSD
T2A	Arm (Ampere Altra)	48 vCPU / 192 GB	Arm workloads, scale-out	Requires **arm64** images/binaries
C2 / C2D	Intel Cascade Lake / AMD EPYC	60 / 112 vCPU	**Compute-optimized**: high clock, HPC, gaming, EDA	Placement policies, Tier_1 networking, local SSD
H3	Intel Sapphire Rapids	88 vCPU / 352 GB	HPC and compute-bound	Dedicated for tightly coupled HPC ⚠️
M1 / M2 / M3	Intel	Up to ~160 / 416 vCPU; **up to ~4 TB / ~12 TB** RAM	**Memory-optimized**: in-memory DBs, SAP HANA	Certified for SAP HANA ⚠️
Z3	Intel	Large local SSD capacity	**Storage-optimized**: dense local NVMe	
A2 / A3 / G2	Intel/AMD + NVIDIA GPUs (A100, H100, L4)	GPUs attached by series	**Accelerator-optimized**: ML, rendering	`TERMINATE` maintenance, GPU quotas needed

Selecting a series (decision logic)

1. **Is the software Arm-compatible and horizontally scalable?** → T2A for the best price-performance.
2. **Scale-out stateless, x86?** → **T2D** or **E2** (E2 if cost is the top priority and workload is not latency-sensitive).
3. **General production, needs custom shapes, SUDs, local SSD, Confidential computing?** → **N2/N2D** (or N4/C3 if newer features and performance matter).

4. **Per-core speed matters (single-thread bound, HPC, game server, license per core)? → C2/C3/H3**. Fewer, faster cores can reduce licensing cost.
5. **RAM per vCPU above ~8 GB (large in-memory dataset)? → M-series** (or N2 highmem for moderate sizes).
6. **Heavy local storage throughput? → Z3** or N2/C3 with Local SSD.
7. **GPU?** → A/G series (check regional availability and quotas first).

4.11.1 Quotas and capacity

- **Quotas** are per project and per region (CPUs, GPUs, in-use IPs, disks, instances). A VM creation fails with `QUOTA_EXCEEDED` when exceeded. Request increases early for big launches.
- **Capacity** is separate from quota: a zone can be **out of capacity** (`ZONE_RESOURCE_POOL_EXHAUSTED`) for a machine type. Fixes: try another zone, another machine type, use **reservations** for guaranteed capacity, or use a regional MIG with **any-shape / flexible distribution** ⚠️.

4.12 Deep dive: disks and storage performance

4.12.1 How persistent disk performance is determined

Persistent Disk (PD) is **network-attached block storage**, so performance depends on **three** limits, and you get the **lowest**:

1. **Per-disk limit**, which scales with **provisioned size** (GB) up to the disk type's maximum.
2. **Per-VM limit**, which depends on the **machine type and vCPU count** (more vCPUs → higher aggregate IOPS/throughput up to a cap).
3. **Workload pattern** (I/O size, queue depth, read/write mix).

Disk type	Relative IOPS per GB ⚠️	Throughput per GB ⚠️	Typical latency	Typical use
pd-standard (HDD)	~0.75 read / ~1.5 write	~0.12 MB/s	Highest	Large sequential, cold data, boot disks that are rarely stressed
pd-balanced (SSD)	~6	~0.28 MB/s	Low	Default for most workloads (3,000 IOPS baseline at small sizes ⚠️)
pd-ssd	~30	~0.48 MB/s	Lowest PD	Databases, latency-sensitive apps
pd-extreme	**Provisioned IOPS** you choose	Higher	Lowest PD	Very high IOPS databases (limited to some series)
Hyperdisk Balanced	**Provisioned IOPS and throughput independent of size**	Independent	Low	Modern default for newer series, tune cost/perf without over-sizing capacity
Hyperdisk Extreme	Highest provisioned IOPS		Lowest	Mission-critical databases
Hyperdisk Throughput	Throughput-optimized	High	Moderate	Analytics, big data scans
Local SSD	Hundreds of thousands to millions IOPS (aggregate)	GB/s	Microseconds	Caches, temp/scratch, replicated NoSQL

Architectural implications:

- **Size for IOPS, not just capacity.** A 100 GB pd-balanced disk has far lower IOPS than a 1 TB pd-balanced disk. For performance-bound workloads, either provision more GB, switch to pd-ssd/Hyperdisk, or **stripe** several disks (LVM/RAID 0) inside the VM.
- **More vCPUs unlock more disk performance.** A big pd-ssd on a 2-vCPU VM will be capped by the VM.
- **Hyperdisk breaks the size ↔ performance coupling**, which is why it is the direction of travel for databases.
- **Boot disk performance** matters for autoscaling startup time. Use pd-balanced or larger boot disks and golden images to shorten boot.

4.12.2 Local SSD in depth

Property	Detail
Size	**375 GB per device**, multiple devices per VM (series-dependent maximum)
Interface	NVMe (or SCSI on older)
Durability	**Ephemeral:** data survives guest reboot and live migration, but is **lost** when the VM is stopped, deleted, preempted, or the host fails
Encryption	Always encrypted at rest
Backup	Not snapshot-able. You must replicate at the application level
Typical designs	Cassandra/Elasticsearch nodes with replication factor ≥ 3; caching tier; shuffle space for Spark; swap/scratch. **Never** the sole copy of important data

4.12.3 Snapshots, images, machine images: when to use which

	Snapshot	Custom image	Machine image
Captures	One disk's data	One boot disk (usually)	**Entire VM**: config, metadata, permissions, **all disks**
Incremental?	**Yes** (after the first)	No (full image)	Yes (uses snapshot-like storage)
Typical use	Backup, restore, DR, move disk across zones/regions	**Golden image** for fleets, templates	Back up or **clone a whole VM**
Create from running VM?	Yes (crash-consistent; app-consistent with quiescing)	Best done from a **stopped** VM or a snapshot	Yes
Scope	Global resource, storage location configurable	Global resource	Global resource
Scheduling	**Snapshot schedules** (resource policy)	Via pipeline	Manual / automation
Sharing	IAM across projects	Share via IAM or org policy `trustedImageProjects`	IAM
Archive option	**Archive snapshot type** (cheaper, 90-day minimum) ⚠️	n/a	n/a

Snapshot internals (conceptually):

- First snapshot copies all used blocks; later ones copy only **changed blocks**.

- Deleting an older snapshot **does not break** newer ones; Google merges required blocks forward automatically.
- Snapshot **frequency** and **retention** drive both RPO and cost. A more frequent schedule means a smaller RPO and typically small incremental cost.
- **Consistency levels:** *crash-consistent* (like pulling the power cord), *application-consistent* (quiesce with `fsfreeze`/VSS or database hooks), *multi-disk consistency* (use **consistency groups** or machine images/stop writes) ⚠️.

4.12.4 Persistent disk resilience options

Option	RPO	RTO	Notes
Zonal PD + snapshots (hourly/daily)	Up to snapshot interval	Minutes–hours (restore)	Cheapest DR
Regional PD	≈ **0** (sync replication across 2 zones)	Minutes (force-attach to standby VM)	Needs a VM in the second zone to attach
Asynchronous PD replication ⚠️	Seconds–minutes	Minutes	Cross-region DR for PD
Application-level replication (database HA, Cloud SQL HA)	≈ 0	Seconds–minutes	Best for databases. Prefer managed services
Machine images + scripted rebuild (IaC)	Backup interval	Hour(s)	Rebuild path for stateless VMs

4.12.5 Disk encryption options in depth

Option	Who holds the key	Operational burden	When to choose
Google-managed (default)	Google	None	Default. Meets most needs
CMEK (Cloud KMS)	You control the key in Cloud KMS (rotation, disable, destroy); Google uses it	Moderate (IAM on the key; **disabling the key makes the disk unreadable**)	Compliance requiring customer-controlled keys. **The usual exam answer**
CMEK with Cloud HSM / External Key Manager	Key in HSM or in an external system you own	Higher	Strict regulatory/sovereignty needs
CSEK (customer-supplied)	You provide the key with each operation; Google doesn't store it	**High**: lose the key and the data is gone; limitations on features (e.g., not usable with all disk types or MIG automation) ⚠️	Rare. Choose only when explicitly required

4.13 Deep dive: the guest environment, metadata and boot sequence

4.13.1 What happens when a VM boots

```
1. API call → control plane reserves capacity, allocates IPs/disks   (PROVISIONING)
2. Resources attached, VM image is staged                              (STAGING)
3. Firmware (UEFI, Shielded VM measurements) → bootloader → kernel     (RUNNING begins)
4. Guest environment starts (google-guest-agent, OS Login/NSS modules)
```

```
5. Network configured via DHCP → internal IP, gateway, DNS 169.254.169.254
6. Metadata scripts runner executes `startup-script` (every boot, as root)
7. Application starts; health checks begin passing
```

Note that "RUNNING" in the API only means the VM has been **started**. It does not mean your application is **ready**. That is why MIG autohealing has an **initial delay** and why **health checks** (not API state) define readiness.

4.13.2 The guest environment (Linux and Windows)

Component	Role
Guest agent (`google-guest-agent`)	Applies SSH keys/OS Login configuration, sets hostname, handles network interface config (multi-NIC, IP forwarding, alias IPs), and exposes guest attributes.
Metadata script runner	Executes startup, shutdown, and Windows PowerShell scripts.
OS Login modules	PAM/NSS modules that map IAM identities to Linux accounts.
Ops Agent (optional but recommended)	Collects logs and system metrics (**memory, disk, processes** are *not* available without it).
OS Config agent	Patch management, inventory, OS policies (VM Manager).
Windows	Uses Sysprep for image generalization; password reset via the guest agent or console; RDP via IAP.

Public images include the guest environment. **Custom images from foreign sources** (imported VMs) need it installed, or SSH/OS Login/startup scripts may silently fail.

4.13.3 Metadata server in depth

Area	Detail
Endpoint	`http://metadata.google.internal/computeMetadata/v1/` (resolves to 169.254.169.254)
Required header	`Metadata-Flavor: Google` (protects against SSRF from simple web requests)
Scopes	**Instance** metadata (project/instance attributes, name, zone, tags, network interfaces, disks, service accounts) and **project** metadata (inherited by all VMs; instance values override)
Dynamic watch	Append `?wait_for_change=true` to long-poll for changes (e.g., `instance/maintenance-event`, `instance/preempted`)
Spot signals	`instance/preempted` flips to `TRUE` when preemption starts (~30 s notice)
Identity	`instance/service-accounts/default/token` → OAuth access token; `.../identity?audience=URL` → signed ID token (JWT) for service-to-service auth (Cloud Run, IAP, etc.)
Guest attributes	Writable key-values from inside the VM, used for readiness handshakes (needs `enable-guest-attributes=TRUE`)
Common metadata keys	`startup-script`, `startup-script-url`, `shutdown-script`, `enable-oslogin`, `block-project-ssh-keys`, `serial-port-enable`, plus your own
Security caution	Any process on the VM (including a compromised app) can request the SA token. **Least privilege on the SA** is the mitigation, and SSRF in an app can leak tokens.

Cheat commands

```
# Is my Spot VM being preempted?
curl -s -H "Metadata-Flavor: Google" \
  http://metadata.google.internal/computeMetadata/v1/instance/preempted

# Wait for a maintenance event
curl -s -H "Metadata-Flavor: Google" \
  "http://metadata.google.internal/computeMetadata/v1/instance/maintenance-event?wait_for_change=true"

# Get an identity token for calling a Cloud Run service
curl -s -H "Metadata-Flavor: Google" \
  "http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/identity?
audience=https://my-service-xyz.a.run.app"
```

4.13.4 Startup script best practices

Practice	Reason
Idempotent (safe to run on every boot)	Runs at **every** boot, not just the first
Fail loudly, log to a file and to Cloud Logging	Silent failures cause unhealthy instances
Keep scripts **short**; bake heavy installs into **images**	Faster scale-out, no dependency on package repos at boot
Pull secrets from **Secret Manager** at runtime, never hardcode in metadata	Instance metadata is readable by anyone with VM read access and by any process on the VM
Use `startup-script-url` (Cloud Storage object) for large/shared scripts	Version control, IAM on the script
Check exit codes and use `set -euo pipefail`	Avoid partial configuration
Troubleshoot with `sudo journalctl -u google-startup-scripts.service` and the serial console	The first places to look when "my script didn't run"

4.14 Deep dive: identity, access and security of VMs

4.14.1 Two directions of access: who can touch the VM vs. what the VM can touch

Direction	Question	Mechanisms
Humans/automation → VM (administering)	Who can create/modify the VM, SSH to it, snapshot it?	IAM roles (`compute.instanceAdmin.v1`, `compute.osLogin`), IAP, OS Login, organization policies
VM → Google Cloud APIs (workload identity)	What can the application on the VM do in Google Cloud?	The VM's **service account**, IAM roles on target resources, access scopes (legacy ceiling)
VM → other VMs (network)	Who can the VM talk to?	Firewall rules (SA/tag/secure tag), routes
Internet/others → VM	Who can reach it?	External IPs, LBs, Cloud Armor, firewall

> Confusing these directions is the most common source of wrong exam answers, so classify each requirement into one of these four rows before choosing a solution.

4.14.2 Service account best practices for VMs

Practice	Detail
One SA per workload/tier	Fine-grained permissions and firewall targeting
Don't use the default compute SA (`PROJECT_NUMBER-compute@developer.gserviceaccount.com`)	Historically has **Editor**, so any compromised VM becomes a project-wide risk. Organization policies can stop auto-granting Editor (`iam.automaticIamGrantsForDefaultServiceAccounts`) ⚠️
Scope = `cloud-platform`**, control with IAM**	Scopes are a legacy coarse filter; the effective permission is the **intersection** of IAM roles and scopes, so a narrow scope can hide an IAM grant and cause confusing 403s
No downloaded SA keys	Use the metadata server credentials; block key creation with org policy
Grant `iam.serviceAccountUser` **narrowly**	It lets a user make a VM run **as** that SA, which effectively delegates the SA's power
Short-lived tokens / impersonation for automation	Avoids long-lived secrets
Cross-project: a VM can use an SA from another project (needs `iam.disableCrossProjectServiceAccountUsage` considerations) ⚠️	

4.14.3 Shielded VM, in more depth

Feature	What it does	Detects/prevents
Secure Boot	UEFI verifies the digital signature of boot components (bootloader, kernel, drivers)	Unsigned or tampered boot software, bootkits
vTPM (virtual Trusted Platform Module)	Measures boot components into PCRs (**measured boot**), enables sealing secrets to a known good state	Tamper evidence; used by integrity monitoring and attestation
Integrity monitoring	Compares current boot measurements to a **baseline** and logs failures to Cloud Logging / Cloud Monitoring	Rootkits, unauthorized kernel modules after patching without baseline update

Remember to **update the integrity baseline** after legitimate kernel/driver updates, or you get false alerts. Shielded VM is **on by default** for supported public images and recommended for all, with an org policy to enforce it.

4.14.4 Confidential VM, in more depth

Aspect	Detail
What	Memory (data in use) is **encrypted in hardware** using keys the hypervisor and Google cannot read
Technologies	AMD SEV / SEV-SNP (N2D, C2D), Intel TDX (C3), others by series ⚠️
Performance	Small overhead, typically a few percent
Limits	Not every series, region, or feature (e.g., live migration support varies) ⚠️

Aspect	Detail
Use when	Regulated data (finance/health) where "no cloud provider access, even to memory" is required. A **Confidential Computing** requirement in the exam text means **Confidential VM** (or Confidential GKE Nodes, Dataproc, etc.)

4.14.5 OS Login and SSH in depth

Item	Detail
Enablement	`enable-oslogin=TRUE` in project or instance metadata (or org policy `compute.requireOsLogin`)
Identity mapping	Posix account derived from the Google identity (or **Cloud Identity/Workspace** profile); service accounts and external users supported with the right roles
Roles	`roles/compute.osLogin` (no sudo), `roles/compute.osAdminLogin` (sudo), `roles/compute.osLoginExternalUser` (for users from other organizations, granted at org level)
2-step verification	Enforce with `enable-oslogin-2fa=TRUE`
SSH keys	Stored in the **user's OS Login profile**, not in instance metadata
Audit	Logins appear in **Cloud Audit Logs** and system logs
Revocation	Remove the IAM role → access removed immediately (no key cleanup)
Interaction with metadata keys	When OS Login is on, **metadata-based SSH keys are ignored**
IAP + OS Login	**Recommended combination**: network access via IAP, authorization via OS Login

4.14.6 Patch, vulnerability and fleet management

Capability	Tool
Patch orchestration (windows, rollouts by zone, pre/post scripts)	**VM Manager: OS patch management**
Software inventory	**OS inventory management**
Vulnerability reports	VM Manager vulnerability reports → **Security Command Center**
Desired-state config	**OS policies / guest policies**
Image hardening	Hardened/**Shielded** images, CIS-hardened images from Marketplace, trusted image projects (org policy)
Immutable replacement	Rebuild golden image → MIG rolling update

4.15 Deep dive: managed instance groups (MIGs)

4.15.1 What a MIG controls

A MIG continuously reconciles **desired state**: "*N instances, each created from version(s) of the template, spread across zones, all healthy, within autoscaler bounds.*" If an instance is deleted, crashes, fails its health check, or a zone goes down, the MIG **recreates** to restore the target size.

Concept	Detail
Target size	Set by you, or by the autoscaler between min and max
Versions	One or two (**canary**) template versions with target sizes
Instance configs (stateful)	Per-instance preserved state: **disks, internal/external IPs, metadata**
Distribution	Regional MIG: `EVEN` (default; spread equally across selected zones) or `BALANCED` / `ANY` ⚠️ shapes for capacity flexibility
Named ports	Map port names to numbers for load-balancer backends
Health check & initial delay	Autohealing configuration
Update policy	Type, surge, unavailable, minimal action, replacement method
Instance lifecycle policy	What to do on failure/ repair, `forceUpdateOnRepair` ⚠️

4.15.2 Regional vs zonal MIG

	Zonal MIG	Regional MIG
Zones	1	2–3+ zones in a region
Survives a zone outage?	**No**	**Yes** (remaining zones serve; autoscaler adds capacity in healthy zones)
Capacity headroom	Single pool	Spreads risk across pools
Recommended for HA production	No	**Yes**
Sizing rule of thumb		Provision so that **losing one zone** leaves enough capacity (e.g., with 3 zones, run at ≤ 66% utilization per zone, or set utilization targets accordingly)

4.15.3 Autoscaler algorithm in depth

Concept	Behavior
Signals	CPU utilization, HTTP LB utilization (serving capacity), Cloud Monitoring metrics (per-instance or per-group, e.g., queue depth), **schedules** (scaling schedules with min replicas at set times)
Multiple signals	The autoscaler computes a **recommended size for each signal** and uses the **largest**
CPU example	If 10 VMs run at 90% CPU and the target is 60%, desired ≈ 10 × (0.9 / 0.6) = **15 VMs**
Cool-down (initialization) period	Time the autoscaler ignores a new instance's metrics while it boots. Set it to **at least** your real startup time, or the autoscaler over-scales because new instances look idle/busy before they are ready
Scale-in stabilization	The autoscaler uses the **highest recommended size over the last ~10 minutes** when scaling in, to avoid flapping ⚠️
Scale-in controls	Limit how many VMs can be removed per time window (protects against sudden big drops)
Predictive autoscaling	For **CPU-based, regional** MIGs, uses history to scale **ahead** of recurring daily/weekly load

Concept	Behavior
Scale to zero	A MIG can scale to 0 with **min replicas = 0** when using certain signals (e.g., schedules or queue metrics) ⚠️
Which metric to choose	CPU for CPU-bound web; **LB utilization** when requests per instance matter; **queue length per worker** (custom metric) for batch/pull workers; **schedules** for predictable events

Common pitfalls

- Autoscaling on CPU for an **I/O-bound or memory-bound** app doesn't trigger when it should. Pick a signal that reflects the bottleneck.
- Cool-down shorter than boot time → **overshoot**.
- Min replicas too low for failover capacity.
- Max replicas bumping into **quotas** (CPUs/IPs/disks).

4.15.4 Health checks: LB vs autohealing

	Load balancer health check	Autohealing health check
Decides	Whether to **send traffic** to the instance	Whether to **recreate** the instance
Action on failure	Removes backend from rotation (no restart)	MIG deletes/recreates VM
Timing	Aggressive is acceptable	Be **conservative** (avoid killing instances in brief spikes); set **initial delay** ≥ boot + warm-up
Protocols	HTTP, HTTPS, HTTP/2, TCP, SSL, gRPC	Same
Defaults (gcloud)	interval 5 s, timeout 5 s, healthy 2, unhealthy 2 ⚠️	Same parameters, different consequences
Firewall	Must allow `130.211.0.0/22` and `35.191.0.0/16` (and for regional proxy-based LBs, the **proxy-only subnet**)	Same probe ranges
Recommendation	Separate, stricter app-level endpoint for readiness; a more forgiving check for autohealing	

Connection draining: when an instance is removed (scale-in, update, delete), LB **connection draining** lets in-flight requests finish for up to a configured timeout (default 300 s ⚠️).

4.15.5 Rolling update mechanics with a worked example

Setup: Regional MIG, target size **10**, new template v2. Policy: `maxSurge=3`, `maxUnavailable=0`, replacement method `SUBSTITUTE`.

Step	State	Notes
0	10 × v1	Start
1	10 × v1 + **3 × v2 (creating)**	Surge adds up to 3 extra instances; capacity never drops below 10
2	3 × v2 healthy → delete 3 × v1	Waits for health (initial delay and health check)
3	7 × v1 + 3 × v2 → create 3 more v2	Repeat
…		

Step	State	Notes
N	10 × v2	Done. Total VMs never exceeded **13** and never fell below **10**

Parameter	Effect	Trade-off
maxSurge	Extra instances allowed above target during update	Faster, higher temporary cost/quota use
maxUnavailable	Instances allowed to be down below target	Faster/cheaper, but reduced capacity
Minimal action	`NONE` / `REFRESH` / `RESTART` / `REPLACE` (the least disruptive action the update may take)	Metadata-only changes can be applied by REFRESH without recreating
Most disruptive allowed action	Cap on how disruptive an update may be	Safety guardrail
Replacement method	`SUBSTITUTE` (new names/IDs) vs `RECREATE` (same names/IPs; used for stateful)	Pick RECREATE when preserving instance identity matters
Update type	`PROACTIVE` (rolls automatically) vs `OPPORTUNISTIC` (only when instances are naturally recreated or you trigger it)	Use OPPORTUNISTIC to avoid disrupting during business hours
Canary	Two versions with a target size (e.g., 20%)	Validate before full rollout; **rollback** = point the MIG back to v1

Rollback: start a new update pointing to the previous template. (There is no "undo" button; **immutable templates make rollback trivial** if you keep the old template.)

4.15.6 Stateful MIGs

- For workloads that need **stable identity** (databases, message brokers, licensed software bound to IP or disk).
- Configure which parts of state are **preserved per instance**: disks (by device name), internal IP, external IP, metadata.
- Autohealing/update recreates the VM but **reattaches** the preserved disks and addresses.
- Trade-offs: slower updates, careful handling of disk lifecycle, usually **zonal-pinned** disks. Managed databases are still preferred where possible.

4.15.7 MIG + load balancer integration

- The MIG is a **backend** of a backend service. Set **named ports** (e.g., `http:80`).
- Backend **balancing mode:** `UTILIZATION`, `RATE`, or `CONNECTION`, with capacity scaler settings.
- **Autoscaling on LB utilization** uses the backend's configured serving capacity.
- Combine: **regional MIG + health check + autoscaler + LB + connection draining** = the canonical resilient web tier (expect this on the exam).

4.16 Deep dive: availability, SLAs and resilience patterns on Compute Engine

4.16.1 Failure domains

Failure	Impact	Mitigation
Process crash	App down	Supervisor (systemd), health check, autohealing
VM crash / host failure	One VM lost (restarts on another host)	Automatic restart, MIGs, multiple instances
Planned host maintenance	None (live migration) or restart (GPU/Spot)	Right `onHostMaintenance`, rolling design
Zone outage	All zonal resources in that zone	**Regional MIG**, regional PD, cross-zone LB
Region outage	Region's resources	**Multi-region** design, global LB, replicated data, DR plan
Bad deployment	Logical failure	Canary, rolling update, rollback
Data corruption / deletion	Data loss	**Snapshots, backups**, versioning (HA ≠ backup)
Credential compromise	Security	Least privilege, short-lived creds, logging

> **HA is not backup.** Replication copies mistakes and deletions instantly. Snapshots/backups protect against logical errors.

4.16.2 SLA concepts ⚠️ (verify current numbers before quoting)

- Compute Engine publishes **different monthly uptime targets** for: **single instance** (lower), **instances in multiple zones** behind load balancing (higher, up to the "four nines" class ⚠️), and Spot (**no SLA**).
- **Composite SLA:** if components are in **series**, multiply availabilities (e.g., LB 99.99% × MIG 99.99% × DB 99.95% ≈ 99.93%). If redundant components are in **parallel**, unavailability multiplies (two independent 99% components → $1 - 0.01^2 = 99.99\%$).
- The *design* determines whether you actually reach the SLA, since SLAs describe credits, not guarantees.

4.16.3 RTO and RPO mapping

Target	Pattern
RTO hours, RPO hours	Backups/snapshots, rebuild with IaC (**backup & restore**)
RTO minutes–hour, RPO minutes	**Pilot light / warm standby** in another region (small footprint ready, scale up on failover)
RTO seconds–minutes, RPO ≈ 0	**Active/active** multi-zone/region with synchronous or near-synchronous replication

4.17 Deep dive: cost optimization, discounts and worked comparisons

4.17.1 Discount interplay

Discount	Applies to	Commitment	Automatic?	Stacks with
Sustained use (SUD)	Eligible series (N1, N2, N2D, C2, M-series… not E2) ⚠️	None	**Yes**, increases with the portion of the month a resource runs	Not stacked with CUD on the same usage (CUD usage is covered first)
Resource-based CUD	vCPU, memory, GPU, local SSD **in a region**, for eligible series	**1 or 3 years**	No (purchase)	Applies **before** SUD; discount up to ~55% (higher for memory-optimized) ⚠️
Flexible (spend-based) CUD	A **dollar-per-hour** commitment across eligible series/regions	1 or 3 years	No	Flexibility across machine families, regions
Spot	Eligible VMs	None	Chosen at creation	Not eligible for CUD coverage
Free tier	e2-micro (specific regions)	None	Yes	
Reservations	Capacity	Optional	No	**Do not discount**; combine with CUD to cover cost

Rules of thumb

- Commit to **what you are sure to use** (baseline, ~70–80% of steady usage), not to peak.
- **Resource-based CUDs** are tied to a **region** and a **resource type** (vCPU/RAM); **flexible CUDs** trade a lower discount for fewer constraints.
- CUDs are **not cancelable**. If you downsize below the commitment, you still pay.

4.17.2 Worked comparison (illustrative numbers only, not real prices)

Assume an on-demand price index of **100** per VM-month for a given type.

Option	Effective price index	Interruptions?	Good for
On-demand, full month	100 (minus SUD if eligible, e.g., ~70–80 for N2 at 100% usage ⚠️)	No	Variable, uncertain
1-yr resource CUD	~63–70 ⚠️	No	Stable baseline
3-yr resource CUD	~45–55 ⚠️	No	Very stable, long-lived
Spot	~10–40 ⚠️	**Yes** (preemption)	Batch/stateless/fault-tolerant
E2 on-demand	~70–80 of N2 for similar shape ⚠️	No	Cost-sensitive general workloads
Right-sized + instance schedule (stop nights/weekends, ~65 of 168 hours)	~40 of 100	No	Dev/test

Exam thinking: compute the *shape of demand* first (steady? bursty? interruptible? scheduled?) and then match it to an option, **layering** them: CUD for baseline + autoscaling on-demand for variable demand + Spot for interruptible burst.

4.17.3 Cost visibility and governance

Tool	Use
Labels	Attribute cost by team/app/environment
Billing export to BigQuery	Query detailed cost; build dashboards (Looker Studio)
Budgets and alerts	Notify (or trigger automation) at thresholds
Recommender	Idle VM, rightsizing, idle disk/IP, CUD recommendations
Quotas	Guardrail against runaway usage
Organization policy	Restrict expensive machine types/regions ⚠️ (via `compute.vmCreate`-related constraints and resource locations)
FinOps routine	Monthly review: unattached disks, old snapshots, unused IPs, oversized VMs, orphaned LBs

4.18 Deep dive: choosing Compute Engine versus other compute options

Criterion	Compute Engine	GKE	Cloud Run	App Engine	Cloud Run functions
Unit of deployment	VM	Container (Pods)	Container (request-driven)	App source/container	Function
OS/kernel control	**Full**	Node-level (Standard)	None	None	None
Scale to zero	MIG min 0 with signals (not typical)	Pods yes (HPA/KEDA), nodes via autoscaler	**Yes**	Standard env yes	**Yes**
Ops overhead	**Highest**	Medium (Autopilot lower)	**Low**	Low	**Lowest**
Stateful/legacy/licensed software	**Best fit**	Possible (StatefulSets)	Poor fit	Poor fit	No
Custom networking (NVA, multi-NIC)	**Yes**	Limited	No	No	No
Long-running / background	Yes	Yes	Jobs / min-instances	Limited	Time-limited
GPU / special hardware	**Yes**	Yes	Limited (GPU on Run) ⚠️	No	No

Criterion	Compute Engine	GKE	Cloud Run	App Engine	Cloud Run functions
Lift-and-shift	**Best**	Replatform	Replatform	Replatform	Refactor
Cost model	Per-VM-second (provisioned)	Per node (Standard) / per Pod resource (Autopilot)	Per request/CPU-second used	Instance-hours	Per invocation
Typical exam trigger	"Custom OS", "legacy", "BYOL", "specific kernel module", "lift and shift"	"Microservices", "containers need orchestration"	"Stateless HTTP container, minimal ops, spiky"	"Standard web app, minimal ops"	"Event-driven glue"

Questions to ask the scenario: *Is it containerized? Does it need OS-level control? Is it stateless? How spiky is load? Who operates it (ops headcount)? Licensing? Latency to cold starts? Special hardware?*

4.19 Deep dive: migrating VMs to Compute Engine

Strategy	Meaning	Tools
Rehost (lift-and-shift)	Move as-is to VMs	**Migrate to Virtual Machines** (continuous replication and test-clone cutover), image import (`gcloud compute images import`), Storage Transfer for disks
Replatform	Small changes (managed DB, managed LB)	Database Migration Service, Cloud SQL, MIGs
Refactor	Re-architect to cloud native	Containers, Cloud Run/GKE, serverless
Retire / retain / repurchase	Decommission, keep on-prem, SaaS	

Migration planning points: discovery/assessment (dependencies, utilization → **rightsizing**), landing-zone design (VPC, IAM, org policy), network connectivity (VPN/Interconnect, DNS), **sequencing waves**, test cutovers, rollback plans, licensing (**BYOL → sole-tenant**), and post-migration optimization (rightsize, CUDs, managed services).

Image import notes: supported formats include VMDK, VHD, RAW, QCOW2; ensure **guest environment** is installed; virtual disk drivers (virtio) required for Linux/Windows; use `--os` to select the correct license/driver handling.

4.20 Deep dive: operating and troubleshooting VMs

4.20.1 Observability stack

Signal	Source	Notes
Platform metrics	Compute Engine → Cloud Monitoring	CPU utilization, disk I/O, network bytes; **no memory or process data** without the agent

Signal	Source	Notes
Agent metrics	**Ops Agent**	Memory, disk space, swap, processes, application integrations (nginx, MySQL, etc.)
Logs	Ops Agent (syslog, app logs), Cloud Audit Logs (admin activity), serial console logs	Admin Activity logs are **always on**; Data Access logs may need enabling
Uptime checks	Cloud Monitoring	External probes of public/private endpoints
Alerting	Alert policies on metrics/logs; SLO-based alerts	Notification channels, escalation
Traces and profiling	Cloud Trace, Cloud Profiler	App-level performance

4.20.2 Troubleshooting guide

Symptom	Likely causes	Where to look / what to do
VM won't boot / stuck in STAGING/PROVISIONING	Quota, capacity, invalid image, org policy	Operation error in Console / `gcloud compute operations list`; try another zone
Boots but unreachable	Firewall, no external IP/IAP, wrong tags, bad route, OS firewall, network config failure in guest	**Connectivity Tests**, firewall logs, **serial console output** (`gcloud compute instances get-serial-port-output`)
SSH fails	No firewall for 22 / IAP range, OS Login role missing, disk full (can't write `authorized_keys`), `sshd` down, bad `fstab`	Serial console, IAP troubleshooting, check disk usage, **startup-script** that enables a recovery user
Boots to emergency mode	Bad `/etc/fstab` (missing disk without `nofail`)	Attach boot disk to a rescue VM, fix; **use `nofail` and UUIDs**
Startup script didn't run	Syntax error, no network for package repos (private VM without NAT), runs as root but depends on user env	`journalctl -u google-startup-scripts.service`, serial console
Disk full / out of inodes	Logs, temp, small boot disk	**Resize disk** online, then grow partition + filesystem
Intermittent timeouts to external service	Cloud NAT port exhaustion	NAT logs, add IPs/ports
High latency/bandwidth cap	Machine type egress cap, MTU mismatch, cross-region path, single TCP flow	Metrics, `iperf`, Performance Dashboard
403 from Google APIs	SA lacks IAM role; **scope** too narrow; wrong project	IAM policy troubleshooter, Policy Analyzer
Timeout to Google APIs from private VM	PGA off, DNS not resolving to VIP, egress firewall deny	Subnet PGA flag, DNS, firewall
MIG repeatedly recreating VMs	Health check failing (firewall for probes, wrong port/path, app slow to start)	Health check status, initial delay, serial logs
Autoscaler not scaling out	Max reached, quota, wrong metric, cool-down, metric not reported (agent missing)	Autoscaler status in Console, quotas, metric explorer

Symptom	Likely causes	Where to look / what to do
Spot VM disappeared	Preemption	`gcloud compute operations list --filter="operationType=compute.instances.preempted"`, handle with shutdown script/MIG
Cannot change machine type	VM running	Stop → change → start
Out of capacity (stockout)	Zone exhausted for that type	Other zone/type, reservations

Debug decision tree for "can't connect":

1. **Is the VM running and the app listening?** (serial console, `ss -ltnp` via IAP)
2. **Route?** (Connectivity Test)
3. **Firewall (both egress at source and ingress at destination)?**
4. **Guest OS firewall** (`iptables` / `ufw` /Windows Firewall)?
5. **NAT/PGA/DNS** (for outbound)?
6. **IAM** (for API/authorization errors: 401/403)?
7. **Application layer** (TLS, auth, config)?

---

5. Module 4: Creating and Customizing VM Instances (Hands-on)

This module is the **practical counterpart** of Module 3. Labs are in Section 6. Here is the framework to use when the instructor demonstrates the Console flow.

5.1 Creating a VM in the Console: field-by-field walkthrough

Navigate: **Console → Compute Engine → VM instances → Create instance**.

Section	What to show	What to say (exam angle)
Name, labels, region, zone	Show that region/zone cannot be changed after creation	Zonal resource → plan HA across zones
Machine configuration	Compare series; show **Custom** tab; show cost estimate on the right	Choose by workload family; cost visibility
Availability policies	VM provisioning model (Standard vs Spot), on host maintenance, automatic restart	Spot has no SLA; GPUs can't live migrate
Boot disk	Change image, size, type; show **custom images** tab; encryption options	Golden image strategy, CMEK
Identity and API access	Default SA vs custom SA; scopes	Least privilege, avoid default SA with Editor
Firewall (checkboxes)	"Allow HTTP/HTTPS traffic" creates rules based on **network tags** `http-server` / `https-server`	These checkboxes just create tag-based rules in the VPC
Networking	Network, subnet, internal/external IP, network tags, IP forwarding, network tier	Private VM design
Disks	Add disk, auto-delete behavior, snapshot schedule	Data durability
Security	Shielded VM, Confidential VM, OS Login toggles, block project-wide keys	Security hardening
Management	Description, **automation (startup script)**, metadata, reservation, maintenance	Customization at boot
Sole tenancy	Node affinity	Licensing/compliance
EQUIVALENT COMMAND LINE	Click it, copy the `gcloud` command or REST/Terraform	Great teaching tool

5.2 Customization levels and when to use each

Level	Technique	Pros	Cons
Boot-time	Startup script	Simple, flexible, easy to change	Slower boot, depends on internet/repo availability at boot, may differ across time
Image-time	Custom image (golden image)	Fast, consistent, reproducible boot	Image sprawl, patch/rebuild cycles required

Level	Technique	Pros	Cons
Hybrid	Golden image with core software and startup script for environment-specific config	**Best practice**	Needs a pipeline
Config management	Ansible, Puppet, Chef, OS Config	Good for long-lived VMs	More tooling
Containers	Container-Optimized OS + Docker, or move to GKE/Cloud Run	Immutable deploys	Requires containerization

> **Immutable infrastructure** is the modern direction: bake config into images and replace VMs instead of patching them in place.

5.3 Day-2 operations on a running VM

Task	Can it be done live?
Resize a persistent disk (up)	Yes
Attach/detach disks	Yes
Add/remove network tags, labels, metadata	Yes
Change machine type	**No**, must stop VM
Change service account	**No**, must stop VM
Change network/subnet of primary NIC	Requires stop (or recreate) ⚠️
Change zone/region	**No**. Use **move** operations, snapshots, or machine images to recreate
Take a snapshot	Yes (consider application consistency)

Moving a VM across zones/regions: snapshot or machine image → create new disk/VM in the target zone. (`gcloud compute instances move` exists with limitations.) ⚠️

5.4 Deep dive: a VM design worksheet (use for every workload)

Before creating any VM, a good architect fills in this worksheet. It turns requirements into configuration choices, and it mirrors how exam questions are built.

#	Question	Drives which setting
1	**What is the workload type?** (web, DB, batch, HPC, ML, legacy)	Machine family and series
2	**Required resources at peak and average?** (vCPU, RAM, IOPS, throughput, network)	Machine type (predefined/custom), disk type and size, bandwidth tier
3	**Can it be interrupted?**	Provisioning model (Standard vs Spot)
4	**How long will it run and how predictable is it?**	CUD / SUD / schedule / autoscaling
5	**Availability target (RTO/RPO)?**	Single VM vs MIG, zonal vs regional, regional PD, backups

#	Question	Drives which setting
6	**Statefulness?**	Boot disk only, data disk, Local SSD (cache only), stateful MIG
7	**Where do users and dependencies sit?**	Region/zone, subnet, network tier
8	**Who may access it and how?**	OS Login, IAP, firewall targets, service account
9	**What can it access in Google Cloud?**	Service account roles
10	**Compliance?**	CMEK, Confidential VM, Shielded VM, sole-tenancy, org policies, location constraints
11	**How is it built and updated?**	Golden image, startup script, config management, rolling updates
12	**How is it monitored, patched, backed up?**	Ops Agent, OS Config, snapshot schedule
13	**How is it created and tracked?**	Terraform, labels, naming convention

Worked example. "*An internal reporting tool, 8 users, runs office hours only, needs 4 vCPU/16 GB, data on a 200 GB disk, must be reachable only from corporate network via VPN.*"

Decision	Choice	Rationale
Series	`e2-standard-4` (or `n2-standard-4` if steady-state performance matters)	Low cost, general purpose
Provisioning	Standard (not Spot)	Interruption unacceptable during office hours
Cost levers	**Instance schedule** (start 07:00, stop 19:00 weekdays)	~65–70% fewer billable hours
Disk	`pd-balanced`, 200 GB, **snapshot schedule** daily	Balanced cost/perf, backup
Network	Private subnet, **no external IP**, firewall allows corporate CIDR range over VPN on the app port	Internal access only
Access	OS Login + IAP	Individual accountability
Identity	Dedicated SA with only the needed roles	Least privilege
Image	Hardened custom image from family	Consistency
Monitoring	Ops Agent, uptime check, alert on disk > 80%	Operations

5.5 Deep dive: configuration as code for VMs and MIGs (Terraform)

The following mirrors Labs 3 and 7.

```
resource "google_service_account" "web" {
  account_id   = "web-sa"
  display_name = "Web tier"
}

resource "google_compute_instance_template" "web_v1" {
```

```
  name_prefix  = "web-tpl-"            # lets Terraform create a new template before destroying the old one
  region       = var.region
  machine_type = "e2-small"
  tags         = ["web", "ssh-iap"]
  labels       = { env = "prod", tier = "web" }

  disk {
    source_image = "projects/debian-cloud/global/images/family/debian-12"
    auto_delete  = true
    boot         = true
    disk_type    = "pd-balanced"
    disk_size_gb = 20
  }

  network_interface {
    subnetwork = google_compute_subnetwork.web.id
    # no access_config block → no external IP
  }

  service_account {
    email  = google_service_account.web.email
    scopes = ["cloud-platform"]
  }

  shielded_instance_config {
    enable_secure_boot          = true
    enable_vtpm                 = true
    enable_integrity_monitoring = true
  }

  metadata = {
    enable-oslogin = "TRUE"
    startup-script = file("${path.module}/mig-startup.sh")
  }

  lifecycle {
    create_before_destroy = true       # immutable template pattern
  }
}

resource "google_compute_health_check" "web" {
  name                = "web-hc"
  check_interval_sec  = 10
  timeout_sec         = 5
  healthy_threshold   = 2
  unhealthy_threshold = 3
  http_health_check {
    port         = 80
    request_path = "/"
  }
}

resource "google_compute_region_instance_group_manager" "web" {
  name                        = "web-mig"
  region                      = var.region
  base_instance_name          = "web"
  distribution_policy_zones = ["us-central1-a", "us-central1-b", "us-central1-c"]

  version {
    instance_template = google_compute_instance_template.web_v1.id
  }

  auto_healing_policies {
    health_check      = google_compute_health_check.web.id
    initial_delay_sec = 120
  }
```

```
  update_policy {
    type                  = "PROACTIVE"
    minimal_action        = "REPLACE"
    max_surge_fixed       = 3
    max_unavailable_fixed = 0
  }
}

resource "google_compute_region_autoscaler" "web" {
  name   = "web-autoscaler"
  region = var.region
  target = google_compute_region_instance_group_manager.web.id

  autoscaling_policy {
    min_replicas    = 2
    max_replicas    = 6
    cooldown_period = 90
    cpu_utilization {
      target = 0.6
    }
  }
}
```

Key idea: `create_before_destroy` plus `name_prefix` implements the **immutable template + rolling update** pattern in code. Changing the startup script creates a new template, and the MIG's update policy rolls it out safely.

5.6 Deep dive: building golden images in a pipeline

```
Git commit (image definition: Packer template + provisioning scripts)
     │
     ▼
Cloud Build trigger ──► Packer launches a temporary VM (private subnet, dedicated SA)
     │                        │ installs/hardens software, runs tests/scanners
     │                        ▼
     │                creates custom image in family "web-golden" (labels: version, commit SHA)
     ▼
Image published ──► vulnerability scan / org policy (trustedImageProjects)
     │
     ▼
Terraform updates instance template (image family) ──► MIG rolling update (canary → full)
```

Practice	Why
Version and label every image (commit SHA, build date)	Traceability and rollback
Build from **hardened base images**, remove SSH keys and secrets before capture	Avoid leaking credentials into images
Test before promoting (boot test, smoke test, security scan)	Quality gate
Deprecate old images (`gcloud compute images deprecate`) instead of deleting immediately	Safe rollback window
Keep images in a **central image project**; share with consumer projects via IAM	Governance, with org policy limiting image sources
Patch cadence: rebuild at least monthly or on critical CVEs	Replace instead of patching in place

5.7 Deep dive: customization patterns and when to use each

Pattern	Mechanism	Example	Trade-off
Bootstrap config via metadata	Custom metadata keys read by startup script	`environment=prod` selects config file	Simple; don't put secrets here
Secrets at runtime	VM SA reads **Secret Manager**	App fetches DB password on start	Needs IAM + API reachability (PGA)
Role-specific images	One image family per role (web, worker)	`web-golden`, `worker-golden`	More images to maintain
One image, role by metadata	Single image + `role` metadata	Startup script enables the right services	Fewer images, a bit more boot logic
Config management	Ansible/Puppet/Chef/OS policies	Fleet-wide desired state for long-lived VMs	Extra tooling, still mutable
Containers on VM	Container-Optimized OS, `gcloud compute instances create-with-container`	Run a single container with auto-restart	Fine for simple cases; use GKE/Cloud Run for more
Cloud-init	`user-data` for supported images	Cross-cloud portability	Different from GCE metadata scripts; supported on some images ⚠️
Instance schedules	Resource policy start/stop	Dev/test shutdown at night	Not for stateful production
Reservations	Pre-allocate capacity	Product launch	Pay while reserved
Labels and naming	Consistent taxonomy	`env`, `app`, `owner`, `cost-center`	Needs governance to stay consistent

5.8 Deep dive: mapping Console choices to gcloud and Terraform

Console choice	`gcloud` flag	Terraform argument	Exam relevance
Machine type	`--machine-type` / `--custom-cpu --custom-memory`	`machine_type`	Right-sizing
Spot	`--provisioning-model=SPOT`	`scheduling { provisioning_model = "SPOT" }`	Cost
On host maintenance	`--maintenance-policy`	`scheduling { on_host_maintenance }`	Availability
Automatic restart	`--restart-on-failure`	`scheduling { automatic_restart }`	Availability
No external IP	`--no-address`	omit `access_config`	Security

Console choice	`gcloud` flag	Terraform argument	Exam relevance
Network tags	`--tags`	`tags`	Firewall targeting
Service account	`--service-account --scopes`	`service_account {}`	Least privilege
Shielded VM	`--shielded-secure-boot --shielded-vtpm --shielded-integrity-monitoring`	`shielded_instance_config {}`	Security
Confidential VM	`--confidential-compute-type` / `--confidential-compute` ⚠️	`confidential_instance_config {}`	Data in use
OS Login	metadata `enable-oslogin=TRUE`	`metadata`	Access control
Disk CMEK	`--boot-disk-kms-key`	`boot_disk { kms_key_self_link }`	Compliance
IP forwarding	`--can-ip-forward`	`can_ip_forward`	NVAs
Sole-tenancy	`--node-group` / `--node-affinity-file`	`scheduling { node_affinities }`	Licensing/isolation
Labels	`--labels`	`labels`	Cost allocation
Snapshot schedule	`gcloud compute disks add-resource-policies`	`google_compute_resource_policy` + attachment	Backup
Instance schedule	`--resource-policies`	`google_compute_resource_policy { instance_schedule_policy }`	Cost

6. Hands-on Labs (Demonstration Track)

> **Instructor flow for each lab:** (1) state the goal, (2) demo in Console briefly, (3) run the `gcloud` commands, (4) verify, (5) ask a "what would happen if..." question, (6) link to the exam concept. **Participants** can follow along in their own sandbox project.

Make sure the variables from Section 1.4 are set in your Cloud Shell session. If the session restarts, re-run them.

LAB Lab 1: Create a custom-mode VPC with subnets (≈ 20 min)

Goal: Build `prod-vpc` with three tiered subnets in `us-central1` and one in `europe-west1`, with Private Google Access and Flow Logs.

Concepts reinforced: custom mode, regional subnets in a global VPC, PGA, flow logs, expand-ip-range, reserved IPs.

Steps

```
# 1. Create the VPC (custom mode, regional dynamic routing)
gcloud compute networks create prod-vpc \
  --subnet-mode=custom \
  --bgp-routing-mode=regional

# 2. Create subnets (us-central1)
gcloud compute networks subnets create web-subnet \
  --network=prod-vpc --region=$REGION --range=10.10.1.0/24 \
  --enable-private-ip-google-access \
  --enable-flow-logs --logging-aggregation-interval=interval-5-sec \
  --logging-flow-sampling=0.5 --logging-metadata=include-all

gcloud compute networks subnets create app-subnet \
  --network=prod-vpc --region=$REGION --range=10.10.2.0/24 \
  --enable-private-ip-google-access \
  --enable-flow-logs

gcloud compute networks subnets create db-subnet \
  --network=prod-vpc --region=$REGION --range=10.10.3.0/24 \
  --enable-private-ip-google-access \
  --enable-flow-logs

# 3. Create a subnet in a second region (same VPC!)
gcloud compute networks subnets create eu-subnet \
  --network=prod-vpc --region=$REGION2 --range=10.20.1.0/24 \
  --enable-private-ip-google-access
```

Verify

```
gcloud compute networks list
gcloud compute networks subnets list --network=prod-vpc
gcloud compute networks describe prod-vpc --format="yaml(name,routingConfig,subnetworks)"
gcloud compute routes list --filter="network:prod-vpc"
```

Expected: four subnets across two regions and system-generated routes: one **default route** (`0.0.0.0/0` to `default-internet-gateway`) and one **subnet route per subnet**.

Demonstrate: expand a subnet

```
gcloud compute networks subnets expand-ip-range web-subnet \
  --region=$REGION --prefix-length=23
# Then try to shrink it to /24. It will fail. Discuss why.
```

Discussion questions

1. Why is there no IP range on `prod-vpc` itself?
2. What are the 4 reserved addresses in `10.10.2.0/24`? How many are usable?
3. What happens if we try to create another subnet `10.10.2.128/25`? (Overlap error.)

LAB Lab 2: Create service accounts and firewall rules (≈ 30 min)

Goal: Implement a three-tier firewall model with service accounts, IAP access, health check ranges, and logging.

Concepts reinforced: implied rules, priorities, service-account targets vs. tags, IAP source range, rule logging.

Step 1: Create service accounts for each tier

```
for tier in web app db; do
  gcloud iam service-accounts create ${tier}-sa \
    --display-name="${tier} tier service account"
done

export WEB_SA=web-sa@${PROJECT_ID}.iam.gserviceaccount.com
export APP_SA=app-sa@${PROJECT_ID}.iam.gserviceaccount.com
export DB_SA=db-sa@${PROJECT_ID}.iam.gserviceaccount.com

# Give each SA only what it needs to write logs and metrics
for sa in $WEB_SA $APP_SA $DB_SA; do
  gcloud projects add-iam-policy-binding $PROJECT_ID \
    --member="serviceAccount:$sa" --role="roles/logging.logWriter" --condition=None
  gcloud projects add-iam-policy-binding $PROJECT_ID \
    --member="serviceAccount:$sa" --role="roles/monitoring.metricWriter" --condition=None
done
```

Step 2: Show what exists BEFORE adding rules

```
gcloud compute firewall-rules list --filter="network:prod-vpc"
```

Expected: an **empty** list. That network is currently subject only to **implied** rules: all egress allowed, all ingress denied.

Step 3: Create the rules

```
# a) IAP SSH access to any VM tagged ssh-iap
gcloud compute firewall-rules create allow-iap-ssh \
  --network=prod-vpc --direction=INGRESS --priority=1000 --action=ALLOW \
  --rules=tcp:22 --source-ranges=35.235.240.0/20 \
  --target-tags=ssh-iap

# b) Web tier: HTTP from anywhere (to be fronted by an LB later)
gcloud compute firewall-rules create allow-http-to-web \
  --network=prod-vpc --direction=INGRESS --priority=1000 --action=ALLOW \
  --rules=tcp:80,tcp:443 --source-ranges=0.0.0.0/0 \
  --target-service-accounts=$WEB_SA

# c) Web → App on 8080 using service accounts (tamper-resistant)
gcloud compute firewall-rules create allow-web-to-app-tcp8080 \
  --network=prod-vpc --direction=INGRESS --priority=1000 --action=ALLOW \
  --rules=tcp:8080 \
  --source-service-accounts=$WEB_SA \
  --target-service-accounts=$APP_SA \
  --enable-logging

# d) App → DB on 5432
gcloud compute firewall-rules create allow-app-to-db-tcp5432 \
  --network=prod-vpc --direction=INGRESS --priority=1000 --action=ALLOW \
  --rules=tcp:5432 \
  --source-service-accounts=$APP_SA \
  --target-service-accounts=$DB_SA \
  --enable-logging

# e) Health check probes (needed for MIG autohealing / LB later)
gcloud compute firewall-rules create allow-health-checks \
  --network=prod-vpc --direction=INGRESS --priority=1000 --action=ALLOW \
  --rules=tcp:80 --source-ranges=130.211.0.0/22,35.191.0.0/16 \
  --target-tags=web

# f) Internal ICMP across the whole private space (for ping tests, cross-region demo)
gcloud compute firewall-rules create allow-icmp-internal \
  --network=prod-vpc --direction=INGRESS --priority=1000 --action=ALLOW \
  --rules=icmp --source-ranges=10.0.0.0/8

# g) Catch-all deny with logging so we can SEE what is being blocked
gcloud compute firewall-rules create deny-all-ingress-log \
  --network=prod-vpc --direction=INGRESS --priority=65000 --action=DENY \
  --rules=all --source-ranges=0.0.0.0/0 \
  --enable-logging
```

Step 4: Inspect and discuss

```
gcloud compute firewall-rules list --filter="network:prod-vpc" \
  --
format="table(name,direction,priority,sourceRanges.list():label=SRC,targetServiceAccounts.list():label=TARG
ET_SA,targetTags.list():label=TAGS,allowed[].map().firewall_rule().list():label=ALLOW,denied[].map().firewa
ll_rule().list():label=DENY)"
```

Discuss:

- Why is `deny-all-ingress-log` priority **65000** and not `1`? (It must be **lower** in precedence than all allows. Deny at priority 1 would block everything.)
- Why does this rule not break return traffic of outbound connections? (**Stateful** firewall.)

- What if we use tags instead of service accounts in (c)? (Works, but any user able to edit instance tags could add the tag.)

Step 5: Demonstrate rule enable/disable and priority

```
# Temporarily disable a rule without deleting it
gcloud compute firewall-rules update allow-web-to-app-tcp8080 --disabled
gcloud compute firewall-rules update allow-web-to-app-tcp8080 --no-disabled
```

LAB Lab 3: Create tiered VMs and test the firewall model (≈ 30 min)

Goal: Launch three private VMs (no external IPs), connect with IAP, and prove which flows are allowed or blocked.

Concepts reinforced: private VMs, IAP, startup scripts, service accounts on VMs, tag-based vs SA-based rules, internal DNS, firewall logs.

Step 1: Startup scripts that simulate services

```
cat > app-startup.sh <<'SCRIPT'
#!/bin/bash
mkdir -p /srv/app && echo "hello from APP tier on $(hostname)" > /srv/app/index.html
cd /srv/app && nohup python3 -m http.server 8080 >/var/log/app.log 2>&1 &
SCRIPT

cat > db-startup.sh <<'SCRIPT'
#!/bin/bash
mkdir -p /srv/db && echo "pretend this is PostgreSQL" > /srv/db/index.html
cd /srv/db && nohup python3 -m http.server 5432 >/var/log/db.log 2>&1 &
SCRIPT

cat > web-startup.sh <<'SCRIPT'
#!/bin/bash
mkdir -p /srv/web && echo "hello from WEB tier on $(hostname)" > /srv/web/index.html
cd /srv/web && nohup python3 -m http.server 80 >/var/log/web.log 2>&1 &
SCRIPT
```

Step 2: Create the VMs

```
# Web VM
gcloud compute instances create web-1 \
  --zone=$ZONE --machine-type=e2-small \
  --subnet=web-subnet --no-address \
  --tags=web,ssh-iap \
  --service-account=$WEB_SA --scopes=cloud-platform \
  --image-family=debian-12 --image-project=debian-cloud \
  --boot-disk-size=20GB --boot-disk-type=pd-balanced \
  --metadata-from-file=startup-script=web-startup.sh \
  --labels=env=lab,tier=web

# App VM
gcloud compute instances create app-1 \
  --zone=$ZONE --machine-type=e2-small \
  --subnet=app-subnet --no-address \
  --tags=ssh-iap \
```

```
  --service-account=$APP_SA --scopes=cloud-platform \
  --image-family=debian-12 --image-project=debian-cloud \
  --metadata-from-file=startup-script=app-startup.sh \
  --labels=env=lab,tier=app

# DB VM
gcloud compute instances create db-1 \
  --zone=$ZONE --machine-type=e2-small \
  --subnet=db-subnet --no-address \
  --tags=ssh-iap \
  --service-account=$DB_SA --scopes=cloud-platform \
  --image-family=debian-12 --image-project=debian-cloud \
  --metadata-from-file=startup-script=db-startup.sh \
  --labels=env=lab,tier=db

# EU VM (cross-region, same VPC)
gcloud compute instances create eu-1 \
  --zone=$ZONE2 --machine-type=e2-small \
  --subnet=eu-subnet --no-address \
  --tags=ssh-iap \
  --image-family=debian-12 --image-project=debian-cloud \
  --labels=env=lab,tier=eu
```

Note the `eu-1` VM uses the **default Compute Engine service account** (no `--service-account`). Use this to show why the default SA is a poor choice: it is not part of our SA-based firewall model, and it carries broad permissions in many projects.

Step 3: Connect through IAP (no public IP, no bastion)

```
gcloud compute ssh web-1 --zone=$ZONE --tunnel-through-iap
```

If the connection fails: confirm the IAP API is enabled, the `allow-iap-ssh` rule exists with the right tag, and your user has `roles/iap.tunnelResourceAccessor` (project Owners normally can). Also confirm the VM has the `ssh-iap` tag.

Step 4: Run the connectivity matrix

```
export APP_IP=$(gcloud compute instances describe app-1 --zone=$ZONE \
  --format='get(networkInterfaces[0].networkIP)')
export DB_IP=$(gcloud compute instances describe db-1 --zone=$ZONE \
  --format='get(networkInterfaces[0].networkIP)')
export EU_IP=$(gcloud compute instances describe eu-1 --zone=$ZONE2 \
  --format='get(networkInterfaces[0].networkIP)')

echo "Test 1: web-1 -> app-1:8080 (EXPECT 200)"
gcloud compute ssh web-1 --zone=$ZONE --tunnel-through-iap \
  --command="curl -s -m 5 -o /dev/null -w '%{http_code}\n' http://$APP_IP:8080 || echo BLOCKED"

echo "Test 2: web-1 -> db-1:5432 (EXPECT BLOCKED, web may not reach db)"
gcloud compute ssh web-1 --zone=$ZONE --tunnel-through-iap \
  --command="curl -s -m 5 -o /dev/null -w '%{http_code}\n' http://$DB_IP:5432 || echo BLOCKED"

echo "Test 3: app-1 -> db-1:5432 (EXPECT 200)"
gcloud compute ssh app-1 --zone=$ZONE --tunnel-through-iap \
  --command="curl -s -m 5 -o /dev/null -w '%{http_code}\n' http://$DB_IP:5432 || echo BLOCKED"

echo "Test 4: web-1 -> eu-1 ping across regions (EXPECT replies)"
```

```
gcloud compute ssh web-1 --zone=$ZONE --tunnel-through-iap \
  --command="ping -c 3 $EU_IP"
```

Expected results

Test	Source → Destination	Result	Why
1	web-1 → app-1:8080	**Allowed**	Rule `allow-web-to-app-tcp8080` (SA to SA)
2	web-1 → db-1:5432	**Blocked**	No allow rule from web-sa to db-sa; implied/explicit deny
3	app-1 → db-1:5432	**Allowed**	Rule `allow-app-to-db-tcp5432`
4	web-1 → eu-1 (ICMP)	**Allowed**	Global VPC + `allow-icmp-internal`. No VPN or peering required

Why does Test 4 TCP not work? Try `curl http://$EU_IP:80` and explain: ICMP is allowed but no TCP rule exists. Routing works; the **firewall** decides.

Step 5: Observe logs

In Console → **Logging → Logs Explorer**, run:

```
resource.type="gce_subnetwork"
logName:"compute.googleapis.com%2Ffirewall"
```

Find the **denied** attempt from Test 2 and note `rule_details.reference` pointing to `deny-all-ingress-log`. Also look at **VPC Flow Logs**:

```
logName:"compute.googleapis.com%2Fvpc_flows"
```

Step 6: Use Connectivity Tests

```
gcloud network-management connectivity-tests create web-to-db \
  --source-instance=projects/$PROJECT_ID/zones/$ZONE/instances/web-1 \
  --destination-instance=projects/$PROJECT_ID/zones/$ZONE/instances/db-1 \
  --protocol=TCP --destination-port=5432

gcloud network-management connectivity-tests describe web-to-db \
  --format="yaml(reachabilityDetails)"
```

Expected: result `UNREACHABLE` with a trace pointing at the denying firewall rule. Show the same in Console → **Network Intelligence Center → Connectivity Tests**.

LAB Lab 4: Cloud NAT and Private Google Access (≈ 20 min)

Goal: Give private VMs outbound internet and access to Google APIs without external IPs.

Step 1: Prove the problem

```
gcloud compute ssh web-1 --zone=$ZONE --tunnel-through-iap \
  --command="curl -s -m 5 -I https://example.com | head -1 || echo 'NO INTERNET'"
```

Expected: `NO INTERNET` (no external IP, no NAT).

Step 2: Prove Private Google Access works already

```
# Create a bucket and test from the private VM
export BUCKET=${PROJECT_ID}-day2-lab
gcloud storage buckets create gs://$BUCKET --location=$REGION
echo "hello" | gcloud storage cp - gs://$BUCKET/hello.txt
gcloud storage buckets add-iam-policy-binding gs://$BUCKET \
  --member="serviceAccount:$WEB_SA" --role="roles/storage.objectViewer"

gcloud compute ssh web-1 --zone=$ZONE --tunnel-through-iap \
  --command="gcloud storage cat gs://$BUCKET/hello.txt"
```

Expected: prints `hello`. This works because **web-subnet has Private Google Access enabled**, and the VM's service account has the right IAM role. (Teaching point: PGA solves *network reachability*, **IAM** still decides *authorization*.)

Step 3: Add Cloud NAT

```
gcloud compute routers create prod-router \
  --network=prod-vpc --region=$REGION

gcloud compute routers nats create prod-nat \
  --router=prod-router --region=$REGION \
  --auto-allocate-nat-external-ips \
  --nat-all-subnet-ip-ranges \
  --enable-logging
```

Step 4: Retest internet access

```
gcloud compute ssh web-1 --zone=$ZONE --tunnel-through-iap \
  --command="curl -s -m 10 -I https://example.com | head -1"
```

Expected: an HTTP status line. Note that **eu-1** still has no internet (the NAT is regional, `us-central1` only). Discuss why.

Discussion

- Can the internet initiate a connection to `web-1` through the NAT? (No.)
- What is the cost/security advantage vs. giving each VM a public IP?
- Where would you put a **second** NAT for `eu-subnet`? (A Cloud Router and NAT in `europe-west1`.)

LAB Lab 5: Customize a VM: custom machine type, disks, metadata, Spot (≈ 30 min)

Goal: Practice the customization options on a VM.

Part A: Custom machine type

```
gcloud compute instances create custom-1 \
  --zone=$ZONE \
  --custom-vm-type=n2 --custom-cpu=6 --custom-memory=20GB \
  --subnet=app-subnet --no-address \
```

```
  --tags=ssh-iap \
  --image-family=debian-12 --image-project=debian-cloud

gcloud compute instances describe custom-1 --zone=$ZONE \
  --format="value(machineType.basename())"
```

Expected: `n2-custom-6-20480`. Discuss the vCPU and memory rules. Try `--custom-cpu=5` and show the error.

Part B: Add and use a data disk

```
gcloud compute disks create data-disk-1 --size=50GB --type=pd-ssd --zone=$ZONE
gcloud compute instances attach-disk custom-1 \
  --disk=data-disk-1 --device-name=data-disk-1 --zone=$ZONE

gcloud compute ssh custom-1 --zone=$ZONE --tunnel-through-iap
```

Inside the VM:

```
lsblk
sudo mkfs.ext4 -m 0 -E lazy_itable_init=0,lazy_journal_init=0,discard \
  /dev/disk/by-id/google-data-disk-1
sudo mkdir -p /mnt/data
sudo mount -o discard,defaults /dev/disk/by-id/google-data-disk-1 /mnt/data
echo "important data" | sudo tee /mnt/data/file.txt
df -h /mnt/data

# Persist across reboots (use UUID in production)
echo UUID=$(sudo blkid -s UUID -o value /dev/disk/by-id/google-data-disk-1) \
  /mnt/data ext4 discard,defaults,nofail 0 2 | sudo tee -a /etc/fstab
exit
```

Resize online (no downtime):

```
gcloud compute disks resize data-disk-1 --size=100GB --zone=$ZONE
gcloud compute ssh custom-1 --zone=$ZONE --tunnel-through-iap \
  --command="sudo resize2fs /dev/disk/by-id/google-data-disk-1 && df -h /mnt/data"
```

> Key lesson: you resize the **cloud disk** and then the **file system**. Disks can **grow** but never **shrink**.

Part C: Metadata server and custom metadata

```
gcloud compute instances add-metadata custom-1 --zone=$ZONE \
  --metadata=environment=training,owner=day2

gcloud compute ssh custom-1 --zone=$ZONE --tunnel-through-iap --command='
  curl -s -H "Metadata-Flavor: Google"
http://metadata.google.internal/computeMetadata/v1/instance/attributes/environment; echo;
  curl -s -H "Metadata-Flavor: Google" http://metadata.google.internal/computeMetadata/v1/instance/name;
echo;
  curl -s -H "Metadata-Flavor: Google" http://metadata.google.internal/computeMetadata/v1/instance/service-
accounts/default/email; echo'
```

Part D: Spot VM with a shutdown script

```
cat > shutdown.sh <<'SCRIPT'
#!/bin/bash
echo "$(date) preempted or stopping" >> /tmp/shutdown.log
SCRIPT

gcloud compute instances create spot-1 \
  --zone=$ZONE --machine-type=e2-small \
  --provisioning-model=SPOT --instance-termination-action=STOP \
  --subnet=app-subnet --no-address --tags=ssh-iap \
  --image-family=debian-12 --image-project=debian-cloud \
  --metadata-from-file=shutdown-script=shutdown.sh

gcloud compute instances describe spot-1 --zone=$ZONE \
  --format="yaml(scheduling)"
```

Discuss: What does `STOP` vs `DELETE` change? Why is a Spot VM wrong for a single-instance production database?

Part E: Change the machine type (requires a stop)

```
gcloud compute instances stop custom-1 --zone=$ZONE
gcloud compute instances set-machine-type custom-1 --zone=$ZONE \
  --machine-type=e2-standard-2
gcloud compute instances start custom-1 --zone=$ZONE
```

LAB Lab 6: Images, snapshots, and snapshot schedules (≈ 25 min)

Goal: Build a golden image, snapshot a disk, schedule snapshots, and launch from the image.

Step 1: Prepare a "golden" VM (needs outbound internet → Cloud NAT from Lab 4)

```
gcloud compute ssh web-1 --zone=$ZONE --tunnel-through-iap --command='
  sudo apt-get update -y && sudo apt-get install -y nginx
  echo "GOLDEN IMAGE v1" | sudo tee /var/www/html/index.nginx-debian.html
  sudo systemctl enable nginx'
```

Step 2: Create a custom image from the boot disk

```
gcloud compute instances stop web-1 --zone=$ZONE

gcloud compute images create web-golden-v1 \
  --source-disk=web-1 --source-disk-zone=$ZONE \
  --family=web-golden \
  --labels=version=v1

gcloud compute images list --no-standard-images \
  --format="table(name,family,status,diskSizeGb)"

gcloud compute instances start web-1 --zone=$ZONE
```

> Caution: nginx and the python server from the startup script would both try to use port 80 after reboot. For the lab, nginx is enabled and the startup script still runs. If port 80 conflicts, demonstrate **reading the serial console / logs** to diagnose (a good troubleshooting teaching moment). In the MIG lab below, use a clean startup script.

Step 3: Snapshot and snapshot schedule

```
# One-time snapshot
gcloud compute disks snapshot data-disk-1 --zone=$ZONE \
  --snapshot-names=data-disk-1-snap1 --storage-location=us

# Daily schedule with 14-day retention
gcloud compute resource-policies create snapshot-schedule daily-snap \
  --region=$REGION --max-retention-days=14 \
  --on-source-disk-delete=keep-auto-snapshots \
  --daily-schedule --start-time=02:00

gcloud compute disks add-resource-policies data-disk-1 \
  --resource-policies=daily-snap --zone=$ZONE

gcloud compute snapshots list
```

Step 4: Restore from a snapshot (simulate disaster recovery)

```
gcloud compute disks create data-disk-restored \
  --source-snapshot=data-disk-1-snap1 --zone=$ZONE --type=pd-balanced
```

Discuss

- Snapshots are **incremental**: the first is full, later ones store only changes.
- To restore in **another region**, create the disk from the snapshot in that region (snapshots are global resources). That is a DR building block.
- **Machine image** vs **custom image**: a machine image captures the **whole VM** (config + all disks), a custom image captures **one boot disk**.

LAB Lab 7: Instance template and Managed Instance Group with autoscaling and autohealing (≈ 40 min)

Goal: Build an HA, self-healing, auto-scaling web tier across zones in one region.

Step 1: Clean startup script (installs nothing from the internet if you use the golden image)

Option A (uses golden image from Lab 6):

```
cat > mig-startup.sh <<'SCRIPT'
#!/bin/bash
echo "Served by $(hostname) in $(curl -s -H 'Metadata-Flavor: Google' \
  http://metadata.google.internal/computeMetadata/v1/instance/zone | awk -F/ '{print $NF}')" \
  > /var/www/html/index.nginx-debian.html
systemctl restart nginx
```

```
SCRIPT

gcloud compute instance-templates create web-tpl-v1 \
  --region=$REGION \
  --machine-type=e2-small \
  --subnet=web-subnet --no-address \
  --tags=web,ssh-iap \
  --service-account=$WEB_SA --scopes=cloud-platform \
  --image=web-golden-v1 \
  --metadata-from-file=startup-script=mig-startup.sh \
  --labels=env=lab,tier=web
```

Option B (no golden image): use `--image-family=debian-12 --image-project=debian-cloud` and a startup script that runs `apt-get install -y nginx` (requires Cloud NAT).

> **Note:** If you built `web-1` with the golden image from Lab 6, delete or stop the standalone `web-1` first to avoid confusion. It is not required for the MIG.

Step 2: Health check and MIG

```
gcloud compute health-checks create http web-hc \
  --port=80 --request-path=/ \
  --check-interval=10s --timeout=5s \
  --healthy-threshold=2 --unhealthy-threshold=3

gcloud compute instance-groups managed create web-mig \
  --region=$REGION \
  --template=web-tpl-v1 \
  --size=3 \
  --health-check=web-hc --initial-delay=120

gcloud compute instance-groups managed list-instances web-mig --region=$REGION
```

Expected: three instances spread across **different zones** of `us-central1` (regional MIG). Show the zone distribution.

Step 3: Autoscaling

```
gcloud compute instance-groups managed set-autoscaling web-mig \
  --region=$REGION \
  --min-num-replicas=2 --max-num-replicas=6 \
  --target-cpu-utilization=0.6 \
  --cool-down-period=90
```

Generate load (from a tunneled session on one of the instances; this installs `stress` and needs NAT):

```
INSTANCE=$(gcloud compute instance-groups managed list-instances web-mig \
  --region=$REGION --format="value(name)" | head -1)
INSTANCE_ZONE=$(gcloud compute instances list --filter="name=$INSTANCE" --format="value(zone.basename())")

gcloud compute ssh $INSTANCE --zone=$INSTANCE_ZONE --tunnel-through-iap \
  --command="sudo apt-get install -y stress && stress --cpu 2 --timeout 600 &"
```

Watch it scale:

```
watch -n 15 "gcloud compute instance-groups managed list-instances web-mig --region=$REGION"
```

Step 4: Autohealing demo

```
# Break nginx on one instance
gcloud compute ssh $INSTANCE --zone=$INSTANCE_ZONE --tunnel-through-iap \
  --command="sudo systemctl stop nginx"

# Watch the instance go unhealthy, then be recreated
gcloud compute instance-groups managed list-instances web-mig --region=$REGION
```

> Remember the cause–effect chain: **health check fails → MIG marks unhealthy → recreates the VM**. A missing health-check firewall rule produces the *same symptom*, but for the wrong reason, which is a classic exam trap.

Step 5: Rolling update and canary

```
# New template with v2 content
sed 's/Served by/Served by V2/' mig-startup.sh > mig-startup-v2.sh
gcloud compute instance-templates create web-tpl-v2 \
  --region=$REGION --machine-type=e2-small \
  --subnet=web-subnet --no-address --tags=web,ssh-iap \
  --service-account=$WEB_SA --scopes=cloud-platform \
  --image=web-golden-v1 \
  --metadata-from-file=startup-script=mig-startup-v2.sh

# Canary: 20% of instances get v2
gcloud compute instance-groups managed rolling-action start-update web-mig \
  --region=$REGION \
  --version=template=web-tpl-v1 \
  --canary-version=template=web-tpl-v2,target-size=20% \
  --max-surge=1 --max-unavailable=0

# Promote to 100%
gcloud compute instance-groups managed rolling-action start-update web-mig \
  --region=$REGION \
  --version=template=web-tpl-v2 \
  --max-surge=1 --max-unavailable=0
```

Discuss: `max-surge` adds extra capacity during the update (more cost, no capacity loss). `max-unavailable=0` keeps full capacity. Combine both for zero-downtime updates.

> **Optional extension (if time permits):** Attach the MIG as a backend of an external Application Load Balancer. This is covered in a later session on load balancing.

LAB Lab 8: Troubleshooting drill (≈ 20 min)

Goal: Practice the diagnostic method that exam questions test indirectly.

The instructor **secretly breaks** the environment and participants fix it. Suggested faults:

#	Fault injected	Symptom	Diagnosis tool	Fix
1	Remove the `ssh-iap` tag from `app-1`	IAP SSH times out	Compare VM tags with rule target tags; Connectivity Test	Re-add tag
2	Disable `allow-health-checks` rule	MIG instances keep being recreated	Firewall logs, MIG status, health-check status	Re-enable rule or add rule for 130.211.0.0/22 and 35.191.0.0/16
3	Delete Cloud NAT	`apt-get` fails on private VMs	Try `curl`, check NAT config	Recreate NAT
4	Add a priority-500 deny on tcp:8080	web → app fails	Firewall logs, Connectivity Test (shows the rule)	Delete or lower precedence of the deny
5	Remove the PGA setting on app-subnet	`gcloud storage` from app-1 hangs	`gcloud compute networks subnets describe`	Enable PGA
6	Remove `roles/storage.objectViewer` from SA	Storage returns 403 not a timeout	Error type: **403 = IAM**, **timeout = network**	Re-grant role

```
# Example injections
gcloud compute instances remove-tags app-1 --zone=$ZONE --tags=ssh-iap
gcloud compute firewall-rules create deny-8080-test \
  --network=prod-vpc --direction=INGRESS --priority=500 --action=DENY \
  --rules=tcp:8080 --source-ranges=0.0.0.0/0 --enable-logging
gcloud compute networks subnets update app-subnet --region=$REGION \
  --no-enable-private-ip-google-access
```

> **Exam-grade takeaway:** *Timeout/hang* usually means **network** (firewall, route, NAT, PGA). *403/permission denied* usually means **IAM**.

Cleanup (run at the end of the day)

```
# MIG, template, health check
gcloud compute instance-groups managed delete web-mig --region=$REGION --quiet
gcloud compute instance-templates delete web-tpl-v1 web-tpl-v2 --region=$REGION --quiet
gcloud compute health-checks delete web-hc --quiet

# VMs and disks
gcloud compute instances delete web-1 app-1 db-1 custom-1 spot-1 --zone=$ZONE --quiet
gcloud compute instances delete eu-1 --zone=$ZONE2 --quiet
gcloud compute disks delete data-disk-1 data-disk-restored --zone=$ZONE --quiet

# Images, snapshots, schedules
gcloud compute images delete web-golden-v1 --quiet
gcloud compute snapshots delete data-disk-1-snap1 --quiet
gcloud compute resource-policies delete daily-snap --region=$REGION --quiet

# NAT, router, bucket
gcloud compute routers nats delete prod-nat --router=prod-router --region=$REGION --quiet
gcloud compute routers delete prod-router --region=$REGION --quiet
gcloud storage rm -r gs://$BUCKET

# Firewall rules
gcloud compute firewall-rules delete allow-iap-ssh allow-http-to-web \
```

```
  allow-web-to-app-tcp8080 allow-app-to-db-tcp5432 allow-health-checks \
  allow-icmp-internal deny-all-ingress-log --quiet
gcloud compute firewall-rules delete deny-8080-test --quiet 2>/dev/null

# Connectivity test
gcloud network-management connectivity-tests delete web-to-db --quiet

# Subnets and VPC
for s in web-subnet app-subnet db-subnet; do
  gcloud compute networks subnets delete $s --region=$REGION --quiet
done
gcloud compute networks subnets delete eu-subnet --region=$REGION2 --quiet
gcloud compute networks delete prod-vpc --quiet

# Service accounts
for tier in web app db; do
  gcloud iam service-accounts delete ${tier}-sa@${PROJECT_ID}.iam.gserviceaccount.com --quiet
done
```

> **Teaching point:** The deletion order shows dependencies. You cannot delete a subnet with VMs in it, nor a VPC with subnets, nor a router with NAT.

---

7. Quick Reference Cheat Sheets

7.1 VPC and networking at a glance

Topic	Remember
VPC scope	Global
Subnet scope	Regional (spans zones)
Auto mode range	`10.128.0.0/9`, `/20` per region; convertible to custom (one-way)
Reserved IPs per primary subnet	4
Expand / shrink subnet	Expand yes (no downtime), shrink never
Cross-region traffic inside a VPC	Works by default routing; firewall must allow
Private VM → Google APIs	Private Google Access (per subnet)
Private VM → internet	Cloud NAT (regional, outbound only, needs Cloud Router)
Admin access to private VM	IAP TCP forwarding (`35.235.240.0/20`), OS Login
Peering	Non-transitive, no overlapping ranges, firewall not shared
Dynamic routing mode	Regional (default) vs Global
Route selection	Longest prefix → lowest priority number

7.2 Firewall at a glance

Topic	Remember
Implied rules	Allow all egress, deny all ingress (priority 65535, cannot delete)
Default network rules	Priority 65534 (internal, SSH, RDP, ICMP)
Priority	0–65535, lower number wins; same priority → deny wins
Stateful	Yes; replies automatically allowed
Scope	Per VPC, enforced at the VM NIC
Targets	All instances, network tags, service accounts
Most secure target	Service accounts
Source tags	Only match VM-originated traffic inside the same VPC
Health check ranges	`130.211.0.0/22`, `35.191.0.0/16`
Log denied traffic	Add your own deny rule with logging (you cannot log implied rules)
Org-wide control	Hierarchical firewall policies (`allow`, `deny`, `goto_next`)
Troubleshooting	Connectivity Tests, firewall logs, flow logs, Firewall Insights

7.3 Compute Engine at a glance

Topic	Remember
Instance scope	Zonal
Regional HA	Regional MIG + load balancer
Change machine type	Stop VM first
Change service account	Stop VM first
Custom machine type	When predefined sizes don't fit; slightly higher unit price
Spot	Up to ~60–91% off, 30 s notice, no SLA, no live migration
CUD	1 or 3 years, steady baseline
SUD	Automatic, series-dependent, not E2
Reservation	Capacity guarantee, not a discount
Boot-time customization	Startup script (idempotent)
Fast and consistent	Golden image + image family
Instance template	Immutable; create a new one for changes
MIG autoscaling signals	CPU, LB capacity, Monitoring metrics, schedules
MIG autohealing	Health check + initial delay + firewall for probes
Disk resize	Up only, then grow file system
Local SSD	Fast, ephemeral, 375 GB units
Regional PD	Sync replication across 2 zones
Snapshots	Incremental, global, schedulable
SSH best practice	OS Login + IAP, no public IP
VM identity	Dedicated service account, `cloud-platform` scope + IAM
Labels vs tags	Labels = billing/organization; network tags = firewall/routes

7.4 Decision table: which compute and purchase option?

Requirement	Pick
Lift-and-shift legacy app with custom OS config	Compute Engine
Nightly batch, can be restarted	Spot VMs in a MIG
24/7 baseline for 3 years	CUD
Need exact 6 vCPU / 20 GB	Custom machine type
In-memory DB with 2 TB RAM	Memory-optimized (M-series)
GPU training	Accelerator-optimized (A/G series)
Compliance requires dedicated hardware or BYOL per-core licensing	Sole-tenant nodes
Encrypt data in use	Confidential VM
Prevent boot-level malware	Shielded VM

Requirement	Pick
Stop dev VMs outside work hours	Instance schedule

---

8. Exam-Style Scenarios

How to use this section: Participants answer individually first (3–4 minutes per question), then the instructor discusses the reasoning. The real exam has **multiple-choice and multiple-select** questions with long scenario text and several plausible answers. The goal is to pick the **best** answer under the stated constraints (cost, security, operational overhead, HA, minimal change).

Reading strategy: (1) Underline the **constraints** ("minimize cost", "least privilege", "no public IPs", "minimal operational overhead", "without downtime"). (2) Eliminate options that violate a constraint. (3) Between the remaining options, choose the **Google-recommended** managed approach.

These scenarios are original practice items written for this course, not real exam questions.

EXAM Scenario 1: Network mode selection

A company is moving to Google Cloud and plans to connect its on-premises data center (using `10.0.0.0/8` addresses) with Cloud VPN. It will also peer several VPCs in the future. A junior engineer proposes using the `default` network because it is already there. What should you recommend?

- A. Keep the default network; auto mode subnets handle growth automatically.
- B. Create a custom mode VPC and define subnets from a documented non-overlapping IP plan.
- C. Create several auto mode VPCs and peer them.
- D. Use a legacy network so there is only one IP range to manage.

Answer and explanation

B. Auto mode uses `10.128.0.0/9`, which overlaps the on-prem `10.0.0.0/8`, and all auto mode networks share the same range so they **cannot be peered** with each other. Legacy networks are deprecated. Custom mode with a planned IP scheme avoids overlaps and supports hybrid connectivity and peering.

Exam pattern: On-prem connectivity + peering + production → **custom mode**.

EXAM Scenario 2: Firewall priority resolution

A VPC has these ingress rules, all targeting instances tagged `web`:

Rule	Priority	Action	Source	Port
R1	1000	Allow	`0.0.0.0/0`	tcp:443
R2	800	Deny	`198.51.100.0/24`	tcp:443
R3	800	Allow	`198.51.100.10/32`	tcp:443

A client at `198.51.100.10` connects to a `web` VM on tcp:443. What happens?

- A. Allowed, because R3 is more specific.
- B. Allowed, because R1 is also an allow.
- C. Denied, because R2 and R3 have the same priority and deny wins.
- D. Denied, because the implied deny ingress rule applies.

Answer and explanation

C. Firewall evaluation uses **priority only** (not specificity). R2 and R3 tie at 800 and both match; when an allow and a deny of equal priority match, **deny wins**. R1 is lower precedence (1000), so it is never reached.

To allow that one client, give R3 a **lower number** (e.g., 700).

EXAM Scenario 3: Securing three tiers

A security team requires that **only the application tier VMs can connect to the database tier on tcp:5432**. Developers have the `compute.instanceAdmin.v1` role on the project and must not be able to bypass the rule by modifying their VMs. Which approach meets the requirement?

- A. Create a firewall rule with source tag `app` and target tag `db`.
- B. Create a firewall rule with source service account `app-sa` and target service account `db-sa`.
- C. Put the tiers in separate subnets and add a rule using subnet ranges as sources.
- D. Enable VPC Flow Logs and alert on unauthorized connections.

Answer and explanation

B. Network tags can be changed by anyone able to edit the VM (including developers with `instanceAdmin`), so tags are not tamper-resistant. Service accounts are controlled by IAM (changing a VM's SA needs `actAs` permission on that SA), so a **service account–based rule** satisfies the "cannot bypass" requirement. Subnet ranges (C) could work technically but would also allow *any* VM placed in that subnet, and developers can create VMs in subnets. Flow logs (D) detect, not prevent.

EXAM Scenario 4: Private VMs with no public IPs

A regulated workload runs on VMs with **no external IPs**. The VMs need to (1) download OS patches from public repositories, (2) read files from Cloud Storage, and (3) be reachable by administrators over SSH without a VPN or bastion host. Which combination is the best fit? **Select THREE.**

- A. Cloud NAT for outbound internet access.
- B. Assign ephemeral external IPs only during patching.
- C. Private Google Access on the subnet.

- D. IAP TCP forwarding with a firewall rule allowing `35.235.240.0/20` on tcp:22.
- E. Open tcp:22 to `0.0.0.0/0`.
- F. Create a VPC peering to Google's services network.

Answer and explanation

A, C, D. - **Cloud NAT** gives outbound-only internet. - **Private Google Access** reaches Cloud Storage and other Google APIs without external IPs. - **IAP TCP forwarding** lets admins SSH to private VMs, authenticated by IAM, with no bastion or VPN. B contradicts the "no external IP" posture; E is insecure; F is not how Google APIs are reached.

EXAM Scenario 5: Managed instance group keeps recreating VMs

A team deploys a regional MIG with autohealing enabled using an HTTP health check on port 80. After deployment, VMs are repeatedly deleted and recreated. Logging into one VM shows nginx is running and serving on port 80. What is the **most likely** cause?

- A. The autoscaler's cool-down period is too long.
- B. The firewall does not allow health check probe ranges to reach the VMs on port 80.
- C. The instance template uses an E2 machine type.
- D. The MIG is regional rather than zonal.

Answer and explanation

B. Health check probes originate from `130.211.0.0/22` and `35.191.0.0/16`. With the implied deny-ingress rule, the probes are blocked, so the service looks unhealthy even though it is running, and autohealing recreates it. Fix: add an ingress allow rule for those ranges on the health check port. (Also verify the **initial delay** is long enough for boot.)

EXAM Scenario 6: Cost: batch processing

A media company transcodes video nightly. Jobs are idempotent, checkpoint progress to Cloud Storage, and can be restarted if interrupted. The business wants the **lowest possible compute cost**. What should you do?

- A. Use standard N2 VMs with a 3-year CUD.
- B. Use a MIG of **Spot VMs** with a startup script that pulls jobs from a queue.
- C. Use a single large on-demand VM running 24/7.
- D. Use sole-tenant nodes.

Answer and explanation

B. The workload is **fault-tolerant, stateless-ish, and interruptible**, which is the Spot VM use case (large discount). A MIG recreates preempted instances. CUDs (A) fit steady 24/7 baselines, not nightly bursts. C wastes money. D is for isolation/licensing and costs more.

EXAM Scenario 7: Right-sizing odd requirements

An application requires **6 vCPUs and 20 GB of memory**. The closest predefined N2 types are `n2-standard-8` (32 GB) or `n2-standard-4` (16 GB, too small). The company wants to avoid paying for unused resources. What do you recommend?

- A. `n2-standard-8`
- B. `n2-highmem-4`
- C. A custom machine type with 6 vCPUs and 20 GB
- D. A shared-core machine type

Answer and explanation

C. Custom machine types are designed for this. (6 vCPUs is an even number, and 20 GB is within the allowed per-vCPU memory range for N2.) `n2-highmem-4` has 32 GB with only 4 vCPUs, which fails the CPU requirement. Shared-core cannot offer 6 vCPUs.

EXAM Scenario 8: Change a VM's capacity with minimal disruption

A production VM is running out of disk space on its data disk and also needs a larger machine type. Which statements are true? **Select TWO.**

- A. The disk can be resized larger while the VM is running; you then extend the file system.
- B. The machine type can be changed while the VM is running.
- C. The machine type change requires stopping the VM.
- D. The disk can be shrunk after resizing if the file system allows it.
- E. You must recreate the VM to enlarge the disk.

Answer and explanation

A and C. Disks can be enlarged online (not shrunk), but the OS needs a partition/file-system resize. Changing machine type needs a **stop → change → start**. Live machine type change is not supported.

EXAM Scenario 9: Stateful database and high availability

A self-managed PostgreSQL VM in `us-central1-a` stores data on a persistent disk. The business needs the database to survive a **zonal outage** with minimal data loss and acceptable manual or scripted failover. Which design best fits?

- A. Use Local SSD for speed and copy data nightly.
- B. Use a **regional persistent disk** (or regional Hyperdisk where available) and a standby VM in another zone, with failover by attaching the disk there.
- C. Use a zonal pd-ssd and take a snapshot weekly.
- D. Use a Spot VM with an instance schedule.

Answer and explanation

B. A **regional PD** replicates **synchronously** across two zones (RPO ≈ 0), enabling failover to a standby VM in the other zone. A and C have large RPO. D is not HA. *(On the real exam, if the question allows, a managed service such as Cloud SQL HA or AlloyDB would often be the preferred answer to reduce operational overhead. Read the constraints: if it says the app must stay on a VM, choose the regional disk design.)*

EXAM Scenario 10: Zero-downtime rollout

A web tier runs in a regional MIG behind a load balancer. You need to roll out a new application version using a new instance template **without reducing serving capacity at any time**, and you want to test the new version on a small portion of traffic first. What should you do?

- A. Delete the MIG and recreate it with the new template.
- B. Update the MIG with a **canary update** (new version at 10–20%) and `max-unavailable=0` with a `max-surge`, then promote to 100% if healthy.
- C. Edit the existing instance template in place.
- D. Create a second MIG and swap DNS manually.

Answer and explanation

B. Instance templates are **immutable** (C is impossible). Canary updates with surge and zero unavailable provide a gradual rollout without capacity loss. D works but is manual and higher overhead. A causes downtime.

EXAM Scenario 11: Org-wide guardrail

The central security team wants to guarantee that **no VM in any project in the organization accepts SSH from the internet (`0.0.0.0/0`)**, even if project teams create their own firewall rules. They still want teams to manage other rules locally. What is the best approach?

- A. Email teams a policy document and audit quarterly.
- B. Create a **hierarchical firewall policy** at the organization node with a deny rule for tcp:22 from `0.0.0.0/0` and a `goto_next` rule for everything else.
- C. Create the same VPC firewall rule in every project manually.
- D. Remove the `compute.instanceAdmin` role from all users.

Answer and explanation

B. Hierarchical firewall policies are evaluated **before** VPC firewall rules and cannot be overridden by project-level rules. `goto_next` delegates unmatched traffic to lower levels. A is not enforcement; C is not scalable or tamper-proof; D does not address firewall rules.

(You might also see `constraints/compute.vmExternalIpAccess` for restricting external IPs: a complementary control.)

EXAM Scenario 12: Hybrid connectivity and routes

A company has a Cloud VPN (HA VPN) with a Cloud Router in `us-central1`. VMs in `us-central1` can reach on-prem, but VMs in `europe-west1` in the same VPC cannot. What is the simplest fix?

- A. Create a second VPN in `europe-west1` and configure static routes.
- B. Change the VPC's **dynamic routing mode to Global**.
- C. Enable Private Google Access in `europe-west1`.
- D. Create a VPC peering between the two regions.

Answer and explanation

B. In **regional** dynamic routing mode, Cloud Router only advertises/learns routes for subnets in its own region. **Global** mode makes learned on-prem routes available to all regions in the VPC (and advertises all subnets to on-prem). Peering is between VPCs, not regions.

EXAM Scenario 13: Compliance and licensing

A company must run a **per-socket licensed** database on Compute Engine and needs **physical isolation** from other customers' VMs for compliance. Which feature fits?

- A. Shielded VM
- B. Confidential VM
- C. **Sole-tenant nodes**
- D. Spot VMs

Answer and explanation

C. Sole-tenant nodes provide dedicated physical servers (supports BYOL by socket/core and isolation requirements). Shielded VM protects boot integrity; Confidential VM encrypts data in use; Spot is a pricing model.

EXAM Scenario 14: Access control for administrators

Auditors require that SSH access to Linux VMs be tied to **individual Google identities**, be **revocable centrally**, support **2-step verification**, and generate **audit logs**. Which solution meets the requirement with the least operational overhead?

- A. Distribute shared SSH keys via project metadata.
- B. Enable **OS Login** (with `roles/compute.osLogin` / `osAdminLogin`) and require 2SV.
- C. Create local user accounts through startup scripts.
- D. Use one bastion with a shared password.

Answer and explanation

B. OS Login ties SSH access to IAM identities, supports 2-step verification, and logs to Cloud Audit Logs. Shared keys/passwords/local accounts do not provide individual accountability or central revocation.

EXAM Scenario 15: Troubleshooting

Developers report that a script on a private VM (no external IP, subnet with Private Google Access enabled, no NAT) fails: `gcloud storage cp gs://prod-data/file .` returns **HTTP 403 Forbidden**. Another script that runs `pip install` times out. What explains both symptoms? **Select TWO.**

- A. The 403 indicates the VM's service account lacks the needed IAM role on the bucket.
- B. The 403 indicates Private Google Access is disabled.
- C. The `pip install` timeout is expected because there is no NAT or external IP for internet access.
- D. The `pip install` timeout indicates the VM's disk is full.
- E. The `pip install` timeout means the firewall needs an ingress rule for tcp:443.

Answer and explanation

A and C. A 403 is an **authorization** response from the API, so the network path to Google APIs works (PGA is functioning). The `pip install` goes to the public internet (PyPI), which a private VM with no NAT cannot reach, so it **times out**. Fix: grant the right IAM role to the service account, and add Cloud NAT (or use a private package mirror such as Artifact Registry). Ingress rules (E) are irrelevant for outbound traffic.

EXAM Scenario 16: Image strategy

An operations team patches 200 VMs manually every month, which causes configuration drift and slow scale-out events (startup scripts install many packages). They want faster, more consistent VM boots and a repeatable patching process. What is the best approach?

- A. Keep the startup script and increase machine size.
- B. Build **golden custom images** in an **image family** via a pipeline, reference the family in the instance template, and roll out updates through a MIG **rolling update**.
- C. Take weekly snapshots of one VM and restore them.
- D. Use Spot VMs.

Answer and explanation

B. Baking software into images makes boots fast and consistent. Image families let templates always pick the newest image; MIG rolling updates replace instances safely (immutable infrastructure). C is for backups, not fleet consistency. A and D do not address drift.

EXAM Scenario 17: Case-study style (fictional retailer)

"*Cymbal Outfitters*" (fictional) runs an e-commerce site on VMs in a single zone. Requirements:

- Survive the loss of a zone without manual action.
- Handle seasonal 10× traffic spikes without pre-provisioning.
- Admins must not use public IPs for administration.
- Finance wants minimal cost for the always-on baseline and lower cost for burst capacity.
- Developers must not be able to alter network access by changing VM settings.

Which design best meets **all** requirements? **Select THREE.**

- A. A **regional MIG** with autoscaling behind a load balancer, using autohealing.
- B. A CUD for the baseline vCPU/memory plus Spot VMs (where application permits) for burst capacity, or standard VMs if bursts are not interruptible.
- C. A single large VM with a static external IP.
- D. Firewall rules using **service accounts** as targets, and **IAP/OS Login** for administration.

- E. Firewall rules using network tags, with SSH open to `0.0.0.0/0`.
- F. A zonal unmanaged instance group.

Answer and explanation

A, B, D. - **A:** Regional MIG across zones + autohealing + autoscaling covers zone failure and spikes. - **B:** CUD for the predictable baseline, cheaper burst capacity via Spot where tolerable. - **D:** Service-account-based firewall rules are tamper-resistant; IAP/OS Login removes the need for public IPs and gives audited access. C is a single point of failure; E is insecure and tag-based; F has no autoscaling/autohealing and is zonal.

EXAM Scenario 18: Subnet exhaustion

A subnet `10.10.1.0/24` in production is nearly out of IP addresses. The application team needs 600 more VMs in the same region and wants **no downtime** and **no VM re-IPing**. What is the best solution?

- A. Delete the subnet and create a new `/22`.
- B. **Expand** the subnet's primary range to a larger prefix (e.g., `/22`), provided it does not overlap other ranges.
- C. Create another subnet with the same range.
- D. Convert the VPC to auto mode.

Answer and explanation

B. Subnet expansion is **non-disruptive**. Existing VMs keep their IPs. Deleting requires removing all VMs; overlapping ranges are not allowed; and auto mode conversion is only one-way (auto → custom), not the reverse.

EXAM Scenario 19: Logging blocked traffic

A security analyst wants to see **all denied inbound connections** to a VPC for investigation. Rule logging is enabled on the allow rules. Where are the denied connections logged?

- A. Nowhere; implied deny rules cannot be logged, so add an explicit **deny-all ingress rule with logging enabled** at a low priority (e.g., 65534).
- B. In Cloud Audit Logs automatically.
- C. In the default firewall log, which always records implied rule hits.
- D. In VPC Flow Logs, which always include rejected packets.

Answer and explanation

A. Implied rules **cannot** be logged. Create your own deny rule (lower precedence than your allows) with logging on. Flow logs sample **allowed** traffic patterns and are not designed as a complete record of denied packets.

EXAM Scenario 20: Multi-step reasoning (long-form style)

"*Northwind Health*" (fictional) must deploy an internal claims-processing application on Compute Engine. The requirements are:

1. The application tier must be reachable only from the internal web tier on tcp:8443.
2. No VM may have an external IP.
3. Engineers need command-line access for troubleshooting, with each login attributable to an individual.
4. VMs must pull container images from Artifact Registry and write logs to Cloud Logging.
5. The hospital data must be encrypted using keys the company controls.
6. The solution must be resilient to a single zone failure.

Which of the following is the **best** overall design?

- A. One zonal MIG per tier; tag-based firewall rules; public IPs for admin SSH; Google-managed encryption.
- B. **Regional MIGs** per tier; **service-account–based** firewall rule web-sa → app-sa on tcp:8443; no external IPs; **Private Google Access** (for Artifact Registry/Logging); **IAP TCP forwarding + OS Login** for engineers; **CMEK** on boot and data disks.
- C. A single large VM per tier with Local SSD; Cloud NAT; shared SSH keys; CSEK keys on every start.
- D. Spot VMs in a regional MIG; allow all internal traffic with `10.0.0.0/8`; bastion host with public IP.

Answer and explanation

B. It addresses every requirement: 1. SA-to-SA rule on the specific port → least privilege tier isolation. 2. No external IPs; PGA and (if needed) NAT for outbound. 3. IAP + OS Login → individual, audited access. 4. PGA lets private VMs reach Artifact Registry and Cloud Logging APIs (and the VM SA needs the right IAM roles). 5. CMEK gives customer-controlled keys with managed operations. 6. Regional MIGs span zones. Other options violate multiple constraints (public IPs, shared keys, Spot for sensitive claims workloads, single points of failure).

Bonus: Fast-fire true/false round (instructor-led, 5 minutes)

#	Statement	T/F
1	A subnet in Google Cloud spans multiple regions.	**F** (regional)
2	Firewall rules in GCP are stateful.	**T**
3	The implied ingress rule allows all internal traffic.	**F** (it denies all ingress)
4	A VM's machine type can be changed while it is running.	**F**
5	Persistent disks can be shrunk.	**F**
6	Local SSD data survives VM deletion.	**F**
7	VPC peering is transitive.	**F**
8	Cloud NAT allows internet hosts to initiate connections to private VMs.	**F**
9	Instance templates can be edited.	**F** (immutable)
10	Labels are used in firewall rules.	**F** (network tags / SAs / secure tags)
11	Auto mode networks can be converted to custom mode.	**T**
12	Spot VMs are covered by a Compute Engine SLA.	**F**
13	Health check probe ranges must be allowed for LB/MIG health checks.	**T**
14	Subnet routes can be removed by deleting a route in the console.	**F** (system-generated; delete the subnet instead)

9. Common Exam Traps and Keyword Map

9.1 Keyword → likely answer

If the question says…	Think…
"Without public IP", "private VMs"	Private Google Access, Cloud NAT, IAP
"Least privilege", "tamper-proof", "developers must not bypass"	**Service account** targets, IAM, org policy
"Centralized control across all projects"	Hierarchical firewall policy, Organization Policy, Shared VPC
"Overlapping IP ranges"	Re-plan with custom mode; peering/VPN will not work with overlap
"Automatically recreate failed VMs"	MIG autohealing
"Handle unpredictable load"	MIG autoscaling
"Survive a zone failure"	**Regional** MIG, regional PD, multi-zone design
"Lowest cost, interruptible"	Spot VMs
"Steady 24/7 for years"	CUD
"Guaranteed capacity"	Reservation
"Exactly X vCPU and Y GB"	Custom machine type
"Software licensing per core/socket"	Sole-tenant nodes
"Encrypt data in use"	Confidential VM
"Verify boot integrity"	Shielded VM
"Customer-managed keys"	CMEK (Cloud KMS)
"Faster scale-out, consistent config"	Golden images + image families
"Track cost by team/project"	Labels + billing export
"Troubleshoot why A can't reach B"	Connectivity Tests, firewall/flow logs
"See which firewall rules are unused or shadowed"	Firewall Insights
"Individual accountability for SSH"	OS Login (+ IAP)
"On-prem must reach all regions through one VPN region"	Global dynamic routing

9.2 Top traps

1. **Zonal vs regional:** VMs are zonal; subnets are regional; VPCs are global. Many wrong answers confuse these.
2. **Tags are not secure.** If the question stresses security, pick service accounts.
3. **Priority direction:** *lower number = higher priority*.
4. **Same priority:** deny beats allow.
5. **Implied rules:** ingress denied, egress allowed. Egress filtering requires explicit rules.

6. **403 vs timeout:** IAM vs network.
7. **PGA ≠ internet.** PGA reaches Google APIs only. NAT reaches the internet.
8. **NAT is outbound only.**
9. **Peering is non-transitive; ranges must not overlap; firewalls are separate.**
10. **Health check ranges** must be allowed.
11. **Spot ≠ HA.** No SLA, 30 s notice, no live migration.
12. **Local SSD is ephemeral.**
13. **Templates are immutable.** Updating means a new template plus a rolling update.
14. **Default compute service account** with broad roles is a red flag. Use dedicated service accounts.
15. **Access scopes are legacy.** Use `cloud-platform` scope + IAM.
16. **The managed option usually wins** when the question stresses "minimal operational overhead" (e.g., Cloud SQL over self-managed DB on a VM, Cloud Run/GKE over hand-managed VMs). But if the scenario requires OS-level control or lift-and-shift, Compute Engine is right.
17. **Don't over-engineer.** If the requirement is "simple, low cost, quickly," choose the simplest design that satisfies the constraints.

---

10. Recap, Homework and Day 3 Preview

10.1 Recap: five things to remember from Day 2

1. **VPC is global; subnets are regional; VMs are zonal.** Always design with this scope model in mind.
2. **Firewall = priority + direction + target + source + ports.** Implied rules deny ingress and allow egress. Service accounts beat tags for security.
3. **Private by default:** no external IPs; use **IAP**, **Cloud NAT**, and **Private Google Access**.
4. **Compute Engine design = family → machine type → disk → provisioning model → image → identity → security**. Then add **templates + MIGs** for scale and HA.
5. **Cost levers:** rightsizing, custom types, Spot, CUDs, schedules, autoscaling.

10.2 Self-check questions (answer without notes)

1. Name three things that make a custom-mode VPC better than the default network for production.
2. How many IP addresses are reserved in each primary subnet range, and which ones?
3. A VM can reach another VM in a different region of the same VPC by default routing. What else must be true for the connection to succeed?
4. Give two scenarios where a service-account-based firewall rule is better than a tag-based one.
5. What are the two health check source ranges for Google Cloud load balancers and MIG autohealing?
6. What must you do (and in what order) to change a VM's machine type?
7. How does a regional persistent disk differ from a zonal one?
8. When would you choose a custom image over a startup script?
9. What changes between `max-surge` and `max-unavailable` in a rolling update?
10. Why do Spot VMs not fit a single-instance production database?

10.3 Homework (60–90 minutes)

1. **Rebuild** the Day 2 environment using **Terraform** or by exporting the equivalent `gcloud` commands from the Console ("Equivalent command line"). Terraform is covered more deeply on a later day, but trying early helps.
2. In a free-tier or sandbox project, create a **VPC with an auto-mode → custom-mode conversion** and note what changes.
3. Create a **hierarchical firewall policy** if you have access to an organization/folder (or read the docs for the syntax) and write down the evaluation order from memory.
4. Take **10 practice questions** on networking and Compute Engine from the official sample/practice questions and **write down why each wrong option is wrong**.
5. Skim the **Google Cloud Architecture Framework** sections on **Reliability** and **Cost optimization**, and relate them to today's Compute Engine design choices.

6. Look at the **current official PCA case studies** and list, for each, which networking and compute choices from today would apply (hybrid connectivity, private VMs, MIGs, Spot, and so on).

10.4 Day 3 preview (adjust to your agenda)

Topics that build directly on today:

- **Storage and databases:** Cloud Storage, Persistent Disk/Filestore choices, Cloud SQL, Spanner, Bigtable, Firestore, BigQuery selection logic.
- **Load balancing deep dive:** external/internal, global/regional, backends (MIGs), health checks, Cloud Armor, Cloud CDN.
- **Hybrid connectivity:** Cloud VPN, Interconnect, Cloud Router (BGP), Shared VPC and peering patterns.

10.5 Suggested resources

- The official **Professional Cloud Architect exam guide** and **sample questions** on the Google Cloud certification site.
- **Google Cloud Skills Boost** labs on VPC networking, firewall rules, Compute Engine, and managed instance groups.
- The Google Cloud **documentation pages** for VPC, firewall rules, firewall policies, Compute Engine machine families, instance templates, and managed instance groups.
- The **Google Cloud Architecture Framework** and **Cloud Architecture Center** reference architectures.

> *Reminder: this guide's numbers (quotas, discount percentages, machine series) can change. Always verify against current Google Cloud documentation before relying on them for the exam or for production designs.*

End of Day 2 training guide.